

Metadata Report

Base Version: 10.5.0.0

Modified Version: 10.5.1.0

CDO Definitions

[AddSPCAnnotation](#)

[AddSPCAnnotationHistory](#)

[BatchStartDetails](#)

[BOMMaterialListItem](#)

[ComponentIssue](#)

[ComponentIssueR2](#)

[ComputationDetail](#)

[ContainerCompletedTask](#)

[ContainerMaterialListItem](#)

[CurrentContainerStatus](#)

[CurrentStatusStartDetails](#)

[DataPointDetails](#)

[DataPointSummary](#)

[ElectronicProcedure](#)

[ElectronicProcedureChanges](#)

[EProcedureDetail](#)

[ERPMaterialListItem](#)

[ExecuteProductionProcess](#)

[ExecuteTask](#)

[InstructionTypeEnum](#)

[LogDetailsForObject](#)

[LogEntryType](#)

[LogEntryTypeChanges](#)

[LogEntryTypeGroup](#)

[LogEntryTypeGroupChanges](#)

[LogEntryTypeGroupMaint](#)

[LogEntryTypeMaint](#)

[LogObjectTypeEnum](#)

[LogTxn](#)

[MasterRecipe](#)

[MasterRecipeChanges](#)

[MasterRecipeDetail](#)

[MaterialListItem](#)

[MaterialListItemChanges](#)

[MfgOrderMaterialListItem](#)

[MfgOrderProcedure](#)

[MfgOrderProcedureChanges](#)

[MfgOrderTaskList](#)

[MfgOrderTaskListChanges](#)

[MoveStd](#)

[MoveTxn](#)

[ObjectDataPoint](#)

[ObjectDataPointChanges](#)

[OnlineQuerySetup](#)

[Open](#)

[OperatorLog](#)

[OperatorLogChanges](#)
[OperatorLogInfo](#)
[OperatorLogInquiry](#)
[OperatorLogMaint](#)
[Part](#)
[PartRequestAcknowledge](#)
[PartSetup](#)
[PartTxn](#)
[ProcessItem](#)
[ProcessItemChanges](#)
[ProcessList](#)
[ProcessListChanges](#)
[ProcessSequenceValidation](#)
[Product](#)
[ProductChanges](#)
[ProductionProcess](#)
[ProductionProcessChanges](#)
[ProductMaterialListItem](#)
[RecipeItem](#)
[RecipeList](#)
[RecipeListChanges](#)
[RecipeMaterialListItem](#)
[RecordLogEntry](#)
[RecordLogEntryDetail](#)
[RecordLogEntryHistory](#)
[ResourceBOMMaterialListItem](#)
[Service](#)
[ShopFloor](#)
[SPCChartCustomInfo](#)
[SPCChartCustomInfoChanges](#)
[SPCChartDef](#)
[SPCChartDefChanges](#)
[SPCChartType](#)
[SPCChartTypeChanges](#)
[SPCChartVisualParams](#)
[SPCChartVisualParamsChanges](#)
[SPCQuery](#)
[SPCQueryChanges](#)
[StartDetails](#)
[TaskItem](#)
[TaskItemChanges](#)
[TaskList](#)
[TaskListChanges](#)
[TaskQueries](#)
[TaskQueriesChanges](#)
[TransactionItem](#)
[UnitStartDetails](#)
[UserQueryParameter](#)
[UserQueryParameterChanges](#)

CLF Definitions

[ApplyTaskMaterialList](#)
[ChangeQty_FinishModifyingEntity](#)
[CILight_Load_PhantomList](#)

[CILight_MaterialList_SelVal](#)
[CLF3606](#)
[CurrentDay_OGV](#)
[CurrentMonth_OGV](#)
[CurrentYear_OGV](#)
[DataPointDetails_EvaluateLimits](#)
[DataPointSummary_Validate](#)
[DataPointSummary_ValidateResource](#)
[ExecuteProdProcess_AddCompletedTask](#)
[ExecuteProdProcess_FinishModifyingEntity](#)
[ExecuteProdProcess_LoadProcessListDetails](#)
[MaxIterationsExpression_OGV](#)
[MinIterationsExpression_OGV](#)
[MoveStd_CheckEProcedureComplete](#)
[MoveStd_ValidateEProcedure](#)
[MoveTxn_CheckEProcedureComplete](#)
[MoveTxn_ClearCompletedTasks](#)
[OnlineQuerySetupParams_SelVal](#)
[Open_ProcessElectronicProcedures](#)
[OperatorLog_SelValEx_Container](#)
[OperatorLog_SelValEx_ObjInstanceId](#)
[OperatorLogInquiry_LogDetails_OGV](#)
[OperatorLogInquiry_LogDetailsForObject_OGV](#)
[OpLog_GetAttachedDocuments_OGV](#)
[PartRequestAcknowledge_SetRequestType2](#)
[PartTxn_EquipmentSetup](#)
[PopulateEProcedureVariantDetails](#)
[PopulateVariantTasks](#)
[ProcessSeqValidation_AreBatchTasksPending](#)
[ProcessSeqValidation_GetCompletedTasks](#)
[ProcessSeqValidation_ValidatePrerequisiteTaskList](#)
[ProcessSeqValidation_ValidatePrerequisiteTasks](#)
[ProcessSeqValidation_ValidateTaskListSequence](#)
[ProcessSeqValidation_ValidateTaskSequence](#)
[RecordLogEntry_ObjectTypeName_OGV](#)
[RecordLogEntry_PreValidate_User](#)
[RecordLogEntry_StartModifyingEntity](#)
[SelectionValuesEx_AllowedLogEntryTypes](#)
[SelectionValuesEx_ExecuteTask_Task](#)
[SelectionValuesEx_ExecuteTask_TaskList](#)
[SelVal_AddSPCAnnotation_Annotations](#)
[SelVal_GetOperatorLogEntries](#)
[SelVal_RDOAndNDOObjects](#)
[Task_ProcessUserQueryDynamicValues](#)
[ValidateVariantExpression](#)

DB Table Definitions

[A_SPCChartVisualParams](#)
[AddSPCAnnotationHistory](#)
[ContainerCompletedTask](#)
[DataPoint](#)
[ElectronicProcedure](#)
[EProcedureDetail](#)
[LogEntryType](#)

- [LogEntryTypeGroup](#)
- [LogEntryTypeGroupDefaultForObj](#)
- [LogEntryTypeGroupEntries](#)
- [LogEntryTypeGroupGroups](#)
- [MasterRecipe](#)
- [MasterRecipeDetail](#)
- [MfgOrderProcedure](#)
- [MfgOrderTaskList](#)
- [OperatorLog](#)
- [RecipeItem](#)
- [RecipeList](#)
- [RecordLogEntryHistory](#)
- [RecordLogEntryHistoryHistoryDe](#)
- [SPCChartCustomInfo](#)
- [SPCChartDef](#)
- [SPCChartQueryParams](#)
- [SPCChartVisualParams](#)
- [TaskItem](#)
- [TaskItemTaskQueries](#)
- [TaskList](#)
- [TaskQueries](#)
- [UserQuery](#)
- [UserQueryParameter](#)

Query Definitions

- [ComponentReplaceInquiry_ContainerSelVal](#)
- [GetCDONameFromInstanceId](#)
- [GetOperatorLogsForObject](#)
- [GetSPCAnnotations](#)
- [MBO_GetDHRHistoryEvents](#)
- [OperatorLog_GetCDOName](#)
- [OperatorLogInquiry_GetLogEntries](#)
- [SelectionValuesEx_OpLog_EntryHistory](#)
- [SelectionValuesEx_OpLog_RDOAndNDOObjects](#)
- [SelectionValuesEx_TaskList](#)
- [SelectionValuesEx_TasksForEProc](#)
- [SelectionValuesEx_TasksForTasklist](#)
- [SelectionValuesEx_TasksForWorkstation](#)
- [SelectResourceCDOs](#)
- [SelectResourceGroupCDOs](#)

Label Categories

- [CSICDName](#)
- [CSIField](#)
- [CSIReject](#)
- [CSIWebClient](#)
- [OtherReject](#)

CDO Definitions

AddSPCAnnotation - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Annotations						
ExternalDataId	Data ID		External Data ID	String,Comments		

ExternalTable	Secondary Data ID		External Table	String.Comments		
SPCChartDef	SPC Chart Def		Defines SPC Charting information for a variable/chart type combination. Includes designation for rules file, macro, display options, OCAPs and 10 user params to use during chart generation, display and OCAP processing	Object. SPCChartDef		

Field: Annotations Events

SelectValuesEx: SelVal_AddSPCAnnotation_Annotations

CLF: SelVal_AddSPCAnnotation_Annotations

Function	Parameter	Value
----------	-----------	-------

Maps**AddSPCAnnotationToAddSPCAnnotationHistory**

Source Field	Target Field
ExternalDataId	ExternalDataId
ExternalTable	ExternalTable
SPCChartDef	SPCChartDef

AddSPCAnnotationHistory - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ExternalDataId		ExternalDataId	Data ID from external data source	String.Comments		
ExternalTable	Secondary Data ID	SecondaryDataId	Table name from external data source	String.Comments		
SPCChartDef	SPC Chart Def	SPCChartDefId	Defines SPC Charting information for a variable/chart type combination. Includes designation for rules file, macro, display options, OCAPs and 10 user params to use during chart generation, display and OCAP processing	Object. SPCChartDef		

BatchStartDetails - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Level						
Owner						
StartReason						

BOMMaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed.Qty		CVE: This _Value;QtyRequired

ComponentIssue - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ApplyStepFilter	Apply Step Filter		Generic Boolean	Boolean.[Boolean]		
AutoLoadPhantomMaterials	Automatically load Phantom Materials		Controls whether Phantom Product BOMs will be automatically loaded for the ServiceDetails Selection Values	Boolean.[Boolean]		
PhantomIssueDetailsTmp	Material List Item		Represents, depending on context, an item or product that is required to complete a given manufacturing or assembly operation. The necessary quantities or proportions of the item are specified as are the appropriate units of measure.	Object. MaterialListItem		
PhantomQtyIssuedListTmp	Qty		Material quantity (individual)	Fixed.Qty		
ServiceDetailsTmp				.MaterialListItemIssueDetails		

Methods**ApplyTaskMaterialList**

CLF: ApplyTaskMaterialList

Function	Parameter	Value
----------	-----------	-------

ComponentIssueR2 - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ServiceDetails						

Field: ServiceDetails Events

SelectValuesEx: CILight_MaterialList_SelVal

CLF: CILight_MaterialList_SelVal

Feature Membership: Base

Function	Parameter	
call	CDO	
	Event	SetMaterialListItemSettings
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
ValidateContainerOpen	Container	Container
call	CDO	
	Event	ValidateHoldStatus
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	zMaterialListItemSettings
{		
InitQueryParametersEx	ParamNames	"Container", "PhantomBillId"
	ParamValues	Container, null
	QueryParameters	CLF::QueryParams
	QueryName	zMaterialListItemSettings.QueryName
ExecuteQueryEx	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::MaterialList
if	Condition	ApplyStepFilter OR AutoLoadPhantomMaterials OR (CalledByTransactionTask && GetListCount(CalledByTransac
{		
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"MaterialListItemId"
	IsList	true
	FieldType	
	FieldName	ServiceDetailsTmp
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"QtyIssued"
	IsList	true
	FieldType	
	FieldName	QtyIssuedListTmp
	UseFieldDataType	true
setfieldex	From	0
	To	CLF::i
setfieldex	From	false
	To	CLF::AnyPhantoms
foreach	Var	CLF::Item
	List	QtyIssuedListTmp
{		
setfieldex	From	CLF::Item
	To	ServiceDetailsTmp[CLF::i].QtyIssued
if	Condition	ServiceDetailsTmp[CLF::i].Product.IsPhantom
setfieldex	From	true
	To	CLF::AnyPhantoms
setfieldex	From	CLF::i + 1
	To	CLF::i
}		
if	Condition	AutoLoadPhantomMaterials && CLF::AnyPhantoms
{		
setfieldex	From	0

	To	CLF::i
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count
while	Condition	CLF::i < CLF::Count
{		
setfieldex	From	ServiceDetailsTmp[CLF::i]
	To	CLF::Item
if	Condition	CLF::Item.Product.IsPhantom
{		
setfieldex	From	CLF::Item.Product.ResolvedBOM
	To	CLF::BOM
DeleteListItem	ListField	ServiceDetailsTmp
	ItemValue	CLF::Item
	ItemIndex	
if	Condition	CLF::BOM
{		
setfieldex	From	StringFromObject(CLF::BOM)
	To	PhantomBillId
call	CDO	
	Event	CLight_Load_PhantomList
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
MergeLists	Lists	ServiceDetailsTmp,PhantomIssueDetailsTmp
	Result	ServiceDetailsTmp
}		
}		
else		
setfieldex	From	CLF::i + 1
	To	CLF::i
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count
}		
}		
call	CDO	
	Event	ApplyTaskMaterialList
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	ApplyStepFilter && IsERPMaterialList
ComponentIssue_FilterMaterialList	MaterialList	ServiceDetailsTmp
	StepIssuePoint	Container.WorkflowStep.RouteStep
	ResultMaterialList	ServiceDetailsTmp
elseif	Condition	ApplyStepFilter
ComponentIssue_FilterMaterialList	MaterialList	ServiceDetailsTmp
	StepIssuePoint	Container.Spec
	ResultMaterialList	ServiceDetailsTmp
if	Condition	IsERPMaterialList
ConvertCDOListToResultset	List	ServiceDetailsTmp
	Fields	Name,InstanceId,Product.Name,Product.Revision,Description,QtyRequired,QtyIssued,UOM?UOM.Name:UOM,A RouteStep.Name:RouteStep,ActualQtyRequired,ObjectTypeName
	ColNames	"MaterialListItem","MaterialListItemId","ComponentProduct","ProductRevision","ComponentDescription","QtyR
	ColLabels	
	ResultsetStartRow	0
	ResultsetSizeLimit	-1
	FilterFieldName	
	FilterParams	
	Result	CLF::MaterialList
else		
ConvertCDOListToResultset	List	ServiceDetailsTmp
	Fields	Name,InstanceId,Product.Name,Product.Revision,Description,QtyRequired,QtyIssued,UOM?UOM.Name:UOM,A Spec.Name:Spec,ActualQtyRequired,ObjectTypeName
	ColNames	"MaterialListItem","MaterialListItemId","ComponentProduct","ProductRevision","ComponentDescription","QtyR

	ColLabels	
	ResultSetStartRow	0
	ResultSetSizeLimit	-1
	FilterFieldName	
	FilterParams	
	Result	CLF::MaterialList
}		
return	Value	CLF::MaterialList
}		
return	Value	null

Methods**CILight_Load_PhantomList**

CLF: CILight_Load_PhantomList
 Feature Membership: Base

Function	Parameter	Value
ClearList	ListCDO	CLF::__CDOID
	ListField	PhantomIssueDetailsTmp
if	Condition	PhantomBillId
{		
call	CDO	
	Event	SetMaterialListItemSettings
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	zMaterialListItemSettings.PhantomQueryName != null
{		
InitQueryParametersEx	ParamNames	"PhantomBillId", "Container"
	ParamValues	PhantomBillId, Container
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	zMaterialListItemSettings.PhantomQueryName
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultSetSizeLimit	-1
	ResultSetStartRow	1
	ResultSet	CLF::MaterialList
ConvertResultSetToListOrScalar	ResultSet	CLF::MaterialList
	ColName	"MaterialListItemId"
	IsList	true
	FieldType	
	FieldName	PhantomIssueDetailsTmp
	UseFieldDataType	true
ConvertResultSetToListOrScalar	ResultSet	CLF::MaterialList
	ColName	"QtyIssued"
	IsList	true
	FieldType	
	FieldName	PhantomQtyIssuedListTmp
	UseFieldDataType	true
setfieldex	From	0
	To	CLF::i
foreach	Var	CLF::Item
	List	PhantomQtyIssuedListTmp
{		
setfieldex	From	CLF::Item
	To	PhantomIssueDetailsTmp[CLF::i].QtyIssued
setfieldex	From	CLF::i + 1
	To	CLF::i
}		
}		
}		

ComputationDetail - Import**Methods****EvaluateUpperLowerLimits**

CLF: DataPointDetails_EvaluateUpperLowerLimits

Function	Parameter	Value
----------	-----------	-------

ContainerCompletedTask - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ElectronicProcedure	Electronic Procedure	ElectronicProcedureId	Provides the framework for executing a series of Tasks at the Spec. An Electronic Procedure defines a set of instructions (a checklisst for performing a set of tasks), and is compose of one ore more TaskLists.	Object. ElectronicProcedure		
WorkflowStep	Step	WorkflowStepId	A step is an individual tracking point within a workflow. The list of steps is the primary data structure within a workflow. Paths control the allowable movements between steps. Each Step contains zero or more Paths, which link to another Step. One Path at each Step is identified as the default Path. A Step normally represents an individual processing point in a workflow, where it references a Spec which in turn describes the work that is to be performed. A Step can reference another workflow, in which case the step represents a series of processing points.	Object. Step		

ContainerMaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed.Qty		CVE: This_Value;QtyRequired

CurrentContainerStatus - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
StatusName						

Field: StatusName Events

OnGetValue: GetEnumLabel

CLF: GetEnumLabel

Function	Parameter	Value
----------	-----------	-------

CurrentStatusStartDetails - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Workflow						

DataPointDetails - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ValidateResource	Validate Resource		Generic Boolean	Boolean.[Boolean]		

Methods

EvaluateUpperLowerLimits

CLF: DataPointDetails_EvaluateUpperLowerLimits

Function	Parameter	Value
----------	-----------	-------

DataPointSummary - Import

Methods

Validate

CLF: DataPointSummary_Validate

Feature Membership: Base

Function	Parameter	Value
SortList	ListFieldExpr	Parent.DataCollectionDef.DataPoints

	Field1	RowPosition
	IsAscending	true
	Field2	ColumnPosition
	IsAscending2	true
	IgnoreNullObjectReferences	false
	Result	CLF::SortedDataPoints
setfieldex	From	false
	To	CLF::ScaleEnabled
foreach	Var	CLF::DataPoint
	List	CLF::SortedDataPoints
{		
if	Condition	CLF::DataPoint.IsRequired
{		
findlistitem	List	DataPointDetails
	ListKey	DataPoint
	KeyValue	CLF::DataPoint
	Result	CLF::FoundDataPoint
	ListIndex	
ValidateNotEqual	LeftValue	CLF::FoundDataPoint
	RightValue	null
	ErrorID	LabelId::MissingDataPoint
	ErrorParmName1	Name
	ErrorParmValue1	CLF::DataPoint.Name
	ErrorParmName2	
	ErrorParmValue2	
	ErrorParmName3	
	ErrorParmValue3	
	ErrorParmName4	
	ErrorParmValue4	
}		
if	Condition	CLF::ScaleEnabled != true && String(CLF::DataPoint.DecimalScale) != ""
{		
setfieldex	From	true
	To	CLF::ScaleEnabled
}		
}		
if	Condition	CLF::ScaleEnabled
{		
setfieldex	From	null
	To	Transaction::FailedDataPoints
foreach	Var	CLF::Detail
	List	DataPointDetails
{		
call	CDO	
	Event	ValidateDecimalScale
	Name1	Detail
	Value1	CLF::Detail
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
}		
if	Condition	Transaction::FailedDataPoints != null
throw	ErrorId	LabelId::Lbl_DecimalScaleExtendValidation
	ParamNames	"Name"
	ParamValues	Transaction::FailedDataPoints
}		
call	CDO	
	Event	DataPointSummary_ValidateResource
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	

DataPointSummary_ValidateResource

CLF: DataPointSummary_ValidateResource
 Feature Membership: Base

Function	Parameter	Value
foreach	Var	CLF::Detail
	List	DataPointDetails
{		
GetCDOType	CDO	CLF::Detail.DataPoint
	Type	CLF::CDOId
	TypeName	CLF::CDOType
if	Condition	CLF::CDOType == "ObjectDataPoint"
{		
if	Condition	CLF::Detail.ValidateResource && (CLF::Detail.ObjectValueCDOType == "Resource" OR CLF::Detail.ObjectValueCDOType == "Carrier" OR CLF::Detail.ObjectValueCDOType == "Part" OR CLF::Detail.ObjectValueCDOType == "Scale" OR CLF::Detail.ObjectValueCDOType == "Tool")
{		
if	Condition	CLF::Detail.NDOValue.Availability != 1
throw	ErrorId	LabelId::ResourceNotAvailable
	ParamNames	"Name"
	ParamValues	CLF::Detail.NDOValue.DisplayName
call	CDO	CLF::Detail.NDOValue
	Event	ValidateResourceMaintenance
	Name1	ServiceQty
	Value1	0
	Name2	ServiceQty2
	Value2	0
	Name3	ThrowErrors
	Value3	true
	Name4	
	Value4	
	Name5	
	Value5	
}		
}		
}		

ElectronicProcedure - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
EProcedureVariantDetails	Production Process Detail		Encapsulates various types of TaskList that are to be assigned to a ProductionProcess in a particular sequence after variant expressions are evaluated	Object. ProductionProcessDetail		
KeepCompletedTasks	Keep Completed Tasks on Move	KeepCompletedTasks	When true will prevent deleting ContainerCompletedTasks data for the container at the current step	Boolean.[Boolean]		
KeepCompletedTasksOnOpen	Keep Completed Tasks on Open	KeepCompletedTasksOnOpen	When true will not delete any completed task history	Boolean.[Boolean]		

EProcedureDetail - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ExportImportKey			A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.			

ERPMaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed.Qty		CVE: This_Value;QtyRequired

ExecuteProductionProcess - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ComponentIssue	Component Issue		Component Issue is used to select and (optionally) track which raw material lots or subassemblies were used at a particular manufacturing step. The correct components to issue and the proper quantities are indicated by the Material List CDO collection of the Bill of Material. Another variation of	Object. ComponentIssueR2		

			component issue is an issue from "floor-stock". In this case, material is decremented against a location or bin not a container. InSite should support both material issues as defined in a Bill structure (Bill Control condition) as well as material issues that are not specified in a predefined BOM (Non-Bill Control condition).		
IssueActualDetails	Issue Actual Detail		This field can be used to send in a single, flat issued component to the service. Prior to any other processing for the service, the appropriate requirement will be retrieved, and an issue detail/issue actual will be built from the requirement/issued component in the normal format of the service. It is expected that this field would be useful for the case that only one issued component is passed in, or where the service is running from an interface that doesn't provide state management for the requirements.	Object. IssueActualDetail	
OnlineQuerySetupParams	Online Query Setup Params		Parameters for OnlineQuerySetup	Object. OnlineQuerySetupParams	
OnlineQuerySetupParamsTmp	Online Query Setup Params		Parameters for OnlineQuerySetup	Object. OnlineQuerySetupParams	
VariantTaskLists	Process List		Non-persistent field used to resolve the variant task lists	Object. ProcessList	
VariantTasks	Variant Tasks		Proxy field to resolve tasks from the variant expressions	Object. ProcessItem	

Field: OnlineQuerySetupParams Events

SelectValuesEx: OnlineQuerySetupParams_SelVal

CLF: OnlineQuerySetupParams_SelVal

Feature Membership: Base

Function	Parameter	Value
ClearList	ListCDO	CLF:: CDOID
	ListField	OnlineQuerySetupParamsTmp
if	Condition	Task
{		
call	CDO	
	Event	Task_ProcessUserQueryDynamicValues
	Name1	Task
	Value1	Task
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
}		
elseif	Condition	TaskList
{		
foreach	Var	CLF::Task
	List	TaskList.Tasks
{		
call	CDO	
	Event	Task_ProcessUserQueryDynamicValues
	Name1	Task
	Value1	CLF::Task
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
}		
}		
ConvertCDOListToResultset	List	OnlineQuerySetupParamsTmp
	Fields	TaskName,QueryName,Name,ResolvedValue,DynamicValue
	ColNames	"TaskName","QueryName","ParameterName","ResolvedValue","DynamicValue"
	ColLabels	
	ResultsetStartRow	CLF:: __ResultsetStartRow
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	FilterFieldName	
	FilterParams	

	Result	Transaction::__Return
--	--------	-----------------------

Methods**LoadProcessListDetails**

CLF: ExecuteProdProcess_LoadProcessListDetails

Function	Parameter	Value
----------	-----------	-------

FinishModifyingEntity

CLF: ExecuteProdProcess_FinishModifyingEntity

Feature Membership: Base

Function	Parameter	Value
if	Condition	GetListCount(IssueActualDetails) > 0
{		
CreateCDO	CDOTypeName	
	UseFieldType	true
	OverwriteExistingCDO	true
	NewCDO	ComponentIssue
mapex	MapFrom	CLF::__CDOID
	MapTo	ComponentIssue
	MapID	ContainerTxnToContainerTxn
if	Condition	IsFieldDefined("Resource", ComponentIssue)
setfieldex	From	Resource
	To	ComponentIssue.Resource
foreach	Var	CLF::Item
	List	IssueActualDetails
{		
Clone	From	CLF::Item
	To	CLF::ClonedItem
setfieldex	From	ComponentIssue
	To	CLF::ClonedItem.Parent
AddListItem	ListField	ComponentIssue.IssueActualDetails
	Item	CLF::ClonedItem
	Result	
}		
call	CDO	
	Event	ExecuteService
	Name1	SubService
	Value1	ComponentIssue
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
}		
//	Line1	If data points were collected and any
	Line2	had a limit violation without a limit override,
	Line3	then this service does not pass
	Line4	
if	Condition	ParametricData
if	Condition	IsObjectType(ParametricData, "DataPointSummary", false)
{		
foreach	Var	CLF::DataPointDetail
	List	ParametricData.DataPointDetails
{		
if	Condition	CLF::DataPointDetail.IsLimitViolation and not(CLF::DataPointDetail.IsLimitOverride)
{		
setfieldex	From	false
	To	Pass
break		
}		
}		
}		
if	Condition	Workstation and (Workstation.ClearanceLevel == Transaction::__Const.ClearanceLevel.Container)
call	CDO	
	Event	ClearStationContainerClearance
	Name1	WorkStation
	Value1	Workstation
	Name2	
	Value2	
	Name3	
	Value3	

	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	(WorkStation) And Workstation.ClearanceLevel == Transaction::_Const.ClearanceLevel.MfgOrder
call	CDO	
	Event	ClearStationMfgOrderClearance
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
//	Line1	Copy the information to the completed
	Line2	Task.
	Line3	
	Line4	
call	CDO	
	Event	UpdateCompletedTask
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	

Task_ProcessUserQueryDynamicValues

CLF: Task_ProcessUserQueryDynamicValues

Feature Membership: Base

Function	Parameter	Value
foreach	Var	CLF::Item
	List	CLF::Task.TaskQueries
{		
foreach	Var	CLF::Param
	List	CLF::Item.OnlineQuerySetup.UserQueryParameters
{		
CreateListItem	ListCDO	CLF::_CDOID
	ListField	OnlineQuerySetupParamsTmp
	ItemType	
	ItemValue	CLF::NewParam
	ItemIndex	
setfieldex	From	CLF::Task.Name
	To	CLF::NewParam.TaskName
setfieldex	From	CLF::Item.OnlineQuerySetup.Name
	To	CLF::NewParam.QueryName
setfieldex	From	CLF::Param.Name
	To	CLF::NewParam.Name
setfieldex	From	CLF::Param.DynamicValue
	To	CLF::NewParam.DynamicValue
EvalParamIndirect	ExprLocation	CLF::Param.DynamicValue
	Context	GetCurrentService()
	IgnoreNullObjectReferences	true
	Result	CLF::Value
if	Condition	NOT(CLF::Value)
setfieldex	From	CLF::Param.DefaultValue
	To	CLF::Value
setfieldex	From	CLF::Value
	To	CLF::NewParam.ResolvedValue
}		
}		

AddCompletedTask

CLF: ExecuteProdProcess_AddCompletedTask

Feature Membership: Base

Function	Parameter	Value
setfieldex	From	null
	To	Transaction::MatchingCompletedTask
setfieldex	From	Container.WorkflowStep

	To	CLF::Step
foreach	Var	CLF::Item
	List	Container.CurrentStatus.ContainerCompletedTasks
{		
setfieldex	From	CLF::Item.WorkflowStep
	To	CLF::TaskStep
if	Condition	CLF::Item.Task == Task && CLF::TaskStep == CLF::Step
setfieldex	From	CLF::Item
	To	Transaction::MatchingCompletedTask
}		
if	Condition	Transaction::MatchingCompletedTask == Null
{		
CreateListItem	ListCDO	Container.CurrentStatus
	ListField	ContainerCompletedTasks
	ItemType	
	ItemValue	Transaction::MatchingCompletedTask
	ItemIndex	
setfieldex	From	Container.CurrentStatus
	To	Transaction::MatchingCompletedTask.Parent
setfieldex	From	0
	To	Transaction::MatchingCompletedTask.NbrTimesProcessed
setfieldex	From	Task
	To	Transaction::MatchingCompletedTask.Task
setfieldex	From	Container.WorkflowStep
	To	Transaction::MatchingCompletedTask.WorkflowStep
setfieldex	From	ElectronicProcedure
	To	Transaction::MatchingCompletedTask.ElectronicProcedure
}		

ExecuteTask - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
EProcedureExecutionModeList	Execution Mode		Enumeration for Execution Mode. 1 = Sequential 2 = Non-Sequential	Integer.ExecutionMode		
IsComputationList	Is Computation List		Generic Boolean	Boolean.[Boolean]		
NbrTimesProcessedList	Number of Times Processed		Identifies how many times the Task has been processed within the context of the current step.	Integer.[Integer]		
Task						
TaskList						
TaskListSequenceList	Sequence		Relative sequence number	Integer.Sequence		
TaskStatusList	Task Status		0 = Failed, 1 = Passed, 2 = Not Executed	Integer.[Integer]		
UIVirtualPageNameList	File Name		FileName - used for fields that specify files.	String.FileName		
VariantTaskLists	Process List		Non-persistent field used to resolve the variant task lists	Object. TaskList		
VariantTasks	Variant Tasks		Proxy field to resolve tasks from the variant expressions	Object. TaskItem		

Field: Task Events

SelectValuesEx: SelectionValuesEx_ExecuteTask_Task

CLF: SelectionValuesEx_ExecuteTask_Task

Feature Membership: Base

Function	Parameter	Value
setfieldex	From	null
	To	CLF::QueryName
if	Condition	TaskList
{		
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"TaskList"
	ParamValue2	TaskList
	ParamName3	"ElectronicProcedure"
	ParamValue3	ElectronicProcedure
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForTaskList"
	To	CLF::QueryName
}		
elseif	Condition	Workstation
{		

InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"Workstation"
	ParamValue2	Workstation
	ParamName3	"ElectronicProcedure"
	ParamValue3	ElectronicProcedure
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForWorkstation"
	To	CLF::QueryName
}		
elseif	Condition	ElectronicProcedure
{		
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"ElectronicProcedure"
	ParamValue2	ElectronicProcedure
	ParamName3	
	ParamValue3	
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForEProc"
	To	CLF::QueryName
}		
if	Condition	CLF::QueryName
{		
ExecuteQueryEx	QueryName	CLF::QueryName
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	CLF::__ResultsetSizeLimit
	ResultsetStartRow	CLF::__ResultsetStartRow
	Resultset	CLF::ResultSet
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"TaskId"
	IsList	true
	FieldType	
	FieldName	VariantTasks
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"NbrTimesProcessed"
	IsList	true
	FieldType	
	FieldName	NbrTimesProcessedList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"EProcedureExecutionMode"
	IsList	true
	FieldType	
	FieldName	EProcedureExecutionModelList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"UIVirtualPageName"
	IsList	true
	FieldType	
	FieldName	UIVirtualPageNameList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"IsComputation"
	IsList	true
	FieldType	
	FieldName	IsComputationList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"TaskStatus"
	IsList	true
	FieldType	
	FieldName	TaskStatusList

	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"TaskListSequence"
	IsList	true
	FieldType	
	FieldName	TaskListSequenceList
	UseFieldDataType	true
setfieldex	From	0
	To	CLF::i
foreach	Var	CLF::Task
	List	VariantTasks
{		
setfieldex	From	true
	To	CLF::Result
setfieldex	From	CLF::Task.Parent.VariantExpression
	To	CLF::VarExpression
if	Condition	CLF::VarExpression
EvalParamIndirect	ExprLocation	CLF::VarExpression
	Context	CLF::__CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result
if	Condition	CLF::Task.VariantExpression
EvalParamIndirect	ExprLocation	CLF::Task.VariantExpression
	Context	CLF::__CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result == false
DeleteListItem	ListField	VariantTasks
	ItemValue	CLF::Task
	ItemIndex	
else		
{		
setfieldex	From	NbrTimesProcessedList[CLF::i]
	To	CLF::Task.NbrTimesProcessed
setfieldex	From	EProcedureExecutionModelList[CLF::i]
	To	CLF::Task.EProcedureExecutionMode
setfieldex	From	UIVirtualPageNameList[CLF::i]
	To	CLF::Task.UIVirtualPageName
setfieldex	From	IsComputationList[CLF::i]
	To	CLF::Task.IsComputation
setfieldex	From	TaskStatusList[CLF::i]
	To	CLF::TaskStatus
if	Condition	CLF::TaskStatus == 1
setfieldex	From	true
	To	CLF::Task.TaskStatus
elseif	Condition	CLF::TaskStatus == 0
setfieldex	From	false
	To	CLF::Task.TaskStatus
setfieldex	From	TaskListSequenceList[CLF::i]
	To	CLF::Task.TaskListSequence
}		
setfieldex	From	CLF::i + 1
	To	CLF::i
}		
ConvertCDOListToResultset	List	VariantTasks
	Fields	Parent.InstanceId,Parent.Name,Parent.Revision,Parent.Instruction,Name,InstanceId,Instruction,TaskListSequence,MinIterations,MaxIterations,TaskStatus,IsComputation,NbrTimesProcessed,DocumentSetId,TransactionPageId,WorkstationRequired,EProcedureExecutionMode,Parent.ExecutionMode,UIVirtualPageName,VariantExpression,MinIterationsExpression,MaxIterationsExpression,RequirePassFail
	ColNames	"TaskListId","TaskListName","TaskListRevision","TaskListInstruction","TaskName","TaskId","TaskInstruction","TaskSequence","MinIterations","MaxIterations","TaskStatus","IsComputation","NbrTimesProcessed","DocumentSetId","TransactionPageId","WorkstationRequired","EProcedureExecutionMode","TaskListExecutionMode","UIVirtualPageName","VariantExpression","MinIterationsExpression","MaxIterationsExpression","RequirePassFail"
	ColLabels	
	ResultsetStartRow	CLF::__ResultsetStartRow
	ResultsetSizeLimit	CLF::__ResultsetSizeLimit
	FilterFieldName	
	FilterParams	
	Result	Transaction::__Return
}		

Field: TaskList Events

SelectValuesEx: SelectionValuesEx_ExecuteTask_TaskList

CLF: SelectionValuesEx_ExecuteTask_TaskList

Function	Parameter	Value
----------	-----------	-------

InstructionTypeEnum - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Review	Review		Task that will display one or more grids of data based on User Query execution	Integer.[Integer]		Default: 4

LogDetailsForObject - Create

Feature Membership: Base

Parent CDO: ServiceData

Holds details of Operator Logs associated to an object. Also has a default name and LogEntryTypeGroup to use when creating a new log if none are currently associated to the object.

Field	Label	Column	Description	Data Type	Constraint	Value Expression
HasLogs	Has Logs		Flag indicating if the object has any associated Operator Logs	Boolean.[Boolean]		
OperatorLogInfos	Operator Log Info		Holds information about an Operator Log. Expands object fields to simplify REST Api requests. Optionally includes recorded entries.	Object. OperatorLogInfo		
Parent	Parent		Id of the CDO containing this CDO.	Object. OperatorLogInquiry		

LogEntryType - Create

Feature Membership: Base

Parent CDO: UserCode

Table: LogEntryType

A user defined type to associate with an operator log entry

Field	Label	Column	Description	Data Type	Constraint	Value Expression
CDOTypeId	CDO Type Id	CDOTypeId	CDO Definition Id	Integer.DefinitionId		
ChangeCount	Change Count	ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer.ChangeCount		
ChangeHistory	Change History	ChangeHistoryId	Details of the last change.	Object. ChangeHistory		
Description	Description	Description	Description of this entity. Defaults to the name of this entity if not specified	String.Description		
FilterTags	Filter Tags	FilterTags	FilterTags	String.FilterTags		
IconId	Icon ID	IconId	Identifier for the icon associated with this Base Entity	Integer.IconId		CVE: //Unique per CDO Type
InstanceID	Instance ID	LogEntryTypeId	Unique ID for an instance of this object, generated by the system.	Object. NamedDataObject		
IsFrozen	Locked/Unlocked	IsFrozen	Determines if changes are allowed to this instance (or any of its revisions).	Boolean.IsFrozen		
Name	Log Entry Type Name	LogEntryTypeName	Unique name for this instance	String.Name		
Notes	Notes	Notes	Notations and comments associated with this object.	String.Notes		

LogEntryTypeGroup - Create

Feature Membership: Base

Parent CDO: UserCodeGroup

Table: LogEntryTypeGroup

A group of LogEntryType objects

Field	Label	Column	Description	Data Type	Constraint	Value Expression
CDOTypeId	CDO Type Id	CDOTypeId	CDO Definition Id	Integer.DefinitionId		
ChangeCount	Change Count	ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer.ChangeCount		
ChangeHistory	Change History	ChangeHistoryId	Details of the last change.	Object. ChangeHistory		
DefaultForObjectTypes	Default For Object Types	DefaultForObjectTypes	List of object types. When an entity is created, if its type exists in this list, this group will be added as a default to the list of groups that will include the entity.	Integer.DefinitionId		
Description	Description	Description	Description of this entity. Defaults to the name of this entity if not specified	String.Description		
Entries	Entries	EntriesId	List of zero or more specific entries. EntryType specifies the object type.	Object. LogEntryType		
EntryType	Entry Type	EntryType	Name of the type of objects (CDO Definition Name) contained in this ObjectGroup. There is no business logic support for this field and it will be eventually removed.	String.CDOTypeName		Default: NamedDataObject

FilterTags	Filter Tags	FilterTags	FilterTags	String.FilterTags		
Groups	Groups	GroupsId	List of zero or more sub-Groups. EntryType specifies the object type. This allows a list of objects to be shared across multiple ObjectGroup s.	Object. LogEntryTypeGroup		
IconId	Icon ID	IconId	Identifier for the icon associated with this Base Entity	Integer.IconId		CVE: //Unique per CDO Type
InstanceID	Instance ID	LogEntryTypeGroupId	Unique ID for an instance of this object, generated by the system.	Object. NamedDataObject		
IsFrozen	Locked/Unlocked	IsFrozen	Determines if changes are allowed to this instance (or any of its revisions).	Boolean.IsFrozen		
Name	Log Entry Type Group Name	LogEntryTypeName	Unique name for this instance	String.Name		
Notes	Notes	Notes	Notations and comments associated with this object.	String.Notes		

LogEntryTypeGroupMaint - Create

Feature Membership: Base
Parent CDO: UserCodeGroupMaint

A group of LogEntryType objects

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ObjectChanges	Object Changes		This CDO holds the changes that are being made to Object until the user decides to make them permanent.	Object. LogEntryTypeGroupChanges		
ObjectListInquiry	Object List Inquiry		This is a list of instance Ids that can be maintained by this service.	Object. LogEntryTypeGroup		
ObjectToChange	Object to Change		This is the entity that is currently being maintained	Object. LogEntryTypeGroup		

LogEntryTypeMaint - Create

Feature Membership: Base
Parent CDO: UserCodeMaint

A user defined type to associate with an operator log entry

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ObjectChanges	Object Changes		This CDO holds the changes that are being made to Object until the user decides to make them permanent.	Object. LogEntryTypeChanges		
ObjectListInquiry	Object List Inquiry		This is a list of instance Ids that can be maintained by this service.	Object. LogEntryType		
ObjectToChange	Object to Change		This is the entity that is currently being maintained	Object. LogEntryType		

LogObjectTypeEnum - Create

Feature Membership: Base
Parent CDO: Enumeration

Identifies the type of object associated to an Operator Log 0 - NDO 1 - RDO 2 - Container 3 - QualityObject

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Container	Container		Container	Integer.[Integer]		Default: 2
NDO	NDO		NDO	Integer.[Integer]		
QualityObject	Quality Object		Quality Object	Integer.[Integer]		Default: 3
RDO	RDO		RDO	Integer.[Integer]		Default: 1

LogTxn - Create

Feature Membership: Base
Parent CDO: ShopFloor

Base class for transactions to record entries into logs

Field	Label	Column	Description	Data Type	Constraint	Value Expression
EndDateTime	End Date		End Date	TimeStamp.[TimeStamp]		
HistorySummary	History Summary		HistorySummary is the summary of the changes caused by a Service CDO that are not covered by the fields in the HistoryMainline object The HistorySummary will be automatically created and filled in by the InSite application server, and will be written to the database when the transaction is successfully completed. This field should not be updated by the client.	Object. ServiceHistorySummary		
LogEntry	Log Entry		Text of a log entry	String.Instruction		

LogEntryFilter	Log Entry Filter		Log Entry Filter	String.[String]		
OperatorLog	Operator Log		Log to which entry is associated	Object. OperatorLog		
StartDateTime	Start Date		Start Date	TimeStamp.[TimeStamp]		

MasterRecipe - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
EProcedureVariantDetails	Production Process Detail		Encapsulates various types of TaskList that are to be assigned to a ProductionProcess in a particular sequence after variant expressions are evaluated	Object. ProductionProcessDetail		
KeepCompletedTasks	Keep Completed Tasks on Move	KeepCompletedTasks	When true will prevent deleting ContainerCompletedTasks data for the container at the current step	Boolean.[Boolean]		
KeepCompletedTasksOnOpen	Keep Completed Tasks on Open	KeepCompletedTasksOnOpen	When true will not delete any completed task history	Boolean.[Boolean]		

MasterRecipeDetail - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ExportImportKey			A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.			

MaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed.Qty		CVE: This_Value;QtyRequired

MfgOrderMaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed.Qty		CVE: This_Value;QtyRequired

MfgOrderProcedure - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
EProcedureVariantDetails	Production Process Detail		Encapsulates various types of TaskList that are to be assigned to a ProductionProcess in a particular sequence after variant expressions are evaluated	Object. ProductionProcessDetail		
KeepCompletedTasks	Keep Completed Tasks on Move	KeepCompletedTasks	When true will prevent deleting ContainerCompletedTasks data for the container at the current step	Boolean.[Boolean]		
KeepCompletedTasksOnOpen	Keep Completed Tasks on Open	KeepCompletedTasksOnOpen	When true will not delete any completed task history	Boolean.[Boolean]		

MfgOrderTaskList - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
VariantExpression	Variant Expression	VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	String.Expression		
VariantTasks	Process Item		Proxy field to handle the validation of tasks due to the resolution of variant expressions	Object. ProcessItem		

MoveStd - Import

Methods**ValidateEProcedure**

CLF: MoveStd_ValidateEProcedure

Function	Parameter	Value
----------	-----------	-------

MoveTxn_CheckEProcedureComplete

CLF: MoveStd_CheckEProcedureComplete

Feature Membership: Base

Function	Parameter	Value
setfieldex	From	true
	To	CLF::Return
if	Condition	Container.Spec.ElectronicProcedure
{		
//	Line1	Validate that all required Tasks are
	Line2	completed successfully.
	Line3	
	Line4	
foreach	Var	CLF::EProcDetail
	List	Container.Spec.ElectronicProcedure.EProcedureDetails
{		
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::EProcDetail.TaskList.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	!Transaction::Return
continue		
foreach	Var	CLF::Task
	List	CLF::EProcDetail.TaskList.Tasks
{		
if	Condition	CLF::Task.MinIterations > 0
{		
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::Task.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	!Transaction::Return
continue		
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::FoundTask
	ListIndex	
if	Condition	NOT(CLF::FoundTask)
setfieldex	From	false
	To	CLF::Return
elseif	Condition	CLF::FoundTask.Pass != true
setfieldex	From	false
	To	CLF::Return
elseif	Condition	(CLF::FoundTask.NbrTimesProcessed < CLF::Task.MinIterations)
setfieldex	From	false
	To	CLF::Return
if	Condition	NOT(CLF::Return)
break		
}		
}		
}		

//	Line1	Determine if the Container is being
	Line2	processed at any Workstations.
	Line3	
	Line4	
if	Condition	GetListCount(Container.CurrentStatus.ClearanceWorkstations) > 0
{		
setfieldex	From	false
	To	CLF::Return
}		
}		
return	Value	CLF::Return

MoveTxn - Import

Methods

MoveTxn_CheckEProcedureComplete

CLF: MoveTxn_CheckEProcedureComplete
Feature Membership: Base

Function	Parameter	Value
return	Value	false

ClearCompletedTasks

CLF: MoveTxn_ClearCompletedTasks

Function	Parameter	Value
----------	-----------	-------

ObjectDataPoint - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
AttributeName	Attribute Name	AttributeName	A unique name for an attribute of a object.	String.AttributeName		
MapToUserAttribute	Map to User Attribute	MapToUserAttribute	Generic Boolean	Boolean.[Boolean]		
ValidateResource	Validate Resource	ValidateResource	When enabled will validate the selected resource to make sure it can be used	Boolean.[Boolean]		

Maps

ObjectDataPointToDataPointDetails

Map ObjectDataPoint to DataPointDetails

Source Field	Target Field
ValidateResource	ValidateResource

OnlineQuerySetup - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
UserQueryParameters						

Open - Import

Methods

ProcessElectronicProcedures

CLF: Open_ProcessElectronicProcedures

Function	Parameter	Value
----------	-----------	-------

OperatorLog - Create

Feature Membership: Base
Parent CDO: NamedDataObject
Table: OperatorLog

Log for shop floor operators to record entries of various types.

Field	Label	Column	Description	Data Type	Constraint	Value Expression
CDOTypeId	CDO Type Id	CDOTypeId	CDO Definition Id	Integer.DefinitionId		

ChangeCount	Change Count	ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer.ChangeCount		
ChangeHistory	Change History	ChangeHistoryId	Details of the last change.	Object. ChangeHistory		
Container	Container	ContainerId	Holds the object associated to the log if object type is a Container	Object. Container		
Description	Description	Description	Description of this entity. Defaults to the name of this entity if not specified	String.Description		
FilterTags	Filter Tags	FilterTags	FilterTags comma separated list	String.FilterTags		
IconId	Icon ID	IconId	Identifier for the icon associated with this Base Entity	Integer.IconId		CVE: //Unique per CDO Type
InstanceID	Instance ID	OperatorLogId	Unique ID for an instance of this object, generated by the system.	Object. NamedDataObject		
IsContainer	Is Container	IsContainer	True when object for log is a Container	Boolean.[Boolean]		
IsFrozen	Locked/Unlocked	IsFrozen	Determines if changes are allowed to this instance (or any of its revisions).	Boolean.IsFrozen		
IsNDO	Is NDO	IsNDO	True when object for log is an NDO	Boolean.[Boolean]		
IsQualityObject	Is Quality Object	IsQualityObject	True when object for log is a Quality Object	Boolean.[Boolean]		
IsRDO	IsRDO	IsRDO	True when object for log is an RDO	Boolean.[Boolean]		
LogEntryTypeGroup	Log Entry Type Group	LogEntryTypeGroupId	Limits the log entry types that may be selected for this log when recording a log entry.	Object. LogEntryTypeGroup		
LogObjectType	Log Object Type	LogObjectType	LogObjectType 0 - NDO 1 - RDO 2 - Container 3 - QualityObject	Integer.LogObjectType		
Name	Operator Log Name	OperatorLogName	Unique name for this instance	String.Name		
NamedDataObject	Named Data Object	NamedDataObjectId	Holds the object for the log if object is an NDO	Object. NamedDataObject		
Notes	Notes	Notes	Notations and comments associated with this object.	String.Notes		
ObjectInstanceId	Object Instance Id	ObjectInstanceId	Instance ID of the object to which the log is associated.	String.[String]		
QualityObject	Quality Object	QualityObjectId	Holds the object for the log if object is a QualityObject	Object. QualityObject		
RevisedObject	Revised Object	RevisedObjectId	Holds the object for the log if object is a RevisedObject	Object. RevisedObject		

OperatorLogInfo - Create

Feature Membership: Base

Parent CDO: ServiceData

Holds information about an Operator Log. Expands object fields to simplify REST Api requests. Optionally includes recorded entries.

Field	Label	Column	Description	Data Type	Constraint	Value Expression
LogEntries	Record Log Entry History		Recorded entries for this log	Object. RecordLogEntryHistory		
LogEntryTypeGroup	Log Entry Type Group		A group of LogEntryType objects	Object. LogEntryTypeGroup		
LogEntryTypes	Log Entry Type		A user defined type to associate with an operator log entry	Object. LogEntryType		CVE: LogEntryTypeGroup.ResolvedEntries
OperatorLog	Operator Log		Log for shop floor operators to record entries of various types.	Object. OperatorLog		

OperatorLogInquiry - Create

Feature Membership: Base

Parent CDO: Inquiry

Inquiry service for getting information related to Operator Logs

Field	Label	Column	Description	Data Type	Constraint	Value Expression
EndTimeFilter	End Time Filter		Used in filtering log entries by requiring they were recorded at a time at or before this value.	TimeStamp.[TimeStamp]		
GetLogEntries	Get Log Entries		Flag to control retrieval of log entries for existing logs.	Boolean.[Boolean]		
LogDetails	Log Details		Request this field to get information about an Operator Log. Can specify either OperatorLogId or OperatorLogName. Optionally includes recorded entries if GetLogEntries set true.	Object. OperatorLogInfo		
LogDetailsForObject	Log Details For Object		Request this field to get details of all Operator Logs associated to an object. Must specify the ObjectInstanceId. If no existing logs found, it will return a default name and LogEntryTypeGroup to use when creating a new log.	Object. LogDetailsForObject		
LogEntryFilter	LogEntryFilter		Input field. Filters log entries when getting LogDetails and including entries. Gets only entries where the entry value contains the filter as a substring.	String.String255		
LogEntryTypeFilter	Log Entry Type Filter		Input field. Filters log entries when getting LogDetails and including entries. Gets only entries with a type name containing the filter value as a substring.	String.Name		
ObjectInstanceId	Object Instance Id		Input field. Used to get Operator Log information for an object with this Instance ID	String.[String]		

ObjectName	Object Name		Input field. Used in building a suggested Operator Log name when no log exists for the object.	String.Name		
ObjectRevision	ObjectRevision		Input field. Used in building a suggested Operator Log name when no log exists for the object.	String.Revision		
ObjectTypeName	Object Type Name		Value determined from the ObjectInstanceId. Used in building a suggested Operator Log name when no log exists for the object.	String.Name		
OperatorLogId	Operator Log ID		Input field. Used to get all information associated to the Operator Log with this instance ID.	String.[String]		
OperatorLogName	OperatorLogName		Input field. Used to get all information associated to the Operator Log with this name.	String.Name		
OperatorLogs	Operator Logs		Holds list of Operator Logs associated to a specified object.	Object. OperatorLog		
StartTimeFilter	Start Time Filter		Used in filtering log entries by requiring they were recorded at a time at or after this value.	TimeStamp.[TimeStamp]		

Field: LogDetails Events

OnGetValue: OperatorLogInquiry_LogDetails_OGV

CLF: OperatorLogInquiry_LogDetails_OGV

Feature Membership: Base

Function	Parameter	Value
//	Line1	Get all log details based on the LogInstanceId or LogName field value
	Line2	
	Line3	
	Line4	
CreateCDO	CDOTypeName	"OperatorLogInfo"
	UseFieldType	false
	OverwriteExistingCDO	true
	NewCDO	CLF::LogDetails
if	Condition	OperatorLogName == null && OperatorLogId != null
{		
ExecuteQueryString	QueryString	"select OperatorLogName from OperatorLog where OperatorLogId = " + OperatorLogId + """
	Parameters	
	ResultsetStartRow	
	ResultsetSizeLimit	
	Result	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::Resultset
	ColName	"OperatorLogName"
	IsList	false
	FieldType	
	FieldName	OperatorLogName
	UseFieldDataType	true
}		
if	Condition	OperatorLogName != null
{		
ResolveNamedObjRef	ObjectName	OperatorLogName
	NamedObjRefField	CLF::LogDetails.OperatorLog
	ThrowError	false
}		
if	Condition	CLF::LogDetails.OperatorLog == null
throw	ErrorId	LabelId::OperatorLogInquiry_LogNoSpecified
	ParamNames	
	ParamValues	
setfieldex	From	CLF::LogDetails.OperatorLog.LogEntryTypeGroup
	To	CLF::LogDetails.LogEntryTypeGroup
if	Condition	GetLogEntries
{		
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"OperatorLog"
	ParamValue1	CLF::LogDetails.OperatorLog
	ParamName2	"StartDateTime"
	ParamValue2	LocalToGMT(StartTimeFilter)
	ParamName3	"EndDateTime"
	ParamValue3	LocalToGMT(EndTimeFilter)
	ParamName4	"LogEntryTypeFilter"
	ParamValue4	LogEntryTypeFilter ? ("% " + LogEntryTypeFilter + "%") : ""
	ParamName5	"LogEntryFilter"
	ParamValue5	LogEntryFilter ? ("% " + LogEntryFilter + "%") : ""
	ParamName6	
	ParamValue6	
ExecuteQueryEx	QueryName	"OperatorLogInquiry_GetLogEntries"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::Resultset
	ColName	"RecordLogEntryHistoryId"

	IsList	true
	FieldType	
	FieldName	CLF::LogDetails.LogEntries
	UseFieldDataType	true
}		
setfieldex	From	CLF::LogDetails
	To	LogDetails

Field: LogDetailsForObject Events

OnGetValue: OperatorLogInquiry_LogDetailsForObject_OGV

CLF: OperatorLogInquiry_LogDetailsForObject_OGV

Feature Membership: Base

Function	Parameter	Value
if	Condition	ObjectInstanceId == null
throw	ErrorId	LabelId::MissingFieldObjectInstanceId
	ParamNames	
	ParamValues	
CreateCDO	CDOTypeName	"LogDetailsForObject"
	UseFieldType	false
	OverwriteExistingCDO	true
	NewCDO	CLF::LogDetailsForObject
//	Line1	Set ObjectTypeName from InstanceId
	Line2	
	Line3	
	Line4	
InitQueryParametersEx	ParamNames	"ObjectInstanceId"
	ParamValues	ObjectInstanceId
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	"GetCDONameFromInstanceId"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"CDOName"
	IsList	false
	FieldType	
	FieldName	ObjectName
	UseFieldDataType	true
//	Line1	Get logs associated to object
	Line2	
	Line3	
	Line4	
InitQueryParametersEx	ParamNames	"ObjectInstanceId"
	ParamValues	ObjectInstanceId
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	"GetOperatorLogsForObject"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::Resultset
	ColName	"OperatorLogId"
	IsList	true
	FieldType	
	FieldName	OperatorLogs
	UseFieldDataType	true
if	Condition	GetListCount(OperatorLogs) > 0
{		
//	Line1	Object associated to a log. Set return data for those existing logs.
	Line2	
	Line3	
	Line4	
setfieldex	From	true
	To	CLF::LogDetailsForObject.HasLogs
foreach	Var	CLF::Log
	List	OperatorLogs
{		
setfieldex	From	CLF::Log.InstanceID
	To	CLF::DebugLogId
CreateListItem	ListCDO	CLF::LogDetailsForObject
	ListField	OperatorLogInfos
	ItemType	
	ItemValue	CLF::LogInfo

	ItemIndex	
setfieldex	From	CLF::Log
	To	CLF::LogInfo.OperatorLog
setfieldex	From	CLF::Log.LogEntryTypeGroup
	To	CLF::LogInfo.LogEntryTypeGroup
}		
}		
setfieldex	From	CLF::LogDetailsForObject
	To	LogDetailsForObject

OperatorLogMaint - Create

Feature Membership: Base

Parent CDO: NamedDataObjectMaint

Log for shop floor operators to record entries of various types.

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ObjectChanges	Object Changes		This CDO holds the changes that are being made to Object until the user decides to make them permanent.	Object. OperatorLogChanges		
ObjectListInquiry	Object List Inquiry		This is a list of instance Ids that can be maintained by this service.	Object. OperatorLog		
ObjectToChange	Object to Change		This is the entity that is currently being maintained	Object. OperatorLog		

Part - Import**Maps****PartToPartSetup**

Map Part to PartSetup

Source Field	Target Field
Product	Product
PartQty	PartQty
PartExpiryDate	PartExpiryDate
CurrentSetup	Setup

PartRequestAcknowledge - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
PartSetup	Part Setup		Setup for Part.	Object. PartSetup		

Methods**PartRequestAcknowledge_SetRequestType2**

CLF: PartRequestAcknowledge_SetRequestType2

Feature Membership: Base

Function	Parameter	Value
foreach	Var	ServiceDetailsItem
	List	ServiceDetails
if	Condition	ServiceDetailsItem.Part
{		
setfieldex	From	ServiceDetailsItem.Part.ProductionStatus.PartQty + ServiceDetailsItem.PartQty
	To	ServiceDetailsItem.Part.ProductionStatus.PartQty
if	Condition	ServiceDetailsItem.ScrapQty == 0
call	CDO	ServiceDetailsItem.Part
	Event	_ValidatePartLimits
	Name1	ThrowErrors
	Value1	false
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	ServiceDetailsItem.PhysicalLocation OR ServiceDetailsItem.PhysicalPosition
call	CDO	
	Event	PartTxn_PartSetup
	Name1	Part
	Value1	ServiceDetailsItem.Part
	Name2	PhysicalLocation

	Value2	ServiceDetailsItem.PhysicalLocation
	Name3	PhysicalPosition
	Value3	ServiceDetailsItem.PhysicalPosition
	Name4	SkipChangeCountValidation
	Value4	true
	Name5	
	Value5	
}		
SumListFields	ObjectList	ServiceDetails
	Field1	ScrapQty
	Sum1	HistorySummary.ScrapQty
	Field2	
	Sum2	
setfieldconditional	Condition	Resource.ProductionStatus.PartRequestCount > 0
	ThenFrom	Resource.ProductionStatus.PartRequestCount - 1
	ElseFrom	0
	To	Resource.ProductionStatus.PartRequestCount

PartSetup - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Availability	Availability		Defines the current availability status for a Resource. Valid values and their meanings are: 1 = Up (Available) 2 = Down (Not available)	Integer.Availability		

PartTxn - Import**Methods****PartTxn_PartSetup**

CLF: PartTxn_PartSetup

Feature Membership: Base

Function	Parameter	Value
if	Condition	(CLF::PhysicalLocation AND (CLF::Part.PhysicalLocation <> CLF::PhysicalLocation)) OR (CLF::PhysicalPosition AND (CLF::Part.PhysicalPosition <> CLF::PhysicalPosition))
{		
CreateCDO	CDOTypeName	
	UseFieldType	true
	OverwriteExistingCDO	true
	NewCDO	PartSetup
setfieldex	From	CLF::Part
	To	PartSetup.Resource
mapex	MapFrom	CLF::Part
	MapTo	PartSetup
	MapID	PartToPartSetup
setfieldconditional	Condition	CLF::PhysicalLocation AND (CLF::Part.PhysicalLocation <> CLF::PhysicalLocation)
	ThenFrom	CLF::PhysicalLocation
	ElseFrom	
	To	PartSetup.PhysicalLocation
setfieldconditional	Condition	CLF::PhysicalPosition AND (CLF::Part.PhysicalPosition <> CLF::PhysicalPosition)
	ThenFrom	CLF::PhysicalPosition
	ElseFrom	
	To	PartSetup.PhysicalPosition
call	CDO	
	Event	ExecuteService
	Name1	SubService
	Value1	PartSetup
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
}		

ProcessItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
MaxIterations						

MaxIterationsExpression	Max Iterations Expression		InSite Unified Expression	String.Expression		
MinIterations						
MinIterationsExpression	Min Iterations Expression		InSite Unified Expression	String.Expression		
VariantExpression	Variant Expression		Variant expression to be resolved to allow for filtering out configurable activities.	String.Expression		

Field: MinIterations Events

OnGetValue: MinIterationsExpression_OGV

CLF: MinIterationsExpression_OGV

Feature Membership: Base

Function	Parameter	Value
setfieldex	From	null
	To	CLF::Result
if	Condition	MinIterationsExpression != null
{		
EvalParamIndirect	ExprLocation	MinIterationsExpression
	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
try		
Trivial	Expression	CLF::Result
	ToType	Transaction::__Const.DataType.Integer
	IgnoreErrors	true
	Result	CLF::Result
catch	ErrorMsg	
	ErrorId	
setfieldex	From	null
	To	CLF::Result
}		
if	Condition	CLF::Result != null
return	Value	CLF::Result
else		
return	Value	CLF::__FieldValue

Field: MaxIterations Events

OnGetValue: MaxIterationsExpression_OGV

CLF: MaxIterationsExpression_OGV

Feature Membership: Base

Function	Parameter	Value
setfieldex	From	null
	To	CLF::Result
if	Condition	MaxIterationsExpression != null
{		
EvalParamIndirect	ExprLocation	MaxIterationsExpression
	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
try		
Trivial	Expression	CLF::Result
	ToType	Transaction::__Const.DataType.Integer
	IgnoreErrors	true
	Result	CLF::Result
catch	ErrorMsg	
	ErrorId	
setfieldex	From	null
	To	CLF::Result
}		
if	Condition	CLF::Result != null
return	Value	CLF::Result
else		
return	Value	CLF::__FieldValue

ProcessList - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
VariantExpression	Variant Expression		Variant expression to be resolved to allow for filtering out configurable activities.	String.Expression		
VariantTasks	Process Item		Proxy field to handle the validation of tasks due to the resolution of variant expressions	Object. ProcessItem		

ProcessSequenceValidation - Import**Methods****PopulateVariantTasks**

CLF: PopulateVariantTasks
Feature Membership: Base

Function	Parameter	Value
if	Condition	ProcessList && GetListCount(ProcessList.Tasks) > 0
setfieldex	From	ProcessList.Tasks
	To	ProcessList.VariantTasks
if	Condition	GetListCount(ProcessList.VariantTasks) > 0
{		
foreach	Var	CLF::Task
	List	ProcessList.VariantTasks
{		
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::Task.VariantExpression
EvalParamIndirect	ExprLocation	CLF::Task.VariantExpression
	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result == false
DeleteListItem	ListField	ProcessList.VariantTasks
	ItemValue	CLF::Task
	ItemIndex	
}		
}		

ValidatePrerequisiteTaskList

CLF: ProcessSeqValidation_ValidatePrerequisiteTaskList

Function	Parameter	Value
----------	-----------	-------

ValidateVariantExpression

CLF: ValidateVariantExpression
Feature Membership: Base

Function	Parameter	Value
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::VariantExpression
EvalParamIndirect	ExprLocation	CLF::VariantExpression
	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result
return	Value	true
else		
return	Value	false

PopulateEProcedureVariantDetails

CLF: PopulateEProcedureVariantDetails
Feature Membership: Base

Function	Parameter	Value
setfieldex	From	ProductionProcess.EProcedureDetails
	To	ProductionProcess.EProcedureVariantDetails
foreach	Var	CLF::Item
	List	ProductionProcess.EProcedureVariantDetails
{		
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::Item.TaskList.VariantExpression
EvalParamIndirect	ExprLocation	CLF::Item.TaskList.VariantExpression
	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result == false
DeleteListItem	ListField	ProductionProcess.EProcedureVariantDetails
	ItemValue	CLF::Item
	ItemIndex	
}		

ValidateTaskListSequence

CLF: ProcessSeqValidation_ValidateTaskListSequence

Function	Parameter	Value
----------	-----------	-------

ValidatePrerequisiteTasks

CLF: ProcessSeqValidation_ValidatePrerequisiteTasks

Function	Parameter	Value
----------	-----------	-------

AreBatchTasksPending

CLF: ProcessSeqValidation_AreBatchTasksPending

Function	Parameter	Value
----------	-----------	-------

ValidateTaskSequence

CLF: ProcessSeqValidation_ValidateTaskSequence

Feature Membership: Base

Function	Parameter	Value
call	CDO	
	Event	PopulateVariantTasks
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	GetListCount(CompletedTaskDetails) > 0
{		
//	Line1	Verify that the Task has not run more
	Line2	than the max iterations (if defined)
	Line3	
	Line4	
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	ProcessItem
	Result	CLF::FoundTask
	ListIndex	
if	Condition	CLF::FoundTask
if	Condition	ProcessItem.MaxIterations
if	Condition	((CLF::FoundTask.NbrTimesProcessed + 1) > ProcessItem.MaxIterations)
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskMaxRepititionExceeded
	ParamNames	"Name"
	ParamValues	ProcessItem.MaxIterations
else		
return	Value	False
}		
//	Line1	If the TaskList has a StartProcessTask,
	Line2	verify that it is executed before any of
	Line3	the other Tasks.
	Line4	
findlistitem	List	ProcessList.VariantTasks
	ListKey	TaskType
	KeyValue	Transaction:: __Const.TaskType.StartProcessTask
	Result	CLF::StartProcessTask
	ListIndex	
if	Condition	(CLF::StartProcessTask) and (ProcessItem <> CLF::StartProcessTask)
{		
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::StartProcessTask
	Result	CLF::CompletedStartProcessTask
	ListIndex	
if	Condition	CLF::CompletedStartProcessTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskStartTaskMustBeFirst
	ParamNames	"Name"
	ParamValues	CLF::StartProcessTask.DisplayName
else		

return	Value	False
}		
//	Line1	For Sequential TaskLists, verify that
	Line2	the Tasks are running in sequence
	Line3	
	Line4	
if	Condition	ProcessList.ExecutionMode == Transaction::__Const.ExecutionMode.Sequential
{		
setfieldex	From	LastCompletedTaskInTaskList
	To	CLF::LastExecutedTaskInListID
//	Line1	If the Task is the same as the last
	Line2	one processed, then no validations
	Line3	are necessary.
	Line4	
if	Condition	not (CLF::LastExecutedTaskInListID) or (ProcessItem <> CLF::LastExecutedTaskInListID)
{		
if	Condition	CLF::LastExecutedTaskInListID
{		
setfieldex	From	CLF::LastExecutedTaskInListID.Name
	To	CLF::LastTaskName
//	Line1	Optional sequential tasks become required task
	Line2	if executed. Validate that they have passed before
	Line3	starting on the next task.
	Line4	
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::LastExecutedTaskInListID
	Result	CLF::CompletedTask
	ListIndex	
if	Condition	CLF::CompletedTask
{		
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::CompletedTask
	Name2	TaskIsRequired
	Value2	True
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::__Return == False
return	Value	False
}		
//	Line1	The Task must have a higher
	Line2	sequence than the last (unless this
	Line3	is a new TaskList).
	Line4	
if	Condition	CLF::LastExecutedTaskInListID.Parent == ProcessList
{		
setfieldex	From	ProcessItem.Sequence
	To	CLF::ThisItemSequence
setfieldex	From	CLF::LastExecutedTaskInListID.Sequence
	To	CLF::LastExecutedSequence
if	Condition	ProcessItem.Sequence < CLF::LastExecutedTaskInListID.Sequence
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskOutOfSequence
	ParamNames	"Name"
	ParamValues	CLF::LastExecutedTaskInListID.Name
else		
return	Value	False
}		
}		
foreach	Var	CLF::TaskListDetail
	List	ProcessList.VariantTasks
{		
//	Line1	Once we have found the Task we
	Line2	are working on, there is no need to
	Line3	continue
	Line4	
if	Condition	CLF::TaskListDetail == ProcessItem
break		
if	Condition	CLF::LastExecutedTaskInListID and (CLF::LastExecutedTaskInListID.Parent == ProcessList) and (CLF::TaskListDetail.Sequence <= CLF::LastExecutedTaskInListID.Sequence)
continue		

if	Condition	CLF::TaskListDetail.MinIterations > 0
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskSkipped
	ParamNames	"Name"
	ParamValues	CLF::TaskListDetail.Name
else		
return	Value	False
}		
}		
}		
return	Value	True

GetCompletedTasks

CLF: ProcessSeqValidation_GetCompletedTasks

Feature Membership: Base

Function	Parameter	Value
if	Condition	(LastCompletedTask == Null) AND (Container.CurrentStatus.LastCompletedTask != null)
{		
setfieldex	From	Container.CurrentStatus.LastCompletedTask
	To	LastCompletedTask
if	Condition	ProcessList == Container.CurrentStatus.LastCompletedTask.Parent
{		
setfieldex	From	Container.CurrentStatus.LastCompletedTask
	To	LastCompletedTaskInTaskList
}		
else		
{		
for	Var	CLF::Index
	Init	0
	Condition	CLF::Index < GetListCount(ProcessList.Tasks)
	Inc	1
{		
setfieldex	From	null
	To	CLF::Task
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	ProcessList.Tasks[CLF::Index]
	Result	CLF::Task
	ListIndex	
if	Condition	CLF::Task
{		
setfieldex	From	ProcessList.Tasks[CLF::Index]
	To	LastCompletedTaskInTaskList
}		
}		
}		
}		
setfieldex	From	Container.Spec.ElectronicProcedure
	To	CLF::SpecProcedure
setfieldex	From	Container.WorkflowStep
	To	CLF::Step
setfieldex	From	null
	To	CLF::LastCompletedTaskInTaskList
foreach	Var	CLF::ContainerCompletedTask
	List	Container.CurrentStatus.ContainerCompletedTasks
{		
setfieldex	From	CLF::ContainerCompletedTask.ElectronicProcedure
	To	CLF::TaskProcedure
setfieldex	From	CLF::ContainerCompletedTask.WorkflowStep
	To	CLF::TaskStep
if	Condition	(NOT(CLF::TaskProcedure) OR CLF::TaskProcedure == CLF::SpecProcedure) AND (NOT(CLF::TaskStep) OR CLF::TaskStep == CLF::Step)
{		
if	Condition	NOT(CLF::LastCompletedTaskInTaskList) OR CLF::ContainerCompletedTask.CompletionTimestampGMT > CLF::LastCompletedTaskInTaskList.CompletionTimestampGMT
{		
setfieldex	From	CLF::ContainerCompletedTask
	To	CLF::LastCompletedTaskInTaskList
}		
setfieldex	From	CLF::ContainerCompletedTask.Task.Name
	To	CLF::TaskName
setfieldex	From	CLF::ContainerCompletedTask.Task.Sequence
	To	CLF::TaskSequence
CreateListItem	ListCDO	CLF::__CDOID
	ListField	CompletedTaskDetails
	ItemType	
	ItemValue	CLF::CompletedTaskDetail

	ItemIndex	CLF::Index
mapex	MapFrom	CLF::ContainerCompletedTask
	MapTo	CLF::CompletedTaskDetail
	MapID	ContainerCompletedTaskToCompletedTaskDetail
}		
}		
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	LastCompletedTask
	Result	CLF::Task
	ListIndex	
if	Condition	CLF::LastCompletedTaskInTaskList
setfieldex	From	CLF::LastCompletedTaskInTaskList.Task
	To	LastCompletedTaskInTaskList
else		
setfieldex	From	null
	To	LastCompletedTaskInTaskList
if	Condition	GetListCount(CompletedTaskDetails) == 0 OR NOT(LastCompletedTaskInTaskList)
{		
setfieldex	From	null
	To	LastCompletedTask
setfieldex	From	null
	To	LastCompletedTaskInTaskList
}		

Product - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ResolvedBOM	Resolved BOM		A bill of material defines the materials needed to produce a specific product.	Object. BOM		CVE: BOM;ERP BOM

ProductionProcess - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
EProcedureVariantDetails	Production Process Detail		Encapsulates various types of TaskList that are to be assigned to a ProductionProcess in a particular sequence after variant expressions are evaluated	Object. ProductionProcessDetail		
KeepCompletedTasks	Keep Completed Tasks on Move		When true will prevent deleting ContainerCompletedTasks data for the container at the current step	Boolean.[Boolean]		
KeepCompletedTasksOnOpen	Keep Completed Tasks on Open		When true will not delete any completed task history	Boolean.[Boolean]		

ProductMaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed.Qty		CVE: This _Value;QtyRequired

RecipeItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
MaxIterationsExpression	Max Iterations Expression	MaxIterationsExpression	InSite Unified Expression	String.Expression		
MinIterationsExpression	Min Iterations Expression	MinIterationsExpression	InSite Unified Expression	String.Expression		
VariantExpression	Variant Expression	VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	String.Expression		

RecipeList - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
VariantExpression	Variant Expression	VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	String.Expression		
VariantTasks	Process Item		Proxy field to handle the validation of tasks due to the resolution of variant expressions	Object. ProcessItem		

RecipeMaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed_Qty		CVE: This_Value;QtyRequired

RecordLogEntry - Create

Feature Membership: Base

Parent CDO: LogTxn

Records a new entry for a specified log

Field	Label	Column	Description	Data Type	Constraint	Value Expression
AttachDocumentDetails	Attach Document Details		Documents attached to the Operator Log	Object. AttachDocumentDetails		
Container	Container		Container associated to the Operator Log	Object. Container		CVE: OperatorLog.Container
HistorySummary	History Summary		HistorySummary is the summary of the changes caused by a Service CDO that are not covered by the fields in the HistoryMainline object The HistorySummary will be automatically created and filled in by the InSite application server, and will be written to the database when the transaction is successfully completed. This field should not be updated by the client.	Object. RecordLogEntryHistory		
IsContainer	Is Container		True if object associated to the log is a Container	Boolean.[Boolean]		CVE: OperatorLog.IsContainer
IsNDO	Is NDO		True if object associated to the log is a named object	Boolean.[Boolean]		CVE: OperatorLog.IsNDO
IsQualityObject	Is Quality Object		True if object associated to the log is a Quality Object	Boolean.[Boolean]		CVE: OperatorLog.IsQualityObject
IsRDO	Is RDO		True if object associated to the log is a revisioned object	Boolean.[Boolean]		CVE: OperatorLog.IsRDO
LogEntryType	Log Entry Type		A user defined type to associate with an operator log entry	Object. LogEntryType		
LogEntryTypeFilter	Log Entry Type Filter		A user defined type to associate with an operator log entry	Object. LogEntryType		
LogEntryTypeName	Log Entry Type Name		Name of the log entry type	String.Name		CVE: LogEntryType.Name
NamedDataObject	Named Data Object		NDO associated to the log	Object. NamedDataObject		CVE: OperatorLog.NamedDataObject
ObjectType	Object Type		CDO Definition Id for the object associated to the log	Integer.DefinitionId		CVE: This_Value;OperatorLog.ObjectType
ObjectTypeName	Object Type Name		CDO name of the object associated to the log	String.[String]		
OperatorLogName	Operator Log Name		Name of the operator log	String.Name		CVE: OperatorLog.Name
QualityObject	Quality Object		Quality Object associated to the log	Object. QualityObject		CVE: OperatorLog.QualityObject
RecordLogEntryDetail	Record Log Entry Detail		Record Log Entry Detail	Object. RecordLogEntryDetail		
RevisionedObject	Revisioned Object		RDO associated to the log	Object. RevisionedObject		CVE: OperatorLog.RevisionedObject

Field: LogEntryType Events

SelectValuesEx: SelectionValuesEx_AllowedLogEntryTypes

CLF: SelectionValuesEx_AllowedLogEntryTypes

Feature Membership: Base

Function	Parameter	Value
ConvertCDOListToResultset	List	OperatorLog.LogEntryTypeGroup.ResolvedEntries
	Fields	Name,InstanceID,IsFrozen,Description
	ColNames	
	ColLabels	
	ResultsetStartRow	CLF: __ResultsetStartRow
	ResultsetSizeLimit	CLF: __ResultsetSizeLimit
	FilterFieldName	
	FilterParams	
	Result	Transaction::__Return

Field: AttachDocumentDetails Events

OnGetValue: OpLog_GetAttachedDocuments_OGV

CLF: OpLog_GetAttachedDocuments_OGV

Feature Membership: Base

Function	Parameter	Value
----------	-----------	-------

if	Condition	OperatorLog
{		
InitQueryParametersEx	ParamNames	"ObjectInstanceId"
	ParamValues	OperatorLog
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	"AttachDocument_GetDocuments"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToObjects	Resultset	CLF::Resultset
	ObjectType	
	UseFieldType	true
	Result	AttachDocumentDetails
	IsList	true
	Method	
}		

Field: LogEntryTypeFilter Events

SelectValuesEx: SelectionValuesEx_AllowedLogEntryTypes

CLF: SelectionValuesEx_AllowedLogEntryTypes

Feature Membership: Base

Function	Parameter	Value
ConvertCDOListToResultset	List	OperatorLog.LogEntryTypeGroup.ResolvedEntries
	Fields	Name,InstanceId,IsFrozen,Description
	ColNames	
	ColLabels	
	ResultsetStartRow	CLF::__ResultsetStartRow
	ResultsetSizeLimit	CLF::__ResultsetSizeLimit
	FilterFieldName	
	FilterParams	
	Result	Transaction::__Return

Field: ObjectTypeName Events

OnGetValue: RecordLogEntry_ObjectTypeName_OGV

CLF: RecordLogEntry_ObjectTypeName_OGV

Feature Membership: Base

Function	Parameter	Value
setfieldex	From	null
	To	CLF::CDOTypeName
if	Condition	IsNDO
GetCDOType	CDO	NamedDataObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
elseif	Condition	IsContainer
GetCDOType	CDO	Container
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
elseif	Condition	IsQualityObject
GetCDOType	CDO	QualityObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
elseif	Condition	IsRDO
GetCDOType	CDO	RevisionedObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
setfieldex	From	CLF::CDOTypeName
	To	Transaction::__Return

Field: RecordLogEntryDetail Events

SelectValuesEx: SelVal_GetOperatorLogEntries

CLF: SelVal_GetOperatorLogEntries

Feature Membership: Base

Function	Parameter	Value
if	Condition	OperatorLog = null
return	Value	
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"OperatorLog"
	ParamValue1	OperatorLog
	ParamName2	"StartDateTime"
	ParamValue2	StartDateTime
	ParamName3	"EndDateTime"

	ParamValue3	EndDateTime
	ParamName4	"LogEntryType"
	ParamValue4	LogEntryTypeFilter
	ParamName5	"LogEntry"
	ParamValue5	LogEntryFilter
	ParamName6	
	ParamValue6	
ExecuteQueryEx	QueryName	"SelectionValuesEx_OpLog_EntryHistory"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	
	ResultsetStartRow	
	Resultset	Transaction::__Return

Field: HistorySummary Events

SelectValuesEx: SelVal_GetOperatorLogEntries

CLF: SelVal_GetOperatorLogEntries

Feature Membership: Base

Function	Parameter	Value
if	Condition	OperatorLog = null
return	Value	
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"OperatorLog"
	ParamValue1	OperatorLog
	ParamName2	"StartDateTime"
	ParamValue2	StartDateTime
	ParamName3	"EndDateTime"
	ParamValue3	EndDateTime
	ParamName4	"LogEntryType"
	ParamValue4	LogEntryTypeFilter
	ParamName5	"LogEntry"
	ParamValue5	LogEntryFilter
	ParamName6	
	ParamValue6	
ExecuteQueryEx	QueryName	"SelectionValuesEx_OpLog_EntryHistory"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	
	ResultsetStartRow	
	Resultset	Transaction::__Return

Methods**StartModifyingEntity**

CLF: RecordLogEntry_StartModifyingEntity

Feature Membership: Base

Function	Parameter	Value
callsuper	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	IncludeLogicFromThisCDO	
if	Condition	IsQualityObject
setfieldex	From	QualityObject
	To	HistoryMainline.HistoryId
if	Condition	IsContainer
{		
setfieldex	From	Container
	To	HistoryMainline.HistoryId
setfieldex	From	Container
	To	HistoryMainline.Container
}		
if	Condition	IsNDO
{		
GetCDOSubTypes	CDOTypeName	"Resource"
	Recursive	true
	IncludeCurrentCDO	false
	SubtypeDisplayNames	
	SubtypeNameNames	CLF::SubTypes
	SubtypeIds	
GetCDOType	CDO	NamedDataObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
findlistitem	List	CLF::SubTypes

	ListKey	
	KeyValue	CLF::CDOTypeName
	Result	CLF::Found
	ListIndex	
if	Condition	CLF::Found == True CLF::CDOTypeName == "Resource"
setfieldex	From	NamedDataObject
	To	HistoryMainline.Resource
}		

PreValidate_User

CLF: RecordLogEntry_PreValidate_User

Feature Membership: Base

Function	Parameter	Value
callsuper	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	IncludeLogicFromThisCDO	
if	Condition	OperatorLog = null LogEntryType = null LogEntry = null
throw	ErrorId	LabelId::OperatorLogEmptyError
	ParamNames	
	ParamValues	

Maps**RecordLogEntryToRecordLogEntryHistory**

Map RecordLogEntry to RecordLogEntryHistory

Source Field	Target Field
HistoryMainline	Parent
Employee	Employee
EmployeeName	EmployeeName
LogEntry	LogEntry
LogEntryType	LogEntryType
LogEntryTypeName	LogEntryTypeName
OperatorLog	OperatorLog
OperatorLogName	OperatorLogName
TxnDate	TxnDateTime

RecordLogEntryDetail - Create

Feature Membership: Base

Parent CDO: ServiceData

Record Log Entry Detail

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Employee	Employee		The user who logged the entry	Object. Employee		
LogEntry	Log Entry		Text of the log entry	String.Instruction		
Parent	Parent		Id of the CDO containing this CDO.	Object. RecordLogEntry		
RecordLogEntryHistory	Record Log Entry History		History record that acts as an Operator Log entry	Object. RecordLogEntryHistory		
TxnDateTime	Txn Date Time		Time stamp of when log entry was recorded	TimeStamp.[TimeStamp]		

RecordLogEntryHistory - Create

Feature Membership: Base

Parent CDO: ServiceHistorySummary

Table: RecordLogEntryHistory

History record that acts as an Operator Log entry

Field	Label	Column	Description	Data Type	Constraint	Value Expression
CDOTypeId	CDO Type Id	CDOTypeId	CDO Definition Id	Integer.DefinitionId		
ChangeCount	Change Count	ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer.ChangeCount		
Employee	Employee	EmployeeId	The operator recording the log entry	Object. Employee		CVE: Parent.Employee
EmployeeName	Employee Name	EmployeeName	Name of operator	String.Name		
ExportImportKey	Record Log Entry History ExportImportKey	RecordLogEntryHistoryName	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.	String.GUID		
HistoryDetails	History Details	HistoryDetailsId	Summary of the changes caused by a single ServiceDetail CDO.	Object. ServiceHistoryDetail		

HistoryId	History ID	HistoryId	Unique identifier associated with all Containers participating in this transaction.	Object. BaseObject		CVE: Parent.HistoryId
InstanceID	Instance ID	RecordLogEntryHistoryId	Unique ID for an instance of this object, generated by the system.	Object. Subentity		
IntegrationTxn	Integration Txn	IntegrationTxnId	Inserts the TxnId after a compound/container/resource transaction with history mainline.	Object. BaseObject		
LogEntry	Log Entry	LogEntry	Text of the log entry	String.Instruction		
LogEntryType	Log Entry Type	LogEntryTypeId	A user defined type to associate with an operator log entry	Object. LogEntryType		
LogEntryTypeName	Log Entry Type Name	LogEntryTypeName	Name of the log entry type	String.Name		
OperatorLog	Operator Log	OperatorLogId	Log for shop floor operators to record entries of various types.	Object. OperatorLog		
OperatorLogName	Operator Log Name	OperatorLogName	Name of the log	String.Name		
Parent	Parent	HistoryMainlineId	Id of the HistoryMainline to which this ServiceHistorySummary applies.	Object. HistoryMainline		
TxnDateTime	TxnDateTime	TxnDateTime	Generic TimeStamp	TimeStamp.[TimeStamp]		
TxnId	Txn Id	TxnId	Unique number used to identify this transaction. This value is used to logically link (and join in SQL) all of the entries associated with a single Service. Note that compound services (availabel in V1.4) can create multiple HistoryMainlines, but the HistoryMainlines and other history records relating to a single transaction will always get assigned the same unique TxnId.	Object. BaseObject		

ResourceBOMMaterialListItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ActualQtyRequired	Actual Qty Required		Material quantity (individual)	Fixed.Qty		CVE: This_Value;QtyRequired

Service - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
CurrentDay	Day		Generic String	String.[String]		
CurrentMonth	Month		Generic String	String.[String]		
CurrentYear	Year		Generic String	String.[String]		

Field: CurrentMonth Events

OnGetValue: CurrentMonth_OGV

CLF: CurrentMonth_OGV

Feature Membership: Base

Function	Parameter	Value
DatePart	Date	TxnDate
	DatePart	"month"
	PartOfDate	CLF::Month
StrPadLeft	String	String(CLF::Month)
	TotalWidth	2
	PaddingChar	"0"
	Result	CLF::Month
return	Value	CLF::Month

Field: CurrentDay Events

OnGetValue: CurrentDay_OGV

CLF: CurrentDay_OGV

Feature Membership: Base

Function	Parameter	Value
DatePart	Date	TxnDate
	DatePart	"day"
	PartOfDate	CLF::Day
StrPadLeft	String	String(CLF::Day)
	TotalWidth	2
	PaddingChar	"0"
	Result	CLF::Day
return	Value	CLF::Day

Field: CurrentYear Events

OnGetValue: CurrentYear_OGV

CLF: CurrentYear_OGV

Feature Membership: Base

Function	Parameter	Value
DatePart	Date	TxnDate
	DatePart	"year"
	PartOfDate	CLF::Year
return	Value	String(CLF::Year)

ShopFloor - Import**Methods****ValidateVariantExpression**

CLF: ValidateVariantExpression

Feature Membership: Base

Function	Parameter	Value
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::VariantExpression
EvalParamIndirect	ExprLocation	CLF::VariantExpression
	Context	CLF::_CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result
return	Value	true
else		
return	Value	false

SPCChartCustomInfo - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
InAnnotationPopup	In Annotation Popup	InAnnotationPopup	In Annotation Popup	Boolean.[Boolean]		

SPCChartDef - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
DisplayDescriptiveStats	Display Descriptive Stats	DisplayDescriptiveStats	Generic Boolean	Boolean.[Boolean]		
SPCChartVisualParams	SPC Chart Visual Parameters		SPC Chart Visual Parameters	Object. SPCChartVisualParams		

SPCChartType - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
VisualParams	SPC Chart Visual Parameters		SPC Chart Visual Parameters	Object. SPCChartVisualParams		

SPCChartVisualParams - Create

Feature Membership: Base

Parent CDO: Subentity

Table: SPCChartVisualParams

SPC Chart Visual Parameters

Field	Label	Column	Description	Data Type	Constraint	Value Expression
CDOTypeId	CDO Type Id	CDOTypeId	CDO Definition Id	Integer.DefinitionId		
ChangeCount	Change Count	ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer.ChangeCount		
ExportImportKey	SPC Chart Visual Params ExportImportKey	SPCChartVisualParamsName	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.	String.GUID		
InstanceID	Instance ID	SPCChartVisualParamsId	Unique ID for an instance of this object, generated by the system.	Object. Subentity		
ParamName	Parameter Name	ParamName	Parameter Name	String.DisplayName		
ParamValue	Parameter Value	ParamValue	Parameter Value	String.DefaultValue		
Parent	Parent	ParentId	Id of the CDO containing this CDO.	Object. BaseObject		

SPCQuery - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
ExternalDataId	Data ID	ExternalDataId	Data ID from external data source	String.Comments		
ExternalTable	Secondary Data ID	ExternalTable	Table Name from external data source	String.Comments		

StartDetails - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Level						
Owner						
StartReason						

TaskItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
DocumentSetId	Document Set Id		Generic String	String.[String]		CVE: DocumentSet.InstanceId
EProcedureExecutionMode	EProcedure Execution Mode		Enumeration for Execution Mode. 1 = Sequential 2 = Non-Sequential	Integer.ExecutionMode		
IsComputation	Is Computation		IsComputation	Boolean.[Boolean]		
MaxIterationsExpression	Max Iterations Expression	MaxIterationsExpression	InSite Unified Expression	String.Expression		
MinIterationsExpression	Min Iterations Expression	MinIterationsExpression	InSite Unified Expression	String.Expression		
NbrTimesProcessed	Number of Times Processed		Identifies how many times the Task has been processed within the context of the current step.	Integer.[Integer]		
RequirePassFail	Require Pass/Fail	RequirePassFail	Require Pass/Fail	Boolean.[Boolean]		
TaskListSequence	Task List Sequence		Relative sequence number	Integer.Sequence		
TaskQueries	Review Task Queries		Review Task Queries	Object. TaskQueries		
TaskStatus	Task Status		Status of Task	Boolean.[Boolean]		
TransactionPageId	Transaction Page Id		ID of the Virtual Page	String.[String]		
UIVirtualPageName	UI Virtual Page Name		FileName - used for fields that specify files.	String.FileName		
VariantExpression	Variant Expression	VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	String.Expression		
WorkstationRequired	Workstation Required		When TaskType = 1, then WorkstationRequired is 1, otherwise 0	Integer.[Integer]		CVE: TaskType == 1 ? 1 : 0

TaskList - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
VariantExpression	Variant Expression	VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	String.Expression		
VariantTasks	Process Item		Proxy field to handle the validation of tasks due to the resolution of variant expressions	Object. ProcessItem		

TaskQueries - Create

Feature Membership: Base
 Parent CDO: Subentity
 Table: TaskQueries

Review Task Queries

Field	Label	Column	Description	Data Type	Constraint	Value Expression
CDOTypeId	CDO Type Id	CDOTypeId	CDO Definition Id	Integer.DefinitionId		
ChangeCount	Change Count	ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer.ChangeCount		
ExportImportKey	Export Import Key	TaskQueriesName	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.	String.GUID		
InstanceID	Instance ID	TaskQueriesId	Unique ID for an instance of this object, generated by the system.	Object. Subentity		
IsFrozen		IsFrozen	IsFrozen	Boolean.IsFrozen		

OnlineQuerySetup	Online Query	OnlineQuerySetupId	These queries are used in an ad-hoc manner to view runtime data using the Online Query service. The Resultset Size Limit must be used to ensure that InSite does not reserve too much memory that may impact performance.	Object. OnlineQuerySetup		
Parent	Parent	TaskItemId	Id of the CDO containing this CDO.	Object. TaskItem		
Sequence	Sequence	Sequence	Relative sequence number	Integer.Sequence		

TransactionItem - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
TransactionPageId	Transaction Page Id		ID of the Virtual Page	String.[String]		CVE: TransactionPage.InstanceId

UnitStartDetails - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
Owner						
StartReason						

UserQueryParameter - Import

Field	Label	Column	Description	Data Type	Constraint	Value Expression
QueryName	Query Name		Query Name	String.String100		
ResolvedValue	Resolved Value		Resolved Value	String.String255		
TaskName	Task Name		Task Name	String.String100		

CLF Definitions**ApplyTaskMaterialList**

Base Functions		
Function	Parameter	Value
foreach	Var	CLF::Item
	List	CalledByTransactionTask.TaskMaterials
if	Condition	CLF::Item.ReferenceDesignator
AddListItem	ListField	TaskMaterialsRefDes
	Item	CLF::Item
	Result	TaskMaterialsRefDes
AddListItem	ListField	TaskMaterialsProduct
	Item	CLF::Item
	Result	TaskMaterialsProduct
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count
setfieldex	From	null
	To	CLF::ObjectName
foreach	Var	CLF::Item
	List	ServiceDetailsTmp
setfieldex	From	CLF::Item.ReferenceDesignator
	To	CLF::RefDes
setfieldex	From	CLF::Item.Product.Name
	To	CLF::ProductName
setfieldex	From	-1
	To	CLF::Index
setfieldex	From	null
	To	CLF::FoundItem
if	Condition	CLF::RefDes
findlistitem	List	TaskMaterialsRefDes
	ListKey	ReferenceDesignator
	KeyValue	CLF::RefDes
	Result	CLF::FoundItem
	ListIndex	CLF::Index
if	Condition	CLF::Index >= 0 && CLF::FoundItem && CLF::FoundItem.Product != CLF::Item.Product
setfieldex	From	-1
	To	CLF::Index
if	Condition	CLF::Index < 0
findlistitem	List	TaskMaterialsProduct
	ListKey	Product
	KeyValue	CLF::Item.Product
	Result	CLF::FoundItem
	ListIndex	CLF::Index

if	Condition	CLF::Index < 0
DeleteListItem	ListField	ServiceDetailsTmp
	ItemValue	CLF::Item
	ItemIndex	
elseif	Condition	CLF::FoundItem
if	Condition	NOT(CLF::ObjectName)
GetCDOType	CDO	CLF::Item
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
setfieldex	From	CLF::CDOTypeName
	To	CLF::Item.ObjectTypeName
setfieldex	From	CLF::FoundItem.Description
	To	CLF::Desc
if	Condition	CLF::Desc
setfieldex	From	CLF::Desc
	To	CLF::Item.Description
if	Condition	CLF::FoundItem.Qty
setfieldex	From	CLF::FoundItem.Qty
	To	CLF::Item.QtyRequired
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count

New Functions		
Function	Parameter	Value
if	Condition	NOT(CalledByTransactionTask) OR GetListCount(CalledByTransactionTask.TaskMaterials) == 0
if	Condition	CalledByTransactionTask && !(IsOfObjectType(CalledByTransactionTask,"TransactionItem",true))
ClearList	ListCDO	CLF::__CDOID
	ListField	ServiceDetailsTmp
return	Value	CalledByTransactionTask
foreach	Var	CLF::Item
	List	CalledByTransactionTask.TaskMaterials
if	Condition	CLF::Item.ReferenceDesignator
AddListItem	ListField	TaskMaterialsRefDes
	Item	CLF::Item
	Result	TaskMaterialsRefDes
AddListItem	ListField	TaskMaterialsProduct
	Item	CLF::Item
	Result	TaskMaterialsProduct
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count
setfieldex	From	null
	To	CLF::ObjectName
foreach	Var	CLF::Item
	List	ServiceDetailsTmp
setfieldex	From	CLF::Item.ReferenceDesignator
	To	CLF::RefDes
setfieldex	From	CLF::Item.Product.Name
	To	CLF::ProductName
setfieldex	From	-1
	To	CLF::Index
setfieldex	From	null
	To	CLF::FoundItem
if	Condition	CLF::RefDes
findlistitem	List	TaskMaterialsRefDes
	ListKey	ReferenceDesignator
	KeyValue	CLF::RefDes
	Result	CLF::FoundItem
	ListIndex	CLF::Index
if	Condition	CLF::Index >= 0 && CLF::FoundItem && CLF::FoundItem.Product != CLF::Item.Product
setfieldex	From	-1
	To	CLF::Index
if	Condition	CLF::Index < 0
findlistitem	List	TaskMaterialsProduct
	ListKey	Product
	KeyValue	CLF::Item.Product
	Result	CLF::FoundItem
	ListIndex	CLF::Index
if	Condition	CLF::Index < 0
DeleteListItem	ListField	ServiceDetailsTmp
	ItemValue	CLF::Item
	ItemIndex	
elseif	Condition	CLF::FoundItem
if	Condition	NOT(CLF::ObjectName)
GetCDOType	CDO	CLF::Item
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
setfieldex	From	CLF::CDOTypeName

	To	CLF::Item.ObjectTypeName
setfieldex	From	CLF::FoundItem.Description
	To	CLF::Desc
if	Condition	CLF::Desc
setfieldex	From	CLF::Desc
	To	CLF::Item.Description
if	Condition	CLF::FoundItem.Qty
setfieldex	From	CLF::Item.QtyRequired
	To	CLF::Item.ActualQtyRequired
setfieldex	From	CLF::FoundItem.Qty
	To	CLF::Item.QtyRequired
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count

ChangeQty_FinishModifyingEntity

Base Functions		
Function	Parameter	Value
callsuper	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	IncludeLogicFromThisCDO	false
foreachobject	List	LostContainers
	Call	Disassociate
	Name1	
	Value1	
	Name2	
	Value2	
if	Condition	Container.CurrentStatus.Carrier and (Container.Qty = 0)
call	CDO	Container
	Event	DetachCarrier
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	GetListCount(HistorySummary.HistoryDetails) > 1
setfieldex	From	ServiceDetails[0].ReasonCode
	To	CLF::SaveReasonCode
setfieldex	From	ServiceDetails[0].EnteredQty
	To	CLF::EnteredQty
if	Condition	HistorySummary.Qty == 0
setfieldex	From	CLF::EnteredQty
	To	HistorySummary.EnteredQty
foreach	Var	CLF::Item
	List	HistorySummary.HistoryDetails
if	Condition	CLF::Item.Qty == 0
setfieldex	From	CLF::EnteredQty
	To	CLF::Item.Qty
setfieldex	From	CLF::Item.ReasonCode
	To	CLF::ReasonCode
if	Condition	NOT(CLF::ReasonCode)
setfieldex	From	CLF::SaveReasonCode
	To	CLF::Item.ReasonCode
setfieldex	From	CLF::Item.ReasonCode.Name
	To	CLF::Item.ReasonCodeName

New Functions		
Function	Parameter	Value
callsuper	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	IncludeLogicFromThisCDO	false
foreachobject	List	LostContainers

	Call	Disassociate
	Name1	
	Value1	
	Name2	
	Value2	
if	Condition	Container.CurrentStatus.Carrier and (Container.Qty = 0)
call	CDO	Container
	Event	DetachCarrier
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	GetListCount(HistorySummary.HistoryDetails) > 1
setfieldex	From	ServiceDetails[0].ReasonCode
	To	CLF::SaveReasonCode
setfieldex	From	ServiceDetails[0].EnteredQty
	To	CLF::EnteredQty
if	Condition	HistorySummary.Qty == 0
setfieldex	From	CLF::EnteredQty
	To	HistorySummary.EnteredQty
foreach	Var	CLF::Item
	List	HistorySummary.HistoryDetails
if	Condition	CLF::Item.Qty == 0
setfieldex	From	CLF::EnteredQty
	To	CLF::Item.Qty
setfieldex	From	CLF::Item.ReasonCode
	To	CLF::ReasonCode
if	Condition	NOT(CLF::ReasonCode)
setfieldex	From	CLF::SaveReasonCode
	To	CLF::Item.ReasonCode

CILight_Load_PhantomList

CILight_Load_PhantomList

New Functions		
Function	Parameter	Value
ClearList	ListCDO	CLF:: CDOID
	ListField	PhantomIssueDetailsTmp
if	Condition	PhantomBillId
call	CDO	
	Event	SetMaterialListItemSettings
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	zMaterialListItemSettings.PhantomQueryName != null
InitQueryParametersEx	ParamNames	"PhantomBillId", "Container"
	ParamValues	PhantomBillId, Container
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	zMaterialListItemSettings.PhantomQueryName
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::MaterialList
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"MaterialListItemId"
	IsList	true
	FieldType	
	FieldName	PhantomIssueDetailsTmp
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"QtyIssued"

	IsList	true
	FieldType	
	FieldName	PhantomQtyIssuedListTmp
	UseFieldDataType	true
setfieldex	From	0
	To	CLF::i
foreach	Var	CLF::Item
	List	PhantomQtyIssuedListTmp
setfieldex	From	CLF::Item
	To	PhantomIssueDetailsTmp[CLF::i].QtyIssued
setfieldex	From	CLF::i + 1
	To	CLF::i

CI Light_MaterialList_SelVal

Function	Parameter	
call	CDO	
	Event	SetMaterialListItemSettings
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
ValidateContainerOpen	Container	Container
call	CDO	
	Event	ValidateHoldStatus
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	zMaterialListItemSettings
InitQueryParametersEx	ParamNames	"Container", "PhantomBillId"
	ParamValues	Container, null
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	zMaterialListItemSettings.QueryName
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::MaterialList
if	Condition	CalledByTransactionTask && GetListCount(CalledByTransactionTask.TaskMaterials) > 0
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"MaterialListItemId"
	IsList	true
	FieldType	
	FieldName	ServiceDetailsTmp
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"QtyIssued"
	IsList	true
	FieldType	
	FieldName	QtyIssuedListTmp
	UseFieldDataType	true
setfieldex	From	0
	To	CLF::i
foreach	Var	CLF::Item
	List	QtyIssuedListTmp
setfieldex	From	CLF::Item
	To	ServiceDetailsTmp[CLF::i].QtyIssued
setfieldex	From	CLF::i + 1
	To	CLF::i
call	CDO	
	Event	ApplyTaskMaterialList

	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	IsERPMaterialList
ConvertCDOListToResultset	List	ServiceDetailsTmp
	Fields	Name,InstanceId,Product.Name,Product.Revision,Description,QtyRequired,QtyIssued,UOM?UOM.Name:UOM,AsseRouteStep.InstanceId:RouteStep,SetupQty,ReferenceDesignator,RouteStep?RouteStep.Name:RouteStep
	ColNames	"MaterialListItem","MaterialListItemId","ComponentProduct","ProductRevision","ComponentDescription","QtyReq
	ColLabels	
	ResultsetStartRow	0
	ResultsetSizeLimit	-1
	FilterFieldName	
	FilterParams	
	Result	CLF::MaterialList
ConvertCDOListToResultset	List	ServiceDetailsTmp
	Fields	Name,InstanceId,Product.Name,Product.Revision,Description,QtyRequired,QtyIssued,UOM?UOM.Name:UOM,AsseSpec.InstanceId:Spec,SetupQty,ReferenceDesignator,Spec?Spec.Name:Spec
	ColNames	"MaterialListItem","MaterialListItemId","ComponentProduct","ProductRevision","ComponentDescription","QtyReq
	ColLabels	
	ResultsetStartRow	0
	ResultsetSizeLimit	-1
	FilterFieldName	
	FilterParams	
	Result	CLF::MaterialList
return	Value	CLF::MaterialList
return	Value	null

Function	Parameter	
call	CDO	
	Event	SetMaterialListItemSettings
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
ValidateContainerOpen	Container	Container
call	CDO	
	Event	ValidateHoldStatus
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	zMaterialListItemSettings
InitQueryParametersEx	ParamNames	"Container", "PhantomBillId"
	ParamValues	Container, null
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	zMaterialListItemSettings.QueryName
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::MaterialList
if	Condition	ApplyStepFilter OR AutoLoadPhantomMaterials OR (CalledByTransactionTask && GetListCount(CalledByTransac
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"MaterialListItemId"
	IsList	true
	FieldType	
	FieldName	ServiceDetailsTmp

	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::MaterialList
	ColName	"QtyIssued"
	IsList	true
	FieldType	
	FieldName	QtyIssuedListTmp
	UseFieldDataType	true
setfieldex	From	0
	To	CLF::i
setfieldex	From	false
	To	CLF::AnyPhantoms
foreach	Var	CLF::Item
	List	QtyIssuedListTmp
setfieldex	From	CLF::Item
	To	ServiceDetailsTmp[CLF::i].QtyIssued
if	Condition	ServiceDetailsTmp[CLF::i].Product.IsPhantom
setfieldex	From	true
	To	CLF::AnyPhantoms
setfieldex	From	CLF::i + 1
	To	CLF::i
if	Condition	AutoLoadPhantomMaterials && CLF::AnyPhantoms
setfieldex	From	0
	To	CLF::i
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count
while	Condition	CLF::i < CLF::Count
setfieldex	From	ServiceDetailsTmp[CLF::i]
	To	CLF::Item
if	Condition	CLF::Item.Product.IsPhantom
setfieldex	From	CLF::Item.Product.ResolvedBOM
	To	CLF::BOM
DeleteListItem	ListField	ServiceDetailsTmp
	ItemValue	CLF::Item
	ItemIndex	
if	Condition	CLF::BOM
setfieldex	From	StringFromObject(CLF::BOM)
	To	PhantomBillId
call	CDO	
	Event	CLight_Load_PhantomList
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
MergeLists	Lists	ServiceDetailsTmp,PhantomIssueDetailsTmp
	Result	ServiceDetailsTmp
setfieldex	From	CLF::i + 1
	To	CLF::i
setfieldex	From	GetListCount(ServiceDetailsTmp)
	To	CLF::Count
call	CDO	
	Event	ApplyTaskMaterialList
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	ApplyStepFilter && IsERPMaterialList
ComponentIssue_FilterMaterialList	MaterialList	ServiceDetailsTmp
	StepIssuePoint	Container.WorkflowStep.RouteStep
	ResultMaterialList	ServiceDetailsTmp
elseif	Condition	ApplyStepFilter
ComponentIssue_FilterMaterialList	MaterialList	ServiceDetailsTmp
	StepIssuePoint	Container.Spec
	ResultMaterialList	ServiceDetailsTmp
if	Condition	IsERPMaterialList
ConvertCDOListToResultset	List	ServiceDetailsTmp

	Fields	Name,InstanceId,Product.Name,Product.Revision,Description,QtyRequired,QtyIssued,UOM?UOM.Name:UOM,A RouteStep.Name:RouteStep,ActualQtyRequired,ObjectTypeName
	ColNames	"MaterialListItem","MaterialListItemId","ComponentProduct","ProductRevision","ComponentDescription","QtyR
	ColLabels	
	ResultSetStartRow	0
	ResultSetSizeLimit	-1
	FilterFieldName	
	FilterParams	
	Result	CLF::MaterialList
ConvertCDOListToResultSet	List	ServiceDetailsTmp
	Fields	Name,InstanceId,Product.Name,Product.Revision,Description,QtyRequired,QtyIssued,UOM?UOM.Name:UOM,A Spec.Name:Spec,ActualQtyRequired,ObjectTypeName
	ColNames	"MaterialListItem","MaterialListItemId","ComponentProduct","ProductRevision","ComponentDescription","QtyR
	ColLabels	
	ResultSetStartRow	0
	ResultSetSizeLimit	-1
	FilterFieldName	
	FilterParams	
	Result	CLF::MaterialList
return	Value	CLF::MaterialList
return	Value	null

CLF3606

CLF3606

CurrentDay_OGV

CurrentDay_OGV

New Functions		
Function	Parameter	Value
DatePart	Date	TxnDate
	DatePart	"day"
	PartOfDate	CLF::Day
StrPadLeft	String	String(CLF::Day)
	TotalWidth	2
	PaddingChar	"0"
	Result	CLF::Day
return	Value	CLF::Day

CurrentMonth_OGV

CurrentDate_OGV

New Functions		
Function	Parameter	Value
DatePart	Date	TxnDate
	DatePart	"month"
	PartOfDate	CLF::Month
StrPadLeft	String	String(CLF::Month)
	TotalWidth	2
	PaddingChar	"0"
	Result	CLF::Month
return	Value	CLF::Month

CurrentYear_OGV

CurrentDate_OGV

New Functions		
Function	Parameter	Value
DatePart	Date	TxnDate
	DatePart	"year"
	PartOfDate	CLF::Year
return	Value	String(CLF::Year)

DataPointDetals_EvaluateLimits

Base Functions		
Function	Parameter	Value
if	Condition	not (CLF::Limit)

return	Value	null
if	Condition	CLF::DataType == Transaction::__Const.DataType.Boolean
if	Condition	StrCompare(CLF::Limit,BooleanTrue,1) == 0
return	Value	"True"
elseif	Condition	StrCompare(CLF::Limit,BooleanFalse,1) == 0
return	Value	"False"
throw	ErrorId	LabelId::Computation_UpperLowerLimitsEvaluationError
	ParamNames	
	ParamValues	
setfieldex	From	false
	To	CLF::ConvError
StrSplit	String	CLF::Limit
	Separators	" "
	Result	CLF::SeparatorResult
StrJoin	StringList	CLF::SeparatorResult
	Delimiter	","
	Result	CLF::TempLimit
Trivial	Expression	CLF::TempLimit
	ToType	CLF::DataType
	IgnoreErrors	false
	Result	CLF::EvalResult
catch	ErrorMsg	CLF::ErrorMfg
	ErrorId	CLF::ErrorId
setfieldex	From	true
	To	CLF::ConvError
if	Condition	CLF::ConvError
EvalParamIndirect	ExprLocation	CLF::Limit
	Context	GetCurrentService()
	IgnoreNullObjectReferences	(CLF::DataType == Transaction::__Const.DataType.String) ? true : false
	Result	CLF::EvalResult
catch	ErrorMsg	
	ErrorId	
throw	ErrorId	LabelId::Computation_UpperLowerLimitsEvaluationError
	ParamNames	
	ParamValues	
setfieldex	From	CLF::EvalResult
	To	CLF::Limit
return	Value	CLF::Limit

New Functions		
Function	Parameter	Value
if	Condition	not (CLF::Limit)
return	Value	null
//	Line1	boolean treated as special case
	Line2	
	Line3	
	Line4	
if	Condition	CLF::DataType == Transaction::__Const.DataType.Boolean
if	Condition	StrCompare(CLF::Limit,BooleanTrue,1) == 0
return	Value	"True"
elseif	Condition	StrCompare(CLF::Limit,BooleanFalse,1) == 0
return	Value	"False"
throw	ErrorId	LabelId::Computation_UpperLowerLimitsEvaluationError
	ParamNames	
	ParamValues	
setfieldex	From	false
	To	CLF::ConvError
setfieldex	From	null
	To	CLF::EvalResult
setfieldex	From	false
	To	CLF::ValidEval
EvalParamIndirect	ExprLocation	CLF::Limit
	Context	GetCurrentService()
	IgnoreNullObjectReferences	(CLF::DataType == Transaction::__Const.DataType.String) ? true : false
	Result	CLF::EvalResult
setfieldex	From	true
	To	CLF::ValidEval
catch	ErrorMsg	CLF::ErrorMfg
	ErrorId	CLF::ErrorId
setfieldex	From	true
	To	CLF::ConvError
if	Condition	CLF::ConvError OR NOT(CLF::EvalResult)
setfieldex	From	false
	To	CLF::ConvError
StrSplit	String	CLF::Limit
	Separators	" "
	Result	CLF::SeparatorResult

StrJoin	StringList	CLF::SeparatorResult
	Delimiter	". "
	Result	CLF::TempLimit
Trivial	Expression	CLF::TempLimit
	ToType	CLF::DataType
	IgnoreErrors	false
	Result	CLF::EvalResult
catch	ErrorMsg	CLF::ErrorMfg
	ErrorId	CLF::ErrorId
if	Condition	NOT(CLF::ValidEval)
setfieldex	From	true
	To	CLF::ConvError
if	Condition	CLF::ConvError
throw	ErrorId	LabelId::Computation_UpperLowerLimitsEvaluationError
	ParamNames	
	ParamValues	
setfieldex	From	CLF::EvalResult
	To	CLF::Limit
return	Value	CLF::Limit

DataPointSummary_Validate

Base Functions		
Function	Parameter	Value
SortList	ListFieldExpr	Parent.DataCollectionDef.DataPoints
	Field1	RowPosition
	IsAscending	true
	Field2	ColumnPosition
	IsAscending2	true
	IgnoreNullObjectReferences	false
	Result	CLF::SortedDataPoints
setfieldex	From	false
	To	CLF::ScaleEnabled
foreach	Var	CLF::DataPoint
	List	CLF::SortedDataPoints
if	Condition	CLF::DataPoint.IsRequired
findlistitem	List	DataPointDetails
	ListKey	DataPoint
	KeyValue	CLF::DataPoint
	Result	CLF::FoundDataPoint
	ListIndex	
ValidateNotEqual	LeftValue	CLF::FoundDataPoint
	RightValue	null
	ErrorID	LabelId::MissingDataPoint
	ErrorParmName1	Name
	ErrorParmValue1	CLF::DataPoint.Name
	ErrorParmName2	
	ErrorParmValue2	
	ErrorParmName3	
	ErrorParmValue3	
	ErrorParmName4	
	ErrorParmValue4	
if	Condition	CLF::ScaleEnabled != true && String(CLF::DataPoint.DecimalScale) != ""
setfieldex	From	true
	To	CLF::ScaleEnabled
if	Condition	CLF::ScaleEnabled
setfieldex	From	null
	To	Transaction::FailedDataPoints
foreach	Var	CLF::Detail
	List	DataPointDetails
call	CDO	
	Event	ValidateDecimalScale
	Name1	Detail
	Value1	CLF::Detail
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::FailedDataPoints != null
throw	ErrorId	LabelId::Lbl_DecimalScaleExtendValidation
	ParamNames	"Name"

	ParamValues	Transaction::FailedDataPoints
--	-------------	-------------------------------

New Functions		
Function	Parameter	Value
SortList	ListFieldExpr	Parent.DataCollectionDef.DataPoints
	Field1	RowPosition
	IsAscending	true
	Field2	ColumnPosition
	IsAscending2	true
	IgnoreNullObjectReferences	false
	Result	CLF::SortedDataPoints
setfieldex	From	false
	To	CLF::ScaleEnabled
foreach	Var	CLF::DataPoint
	List	CLF::SortedDataPoints
if	Condition	CLF::DataPoint.IsRequired
findlistitem	List	DataPointDetails
	ListKey	DataPoint
	KeyValue	CLF::DataPoint
	Result	CLF::FoundDataPoint
	ListIndex	
ValidateNotEqual	LeftValue	CLF::FoundDataPoint
	RightValue	null
	ErrorID	LabelId::MissingDataPoint
	ErrorParmName1	Name
	ErrorParmValue1	CLF::DataPoint.Name
	ErrorParmName2	
	ErrorParmValue2	
	ErrorParmName3	
	ErrorParmValue3	
	ErrorParmName4	
	ErrorParmValue4	
if	Condition	CLF::ScaleEnabled != true && String(CLF::DataPoint.DecimalScale) != ""
setfieldex	From	true
	To	CLF::ScaleEnabled
if	Condition	CLF::ScaleEnabled
setfieldex	From	null
	To	Transaction::FailedDataPoints
foreach	Var	CLF::Detail
	List	DataPointDetails
call	CDO	
	Event	ValidateDecimalScale
	Name1	Detail
	Value1	CLF::Detail
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::FailedDataPoints != null
throw	ErrorId	LabelId::Lbl_DecimalScaleExtendValidation
	ParamNames	"Name"
	ParamValues	Transaction::FailedDataPoints
call	CDO	
	Event	DataPointSummary_ValidateResource
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	

DataPointSummary_ValidateResource

DataPointSummary_ValidateResource

New Functions		
Function	Parameter	Value
foreach	Var	CLF::Detail

	List	DataPointDetails
GetCDOType	CDO	CLF::Detail.DataPoint
	Type	CLF::CDOId
	TypeName	CLF::CDOType
if	Condition	CLF::CDOType == "ObjectDataPoint"
if	Condition	CLF::Detail.ValidateResource && (CLF::Detail.ObjectValueCDOType == "Resource" OR CLF::Detail.ObjectValueCDOType == "Carrier" OR CLF::Detail.ObjectValueCDOType == "Part" OR CLF::Detail.ObjectValueCDOType == "Scale" OR CLF::Detail.ObjectValueCDOType == "Tool")
if	Condition	CLF::Detail.NDOValue.Availability != 1
throw	ErrorId	LabelId::ResourceNotAvailable
	ParamNames	"Name"
	ParamValues	CLF::Detail.NDOValue.DisplayName
call	CDO	CLF::Detail.NDOValue
	Event	ValidateResourceMaintenance
	Name1	ServiceQty
	Value1	0
	Name2	ServiceQty2
	Value2	0
	Name3	ThrowErrors
	Value3	true
	Name4	
	Value4	
	Name5	
	Value5	

ExecuteProdProcess_AddCompletedTask

Base Functions		
Function	Parameter	Value
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	Task
	Result	Transaction::MatchingCompletedTask
	ListIndex	
if	Condition	Transaction::MatchingCompletedTask == Null
CreateListItem	ListCDO	Container.CurrentStatus
	ListField	ContainerCompletedTasks
	ItemType	
	ItemValue	Transaction::MatchingCompletedTask
	ItemIndex	
setfieldex	From	Container.CurrentStatus
	To	Transaction::MatchingCompletedTask.Parent
setfieldex	From	0
	To	Transaction::MatchingCompletedTask.NbrTimesProcessed
setfieldex	From	Task
	To	Transaction::MatchingCompletedTask.Task

New Functions		
Function	Parameter	Value
setfieldex	From	null
	To	Transaction::MatchingCompletedTask
setfieldex	From	Container.WorkflowStep
	To	CLF::Step
foreach	Var	CLF::Item
	List	Container.CurrentStatus.ContainerCompletedTasks
setfieldex	From	CLF::Item.WorkflowStep
	To	CLF::TaskStep
if	Condition	CLF::Item.Task == Task && CLF::TaskStep == CLF::Step
setfieldex	From	CLF::Item
	To	Transaction::MatchingCompletedTask
if	Condition	Transaction::MatchingCompletedTask == Null
CreateListItem	ListCDO	Container.CurrentStatus
	ListField	ContainerCompletedTasks
	ItemType	
	ItemValue	Transaction::MatchingCompletedTask
	ItemIndex	
setfieldex	From	Container.CurrentStatus
	To	Transaction::MatchingCompletedTask.Parent
setfieldex	From	0
	To	Transaction::MatchingCompletedTask.NbrTimesProcessed
setfieldex	From	Task
	To	Transaction::MatchingCompletedTask.Task
setfieldex	From	Container.WorkflowStep
	To	Transaction::MatchingCompletedTask.WorkflowStep
setfieldex	From	ElectronicProcedure
	To	Transaction::MatchingCompletedTask.ElectronicProcedure

ExecuteProdProcess_FinishModifyingEntity

Base Functions		
Function	Parameter	Value
//	Line1	If data points were collected and any
	Line2	had a limit violation without a limit override,
	Line3	then this service does not pass
	Line4	
if	Condition	ParametricData
if	Condition	IsObjectType(ParametricData,"DataPointSummary",false)
foreach	Var	CLF::DataPointDetail
	List	ParametricData.DataPointDetails
if	Condition	CLF::DataPointDetail.IsLimitViolation and not(CLF::DataPointDetail.IsLimitOverride)
setfieldex	From	false
	To	Pass
if	Condition	Workstation and (Workstation.ClearanceLevel == Transaction::__Const.ClearanceLevel.Container)
call	CDO	
	Event	ClearStationContainerClearance
	Name1	WorkStation
	Value1	Workstation
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
	Condition	(WorkStation) And Workstation.ClearanceLevel == Transaction::__Const.ClearanceLevel.MfgOrder
	CDO	
	Event	ClearStationMfgOrderClearance
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
//	Line1	Copy the information to the completed
	Line2	Task.
	Line3	
	Line4	
call	CDO	
	Event	UpdateCompletedTask
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
	Value5	
	Value5	
	Value5	
	Value5	

New Functions		
Function	Parameter	Value
if	Condition	GetListCount(IssueActualDetails) > 0
CreateCDO	CDOTypeName	
	UseFieldType	true
	OverwriteExistingCDO	true
	NewCDO	ComponentIssue
mapex	MapFrom	CLF::__CDOID
	MapTo	ComponentIssue
	MapID	ContainerTxnToContainerTxn
if	Condition	IsFieldDefined("Resource",ComponentIssue)
setfieldex	From	Resource
	To	ComponentIssue.Resource
foreach	Var	CLF::Item
	List	IssueActualDetails
Clone	From	CLF::Item
	To	CLF::ClonedItem

setfieldex	From	ComponentIssue
	To	CLF::ClonedItem.Parent
AddListItem	ListField	ComponentIssue.IssueActualDetails
	Item	CLF::ClonedItem
	Result	
call	CDO	
	Event	ExecuteService
	Name1	SubService
	Value1	ComponentIssue
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
//	Line1	If data points were collected and any
	Line2	had a limit violation without a limit override,
	Line3	then this service does not pass
	Line4	
if	Condition	ParametricData
if	Condition	IsOfObjectType(ParametricData,"DataPointSummary",false)
foreach	Var	CLF::DataPointDetail
	List	ParametricData.DataPointDetails
if	Condition	CLF::DataPointDetail.IsLimitViolation and not(CLF::DataPointDetail.IsLimitOverride)
setfieldex	From	false
	To	Pass
if	Condition	Workstation and (Workstation.ClearanceLevel == Transaction::__Const.ClearanceLevel.Container)
call	CDO	
	Event	ClearStationContainerClearance
	Name1	WorkStation
	Value1	Workstation
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	(WorkStation) And Workstation.ClearanceLevel == Transaction::__Const.ClearanceLevel.MfgOrder
call	CDO	
	Event	ClearStationMfgOrderClearance
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
//	Line1	Copy the information to the completed
	Line2	Task.
	Line3	
	Line4	
call	CDO	
	Event	UpdateCompletedTask
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	

ExecuteProdProcess_LoadProcessListDetails

Base Functions		
Function	Parameter	Value

if	Condition	Container.Spec.ElectronicProcedure
foreach	Var	CLF::TaskList
	List	Container.Spec.ElectronicProcedure.EProcedureDetails
foreach	Var	CLF::Task
	List	CLF::TaskList.TaskList.Tasks
CreateListItem	ListCDO	CLF:: _CDOID
	ListField	ProcessListDetails
	ItemType	"ProcessListDetails"
	ItemValue	CLF::TLD
	ItemIndex	
setfieldex	From	CLF::Task
	To	CLF::TLD.ProcessItem

New Functions		
Function	Parameter	Value
if	Condition	Container.Spec.ElectronicProcedure
foreach	Var	CLF::TaskList
	List	Container.Spec.ElectronicProcedure.EProcedureDetails
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::TaskList.TaskList.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::__Return == false
foreach	Var	CLF::Task
	List	CLF::TaskList.TaskList.Tasks
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::Task.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::__Return == false
CreateListItem	ListCDO	CLF:: _CDOID
	ListField	ProcessListDetails
	ItemType	"ProcessListDetails"
	ItemValue	CLF::TLD
	ItemIndex	
setfieldex	From	CLF::Task
	To	CLF::TLD.ProcessItem

MaxIterationsExpression_OGV

MaxIterationsExpression_OGV

New Functions		
Function	Parameter	Value
setfieldex	From	null
	To	CLF::Result
if	Condition	MaxIterationsExpression != null
EvalParamIndirect	ExprLocation	MaxIterationsExpression
	Context	CLF::__CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
Trivial	Expression	CLF::Result
	ToType	Transaction::__Const.DataType.Integer
	IgnoreErrors	true
	Result	CLF::Result
catch	ErrorMsg	
	ErrorId	
setfieldex	From	null
	To	CLF::Result
if	Condition	CLF::Result != null

return	Value	CLF::Result
return	Value	CLF::__FieldValue

MinIterationsExpression_OGV

MaxIterationsExpression_OGV

New Functions		
Function	Parameter	Value
setfieldex	From	null
	To	CLF::Result
if	Condition	MinIterationsExpression != null
EvalParamIndirect	ExprLocation	MinIterationsExpression
	Context	CLF::__CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
Trivial	Expression	CLF::Result
	ToType	Transaction::__Const.DataType.Integer
	IgnoreErrors	true
	Result	CLF::Result
catch	ErrorMsg	
	ErrorId	
setfieldex	From	null
	To	CLF::Result
if	Condition	CLF::Result != null
return	Value	CLF::Result
return	Value	CLF::__FieldValue

MoveStd_CheckEProcedureComplete

Validate that all required tasks have been completed (Passed) and that each task has been executed the minimum number of times as defined in the task. The container is also checked to make sure it is not being processed at any other workstation.

New Functions		
Function	Parameter	Value
setfieldex	From	true
	To	CLF::Return
if	Condition	Container.Spec.ElectronicProcedure
//	Line1	Validate that all required Tasks are
	Line2	completed successfully.
	Line3	
	Line4	
foreach	Var	CLF::EProcDetail
	List	Container.Spec.ElectronicProcedure.EProcedureDetails
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::EProcDetail.TaskList.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
	Condition	!Transaction::__Return
	Var	CLF::Task
	List	CLF::EProcDetail.TaskList.Tasks
if	Condition	CLF::Task.MinIterations > 0
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::Task.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	!Transaction::__Return
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	CLF::Task

	Result	CLF::FoundTask
	ListIndex	
if	Condition	NOT(CLF::FoundTask)
setfieldex	From	false
	To	CLF::Return
elseif	Condition	CLF::FoundTask.Pass != true
setfieldex	From	false
	To	CLF::Return
elseif	Condition	(CLF::FoundTask.NbrTimesProcessed < CLF::Task.MinIterations)
setfieldex	From	false
	To	CLF::Return
if	Condition	NOT(CLF::Return)
//	Line1	Determine if the Container is being
	Line2	processed at any Workstations.
	Line3	
	Line4	
if	Condition	GetListCount(Container.CurrentStatus.ClearanceWorkstations) > 0
setfieldex	From	false
	To	CLF::Return
return	Value	CLF::Return

MoveStd_ValidateEProcedure

Base Functions		
Function	Parameter	Value
//	Line1	Validate that all required Tasks are
	Line2	completed successfully.
	Line3	
	Line4	
foreach	Var	CLF::EProcDetail
	List	Container.Spec.ElectronicProcedure.EProcedureDetails
foreach	Var	CLF::Task
	List	CLF::EProcDetail.TaskList.Tasks
if	Condition	CLF::Task.MinIterations > 0
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::FoundTask
	ListIndex	
ValidateNotEqual	LeftValue	CLF::FoundTask
	RightValue	null
	ErrorID	LabelId::RequiredTaskIncomplete
	ErrorParmName1	Name
	ErrorParmValue1	CLF::Task.Name
	ErrorParmName2	
	ErrorParmValue2	
	ErrorParmName3	
	ErrorParmValue3	
	ErrorParmName4	
	ErrorParmValue4	
validateequal	LeftValue	CLF::FoundTask.Pass
	RightValue	true
	ErrorID	LabelId::RequiredTaskFailed
	ErrorParmName1	Name
	ErrorParmValue1	CLF::Task.Name
	ErrorParmName2	
	ErrorParmValue2	
	ErrorParmName3	
	ErrorParmValue3	
	ErrorParmName4	
	ErrorParmValue4	
if	Condition	(CLF::FoundTask.NbrTimesProcessed < CLF::Task.MinIterations)
throw	ErrorId	LabelId::PrereqTaskList_MinIterationsNotMet
	ParamNames	"Name", "Name2", "Parameter1Value", "Parameter2Value"
	ParamValues	CLF::Task.Name, CLF::Task.Parent.Name, CLF::Task.MinIterations, CLF::FoundTask.NbrTimesProcessed
//	Line1	Determine if the Container is being
	Line2	processed at any Workstations.
	Line3	
	Line4	
if	Condition	GetListCount(Container.CurrentStatus.ClearanceWorkstations) > 0
setfieldex	From	"("
	To	CLF::WorkstationList
foreach	Var	CLF::Workstation
	List	Container.CurrentStatus.ClearanceWorkstations
StrConcat	String1	CLF::WorkstationList

	String2	CLF::Workstation.Name
	Result	CLF::WorkstationList
StrConcat	String1	CLF::WorkstationList
	String2	","
	Result	CLF::WorkstationList
StrConcat	String1	CLF::WorkstationList
	String2	")"
	Result	CLF::WorkstationList
throw	ErrorId	LabelId::ContainerAtWorkstation
	ParamNames	"Name"
	ParamValues	CLF::WorkstationList

New Functions		
Function	Parameter	Value
//	Line1	Validate that all required Tasks are
	Line2	completed successfully.
	Line3	
	Line4	
foreach	Var	CLF::EProcDetail
	List	Container.Spec.ElectronicProcedure.EProcedureDetails
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::EProcDetail.TaskList.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	!Transaction::__Return
foreach	Var	CLF::Task
	List	CLF::EProcDetail.TaskList.Tasks
setfieldex	From	CLF::Task.Name
	To	CLF::TaskName
setfieldex	From	CLF::Task.Sequence
	To	CLF::TaskSequence
setfieldex	From	CLF::Task
	To	CLF::Debug1
if	Condition	CLF::Task.MinIterations > 0
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::Task.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	!Transaction::__Return
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::FoundTask
	ListIndex	
setfieldex	From	CLF::FoundTask
	To	CLF::DebugFoundTask
ValidateNotEqual	LeftValue	CLF::FoundTask
	RightValue	null
	ErrorID	LabelId::RequiredTaskIncomplete
	ErrorParmName1	Name
	ErrorParmValue1	CLF::Task.Name
	ErrorParmName2	
	ErrorParmValue2	
	ErrorParmName3	
	ErrorParmValue3	
	ErrorParmName4	
	ErrorParmValue4	
validateequal	LeftValue	CLF::FoundTask.Pass
	RightValue	true
	ErrorID	LabelId::RequiredTaskFailed
	ErrorParmName1	Name

	ErrorParmValue1	CLF::Task.Name
	ErrorParmName2	
	ErrorParmValue2	
	ErrorParmName3	
	ErrorParmValue3	
	ErrorParmName4	
	ErrorParmValue4	
if	Condition	(CLF::FoundTask.NbrTimesProcessed < CLF::Task.MinIterations)
throw	ErrorId	LabelId::PrereqTaskList_MinIterationsNotMet
	ParamNames	"Name", "Name2", "Parameter1Value", "Parameter2Value"
	ParamValues	CLF::Task.Name, CLF::Task.Parent.Name, CLF::Task.MinIterations, CLF::FoundTask.NbrTimesProcessed
//	Line1	Determine if the Container is being
	Line2	processed at any Workstations.
	Line3	
	Line4	
if	Condition	GetListCount(Container.CurrentStatus.ClearanceWorkstations) > 0
setfieldex	From	"("
	To	CLF::WorkstationList
foreach	Var	CLF::Workstation
	List	Container.CurrentStatus.ClearanceWorkstations
StrConcat	String1	CLF::WorkstationList
	String2	CLF::Workstation.Name
	Result	CLF::WorkstationList
StrConcat	String1	CLF::WorkstationList
	String2	", "
	Result	CLF::WorkstationList
StrConcat	String1	CLF::WorkstationList
	String2	")"
	Result	CLF::WorkstationList
throw	ErrorId	LabelId::ContainerAtWorkstation
	ParamNames	"Name"
	ParamValues	CLF::WorkstationList

MoveTxn_CheckEProcedureComplete

MoveTxn_CheckEProcedureComplete

New Functions		
Function	Parameter	Value
return	Value	false

MoveTxn_ClearCompletedTasks

Base Functions		
Function	Parameter	Value
foreach	Var	CLF::CompletedTask
	List	Container.CurrentStatus.ContainerCompletedTasks
DeleteListItem	ListField	Container.CurrentStatus.ContainerCompletedTasks
	ItemValue	CLF::CompletedTask
	ItemIndex	
setfieldex	From	null
	To	Container.CurrentStatus.LastCompletedTask

New Functions		
Function	Parameter	Value
setfieldex	From	Container.WorkflowStep
	To	CLF::CurStep
if	Condition	CLF::CurStep.Spec.ElectronicProcedure
setfieldex	From	NOT(CLF::CurStep.Spec.ElectronicProcedure.KeepCompletedTasks)
	To	CLF::DeleteTasks
if	Condition	NOT(CLF::DeleteTasks)
call	CDO	
	Event	MoveTxn_CheckEProcedureComplete
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
setfieldex	From	Transaction::__Return
	To	CLF::DeleteTasks

if	Condition	CLF::DeleteTasks
foreach	Var	CLF::CompletedTask
	List	Container.CurrentStatus.ContainerCompletedTasks
setfieldex	From	CLF::CompletedTask.WorkflowStep
	To	CLF::TaskStep
setfieldex	From	CLF::CompletedTask.ElectronicProcedure
	To	CLF::TaskProcedure
//	Line1	If CompletedTask has no WorkflowStep or is equal to current step then delete it
	Line2	
	Line3	
	Line4	
if	Condition	(CLF::TaskStep == CLF::CurStep OR NOT(CLF::TaskStep)) AND (NOT(CLF::TaskProcedure) OR CLF::TaskProcedure == CLF::CurStep.Spec.ElectronicProcedure)
DeleteListItem	ListField	Container.CurrentStatus.ContainerCompletedTasks
	ItemValue	CLF::CompletedTask
	ItemIndex	
setfieldex	From	null
	To	Container.CurrentStatus.LastCompletedTask

OnlineQuerySetupParams_SelVal

OnlineQuerySetupParams_SelVal

New Functions		
Function	Parameter	Value
ClearList	ListCDO	CLF::__CDOID
	ListField	OnlineQuerySetupParamsTmp
if	Condition	Task
call	CDO	
	Event	Task_ProcessUserQueryDynamicValues
	Name1	Task
	Value1	Task
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
elseif	Condition	TaskList
foreach	Var	CLF::Task
	List	TaskList.Tasks
call	CDO	
	Event	Task_ProcessUserQueryDynamicValues
	Name1	Task
	Value1	CLF::Task
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
ConvertCDOListToResultset	List	OnlineQuerySetupParamsTmp
	Fields	TaskName,QueryName,Name,ResolvedValue,DynamicValue
	ColNames	"TaskName","QueryName","ParameterName","ResolvedValue","DynamicValue"
	ColLabels	
	ResultsetStartRow	CLF::__ResultsetStartRow
	ResultsetSizeLimit	CLF::__ResultsetSizeLimit
	FilterFieldName	
	FilterParams	
	Result	Transaction::__Return

Open_ProcessElectronicProcedures

Base Functions		
Function	Parameter	Value
foreach	Var	CLF::CompletedTask
	List	Container.CurrentStatus.ContainerCompletedTasks
DeleteListItem	ListField	Container.CurrentStatus.ContainerCompletedTasks
	ItemValue	CLF::CompletedTask
	ItemIndex	
setfieldex	From	Null

	To	Container.CurrentStatus.LastCompletedTask
--	----	---

New Functions		
Function	Parameter	Value
setfieldex	From	true
	To	CLF::ClearLastCompletedTask
foreach	Var	CLF::CompletedTask
	List	Container.CurrentStatus.ContainerCompletedTasks
setfieldex	From	CLF::CompletedTask.ElectronicProcedure
	To	CLF::TaskProcedure
if	Condition	NOT(CLF::TaskProcedure) OR NOT(CLF::TaskProcedure.KeepCompletedTasksOnOpen)
DeleteListItem	ListField	Container.CurrentStatus.ContainerCompletedTasks
	ItemValue	CLF::CompletedTask
	ItemIndex	
elseif	Condition	CLF::CompletedTask.Task == Container.CurrentStatus.LastCompletedTask
setfieldex	From	false
	To	CLF::ClearLastCompletedTask
if	Condition	CLF::ClearLastCompletedTask
setfieldex	From	null
	To	Container.CurrentStatus.LastCompletedTask

OperatorLog_SelValEx_Container

OperatorLog_SelValEx_Container

New Functions		
Function	Parameter	Value
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Factory"
	ParamValue1	GetCurrentService().Employee.SessionValues.Factory
	ParamName2	"Operation"
	ParamValue2	
	ParamName3	"WorkCenter"
	ParamValue3	
	ParamName4	"Spec"
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
MergeQueryParameters	Parameters1	CLF::__Parameters
	Parameters2	CLF::QueryParams
	Result	CLF::QueryParams
ExecuteQueryEx	QueryName	"DefaultDispatch_ContainerTxn"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	
	ResultsetStartRow	
	Resultset	Transaction::__Return

OperatorLog_SelValEx_ObjInstanceId

OperatorLog_SelValEx_ObjInstanceId

New Functions		
Function	Parameter	Value
InitQueryParametersEx	ParamNames	"CDODEFID"
	ParamValues	LogObjectType?LogObjectType:0
	QueryParameters	CLF::CDOQParams
ExecuteQueryEx	QueryName	"OperatorLog_GetCDOName"
	Parameters	CLF::CDOQParams
	CDODefId	0
	ResultsetSizeLimit	1
	ResultsetStartRow	1
	Resultset	CLF::CDONames
ConvertResultsetToListOrScalar	Resultset	CLF::CDONames
	ColName	"CDONAME"
	IsList	false
	FieldType	Transaction::__Const.DataType.String
	FieldName	CLF::CDOTypeName
	UseFieldType	false
CreateCDO	CDOTypeName	CLF::CDOTypeName
	UseFieldType	false
	OverwriteExistingCDO	true
	NewCDO	CLF::CDO
IsFieldDefined	FieldName	"Name"

	CDO	CLF::CDO
	Result	CLF::Exists
if	Condition	not(CLF::Exists)
return	Value	
IsFieldDefined	FieldName	"Revision"
	CDO	CLF::CDO
	Result	CLF::IsRDO
IsFieldDefined	FieldName	"IsContainer"
	CDO	CLF::CDO
	Result	CLF::IsContainer
IsFieldDefined	FieldName	"Classification"
	CDO	CLF::CDO
	Result	CLF::IsQualityObject
if	Condition	CLF::IsRDO
setfieldex	From	"SelectionValuesEx_RDO"
	To	CLF::QueryName
elseif	Condition	CLF::IsContainer && CLF::CDO.IsContainer
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Factory"
	ParamValue1	GetCurrentService().Employee.SessionValues.Factory
	ParamName2	"Operation"
	ParamValue2	
	ParamName3	"WorkCenter"
	ParamValue3	
	ParamName4	"Spec"
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
MergeQueryParameters	Parameters1	CLF::QueryParams
	Parameters2	CLF::__Parameters
	Result	CLF::QueryParams
ExecuteQueryEx	QueryName	"DefaultDispatch_ContainerTxn"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	
	ResultsetStartRow	
	Resultset	Transaction::__Return
return	Value	
elseif	Condition	CLF::IsQualityObject
setfieldex	From	"SelectionValuesEx_QualityObjectShort"
	To	CLF::QueryName
setfieldex	From	"SelectionValuesEx_NDO"
	To	CLF::QueryName
InitQueryParametersEx	ParamNames	"NameFilter", "SpecificTypeOnly"
	ParamValues	CLF::CDOTypeName,true
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	CLF::QueryName
	Parameters	CLF::__Parameters
	CDODefId	LogObjectType?LogObjectType:0
	ResultsetSizeLimit	CLF::__ResultsetSizeLimit
	ResultsetStartRow	CLF::__ResultsetStartRow
	Resultset	Transaction::__Return

OperatorLogInquiry_LogDetails_OGV

OperatorLogInquiry_LogDetails_OGV

New Functions		
Function	Parameter	Value
//	Line1	Get all log details based on the LogInstanceId or LogName field value
	Line2	
	Line3	
	Line4	
CreateCDO	CDOTypeName	"OperatorLogInfo"
	UseFieldType	false
	OverwriteExistingCDO	true
	NewCDO	CLF::LogDetails
if	Condition	OperatorLogName == null && OperatorLogId != null
ExecuteQueryString	QueryString	"select OperatorLogName from OperatorLog where OperatorLogId = " + OperatorLogId + """
	Parameters	
	ResultsetStartRow	
	ResultsetSizeLimit	
	Result	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::Resultset

	ColName	"OperatorLogName"
	IsList	false
	FieldType	
	FieldName	OperatorLogName
	UseFieldDataType	true
if	Condition	OperatorLogName != null
ResolveNamedObjRef	ObjectName	OperatorLogName
	NamedObjRefField	CLF::LogDetails.OperatorLog
	ThrowError	false
if	Condition	CLF::LogDetails.OperatorLog == null
throw	ErrorId	LabelId::OperatorLogInquiry_LogNoSpecified
	ParamNames	
	ParamValues	
setfieldex	From	CLF::LogDetails.OperatorLog.LogEntryTypeGroup
	To	CLF::LogDetails.LogEntryTypeGroup
if	Condition	GetLogEntries
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"OperatorLog"
	ParamValue1	CLF::LogDetails.OperatorLog
	ParamName2	"StartDateTime"
	ParamValue2	LocalToGMT(StartTimeFilter)
	ParamName3	"EndDateTime"
	ParamValue3	LocalToGMT(EndTimeFilter)
	ParamName4	"LogEntryTypeFilter"
	ParamValue4	LogEntryTypeFilter ? ("% " + LogEntryTypeFilter + "%") : ""
	ParamName5	"LogEntryFilter"
	ParamValue5	LogEntryFilter ? ("% " + LogEntryFilter + "%") : ""
	ParamName6	
	ParamValue6	
ExecuteQueryEx	QueryName	"OperatorLogInquiry_GetLogEntries"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::Resultset
	ColName	"RecordLogEntryHistoryId"
	IsList	true
	FieldType	
	FieldName	CLF::LogDetails.LogEntries
	UseFieldDataType	true
setfieldex	From	CLF::LogDetails
	To	LogDetails

OperatorLogInquiry_LogDetailsForObject_OGV

OperatorLogInquiry_LogDetailsForObject_OGV

New Functions		
Function	Parameter	Value
if	Condition	ObjectInstanceId == null
throw	ErrorId	LabelId::MissingFieldObjectInstanceId
	ParamNames	
	ParamValues	
CreateCDO	CDOTypeName	"LogDetailsForObject"
	UseFieldType	false
	OverwriteExistingCDO	true
	NewCDO	CLF::LogDetailsForObject
//	Line1	Set ObjectTypeName from InstanceId
	Line2	
	Line3	
	Line4	
InitQueryParametersEx	ParamNames	"ObjectInstanceId"
	ParamValues	ObjectInstanceId
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	"GetCDONameFromInstanceId"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"CDOName"
	IsList	false
	FieldType	
	FieldName	ObjectTypeName

	UseFieldDataType	true
//	Line1	Get logs associated to object
	Line2	
	Line3	
	Line4	
InitQueryParametersEx	ParamNames	"ObjectInstanceId"
	ParamValues	ObjectInstanceId
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	"GetOperatorLogsForObject"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToListOrScalar	Resultset	CLF::Resultset
	ColName	"OperatorLogId"
	IsList	true
	FieldType	
	FieldName	OperatorLogs
	UseFieldDataType	true
if	Condition	GetListCount(OperatorLogs) > 0
//	Line1	Object associated to a log. Set return data for those existing logs.
	Line2	
	Line3	
	Line4	
setfieldex	From	true
	To	CLF::LogDetailsForObject.HasLogs
foreach	Var	CLF::Log
	List	OperatorLogs
setfieldex	From	CLF::Log.InstanceID
	To	CLF::DebugLogId
CreateListItem	ListCDO	CLF::LogDetailsForObject
	ListField	OperatorLogInfos
	ItemType	
	ItemValue	CLF::LogInfo
	ItemIndex	
setfieldex	From	CLF::Log
	To	CLF::LogInfo.OperatorLog
setfieldex	From	CLF::Log.LogEntryTypeGroup
	To	CLF::LogInfo.LogEntryTypeGroup
setfieldex	From	CLF::LogDetailsForObject
	To	LogDetailsForObject

OpLog_GetAttachedDocuments_OGV

OpLog_GetAttachedDocuments_OGV

New Functions		
Function	Parameter	Value
if	Condition	OperatorLog
InitQueryParametersEx	ParamNames	"ObjectInstanceId"
	ParamValues	OperatorLog
	QueryParameters	CLF::QueryParams
ExecuteQueryEx	QueryName	"AttachDocument_GetDocuments"
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	CLF::Resultset
ConvertResultsetToObjects	Resultset	CLF::Resultset
	ObjectType	
	UseFieldType	true
	Result	AttachDocumentDetails
	IsList	true
	Method	

PartRequestAcknowledge_SetRequestType2

Base Functions		
Function	Parameter	Value
foreach	Var	ServiceDetailsItem
	List	ServiceDetails
if	Condition	ServiceDetailsItem.Part
setfieldex	From	ServiceDetailsItem.Part.ProductionStatus.PartQty + ServiceDetailsItem.PartQty
	To	ServiceDetailsItem.Part.ProductionStatus.PartQty

if	Condition	ServiceDetailsItem.ScrapQty == 0
call	CDO	ServiceDetailsItem.Part
	Event	_ValidatePartLimits
	Name1	ThrowErrors
	Value1	false
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
SumListFields	ObjectList	ServiceDetails
	Field1	ScrapQty
	Sum1	HistorySummary.ScrapQty
	Field2	
	Sum2	
setfieldconditional	Condition	Resource.ProductionStatus.PartRequestCount > 0
	ThenFrom	Resource.ProductionStatus.PartRequestCount - 1
	ElseFrom	0
	To	Resource.ProductionStatus.PartRequestCount

New Functions		
Function	Parameter	Value
foreach	Var	ServiceDetailsItem
	List	ServiceDetails
if	Condition	ServiceDetailsItem.Part
setfieldex	From	ServiceDetailsItem.Part.ProductionStatus.PartQty + ServiceDetailsItem.PartQty
	To	ServiceDetailsItem.Part.ProductionStatus.PartQty
if	Condition	ServiceDetailsItem.ScrapQty == 0
call	CDO	ServiceDetailsItem.Part
	Event	_ValidatePartLimits
	Name1	ThrowErrors
	Value1	false
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	ServiceDetailsItem.PhysicalLocation OR ServiceDetailsItem.PhysicalPosition
call	CDO	
	Event	PartTxn_PartSetup
	Name1	Part
	Value1	ServiceDetailsItem.Part
	Name2	PhysicalLocation
	Value2	ServiceDetailsItem.PhysicalLocation
	Name3	PhysicalPosition
	Value3	ServiceDetailsItem.PhysicalPosition
	Name4	SkipChangeCountValidation
	Value4	true
	Name5	
	Value5	
SumListFields	ObjectList	ServiceDetails
	Field1	ScrapQty
	Sum1	HistorySummary.ScrapQty
	Field2	
	Sum2	
setfieldconditional	Condition	Resource.ProductionStatus.PartRequestCount > 0
	ThenFrom	Resource.ProductionStatus.PartRequestCount - 1
	ElseFrom	0
	To	Resource.ProductionStatus.PartRequestCount

PartTxn_EquipmentSetup

PartTxn_PartSetupPartTxn_EquipmentSetup

Base Functions		
Function	Parameter	Value
if	Condition	(CLF::PhysicalLocation AND (CLF::Part.PhysicalLocation <> CLF::PhysicalLocation)) OR (CLF::PhysicalPosition AND (CLF::Part.PhysicalPosition <> CLF::PhysicalPosition))
CreateCDO	CDOTypeName	
	UseFieldType	true
	OverwriteExistingCDO	true

	NewCDO	EquipmentSetup
setfieldex	From	CLF::Part
	To	EquipmentSetup.Resource
setfieldex	From	CLF::SkipChangeCountValidation
	To	EquipmentSetup.SkipChangeCountValidation
mapex	MapFrom	CLF::Part
	MapTo	EquipmentSetup
	MapID	0
setfieldconditional	Condition	CLF::PhysicalLocation AND (CLF::Part.PhysicalLocation <> CLF::PhysicalLocation)
	ThenFrom	CLF::PhysicalLocation
	ElseFrom	
	To	EquipmentSetup.PhysicalLocation
setfieldconditional	Condition	CLF::PhysicalPosition AND (CLF::Part.PhysicalPosition <> CLF::PhysicalPosition)
	ThenFrom	CLF::PhysicalPosition
	ElseFrom	
	To	EquipmentSetup.PhysicalPosition
call	CDO	
	Event	ExecuteService
	Name1	SubService
	Value1	EquipmentSetup
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	

New Functions		
Function	Parameter	Value
if	Condition	(CLF::PhysicalLocation AND (CLF::Part.PhysicalLocation <> CLF::PhysicalLocation)) OR (CLF::PhysicalPosition AND (CLF::Part.PhysicalPosition <> CLF::PhysicalPosition))
CreateCDO	CDOTypeName	
	UseFieldType	true
	OverwriteExistingCDO	true
	NewCDO	PartSetup
setfieldex	From	CLF::Part
	To	PartSetup.Resource
mapex	MapFrom	CLF::Part
	MapTo	PartSetup
	MapID	PartToPartSetup
setfieldconditional	Condition	CLF::PhysicalLocation AND (CLF::Part.PhysicalLocation <> CLF::PhysicalLocation)
	ThenFrom	CLF::PhysicalLocation
	ElseFrom	
	To	PartSetup.PhysicalLocation
setfieldconditional	Condition	CLF::PhysicalPosition AND (CLF::Part.PhysicalPosition <> CLF::PhysicalPosition)
	ThenFrom	CLF::PhysicalPosition
	ElseFrom	
	To	PartSetup.PhysicalPosition
call	CDO	
	Event	ExecuteService
	Name1	SubService
	Value1	PartSetup
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	

PopulateEProcedureVariantDetails

PopulateEProcedureVariantDetails

New Functions		
Function	Parameter	Value
setfieldex	From	ProductionProcess.EProcedureDetails
	To	ProductionProcess.EProcedureVariantDetails
foreach	Var	CLF::Item
	List	ProductionProcess.EProcedureVariantDetails
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::Item.TaskList.VariantExpression

EvalParamIndirect	ExprLocation	CLF::Item.TaskList.VariantExpression
	Context	CLF::__CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result == false
DeleteListItem	ListField	ProductionProcess.EProcedureVariantDetails
	ItemValue	CLF::Item
	ItemIndex	

PopulateVariantTasks

PopulateVariantTasks

New Functions		
Function	Parameter	Value
if	Condition	ProcessList && GetListCount(ProcessList.Tasks) > 0
setfieldex	From	ProcessList.Tasks
	To	ProcessList.VariantTasks
if	Condition	GetListCount(ProcessList.VariantTasks) > 0
foreach	Var	CLF::Task
	List	ProcessList.VariantTasks
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::Task.VariantExpression
EvalParamIndirect	ExprLocation	CLF::Task.VariantExpression
	Context	CLF::__CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result == false
DeleteListItem	ListField	ProcessList.VariantTasks
	ItemValue	CLF::Task
	ItemIndex	

ProcessSeqValidation_AreBatchTasksPending

Base Functions		
Function	Parameter	Value
foreach	Var	CLF::Task
	List	CLF::TaskList.Tasks
if	Condition	CLF::Task.TaskUsage == Transaction::__Const.TaskUsage.Batch
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::CompletedTask
	ListIndex	
if	Condition	(not(CLF::CompletedTask) and CLF::Task.MinIterations > 0)
return	Value	True
if	Condition	CLF::CompletedTask
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::CompletedTask
	Name2	TaskIsRequired
	Value2	(CLF::Task.MinIterations > 0)
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	False
if	Condition	Transaction::__Return == false
return	Value	True
return	Value	False

New Functions		
Function	Parameter	Value
foreach	Var	CLF::Task
	List	CLF::TaskList.Tasks
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::Task.VariantExpression
	Name2	
	Value2	
	Name3	

	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::__Return == true && CLF::Task.TaskUsage == Transaction::__Const.TaskUsage.Batch
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::CompletedTask
	ListIndex	
if	Condition	(not(CLF::CompletedTask) and CLF::Task.MinIterations > 0)
return	Value	True
if	Condition	CLF::CompletedTask
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::CompletedTask
	Name2	TaskIsRequired
	Value2	(CLF::Task.MinIterations > 0)
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	False
if	Condition	Transaction::__Return == false
return	Value	True
return	Value	False

ProcessSeqValidation_GetCompletedTasks

Base Functions		
Function	Parameter	Value
if	Condition	(LastCompletedTask == Null) AND (Container.CurrentStatus.LastCompletedTask != null)
setfieldex	From	Container.CurrentStatus.LastCompletedTask
	To	LastCompletedTask
if	Condition	ProcessList == Container.CurrentStatus.LastCompletedTask.Parent
setfieldex	From	Container.CurrentStatus.LastCompletedTask
	To	LastCompletedTaskInTaskList
for	Var	CLF::Index
	Init	0
	Condition	CLF::Index < GetListCount(ProcessList.Tasks)
	Inc	1
setfieldex	From	null
	To	CLF::Task
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	ProcessList.Tasks[CLF::Index]
	Result	CLF::Task
	ListIndex	
if	Condition	CLF::Task
setfieldex	From	ProcessList.Tasks[CLF::Index]
	To	LastCompletedTaskInTaskList
foreach	Var	CLF::ContainerCompletedTask
	List	Container.CurrentStatus.ContainerCompletedTasks
CreateListItem	ListCDO	CLF::__CDOID
	ListField	CompletedTaskDetails
	ItemType	
	ItemValue	CLF::CompletedTaskDetail
	ItemIndex	CLF::Index
mapex	MapFrom	CLF::ContainerCompletedTask
	MapTo	CLF::CompletedTaskDetail
	MapID	ContainerCompletedTaskToCompletedTaskDetail

New Functions		
Function	Parameter	Value
if	Condition	(LastCompletedTask == Null) AND (Container.CurrentStatus.LastCompletedTask != null)
setfieldex	From	Container.CurrentStatus.LastCompletedTask
	To	LastCompletedTask
if	Condition	ProcessList == Container.CurrentStatus.LastCompletedTask.Parent
setfieldex	From	Container.CurrentStatus.LastCompletedTask
	To	LastCompletedTaskInTaskList
for	Var	CLF::Index
	Init	0

	Condition	CLF::Index<GetListCount(ProcessList.Tasks)
	Inc	1
setfieldex	From	null
	To	CLF::Task
findlistitem	List	Container.CurrentStatus.ContainerCompletedTasks
	ListKey	Task
	KeyValue	ProcessList.Tasks[CLF::Index]
	Result	CLF::Task
	ListIndex	
if	Condition	CLF::Task
setfieldex	From	ProcessList.Tasks[CLF::Index]
	To	LastCompletedTaskInTaskList
setfieldex	From	Container.Spec.ElectronicProcedure
	To	CLF::SpecProcedure
setfieldex	From	Container.WorkflowStep
	To	CLF::Step
setfieldex	From	null
	To	CLF::LastCompletedTaskInTaskList
foreach	Var	CLF::ContainerCompletedTask
	List	Container.CurrentStatus.ContainerCompletedTasks
setfieldex	From	CLF::ContainerCompletedTask.ElectronicProcedure
	To	CLF::TaskProcedure
setfieldex	From	CLF::ContainerCompletedTask.WorkflowStep
	To	CLF::TaskStep
if	Condition	(NOT(CLF::TaskProcedure) OR CLF::TaskProcedure == CLF::SpecProcedure) AND (NOT(CLF::TaskStep) OR CLF::TaskStep == CLF::Step)
if	Condition	NOT(CLF::LastCompletedTaskInTaskList) OR CLF::ContainerCompletedTask.CompletionTimestampGMT > CLF::LastCompletedTaskInTaskList.CompletionTimestampGMT
setfieldex	From	CLF::ContainerCompletedTask
	To	CLF::LastCompletedTaskInTaskList
setfieldex	From	CLF::ContainerCompletedTask.Task.Name
	To	CLF::TaskName
setfieldex	From	CLF::ContainerCompletedTask.Task.Sequence
	To	CLF::TaskSequence
CreateListItem	ListCDO	CLF:: __CDOID
	ListField	CompletedTaskDetails
	ItemType	
	ItemValue	CLF::CompletedTaskDetail
	ItemIndex	CLF::Index
mapex	MapFrom	CLF::ContainerCompletedTask
	MapTo	CLF::CompletedTaskDetail
	MapID	ContainerCompletedTaskToCompletedTaskDetail
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	LastCompletedTask
	Result	CLF::Task
	ListIndex	
if	Condition	CLF::LastCompletedTaskInTaskList
setfieldex	From	CLF::LastCompletedTaskInTaskList.Task
	To	LastCompletedTaskInTaskList
setfieldex	From	null
	To	LastCompletedTaskInTaskList
if	Condition	GetListCount(CompletedTaskDetails) == 0 OR NOT(LastCompletedTaskInTaskList)
setfieldex	From	null
	To	LastCompletedTask
setfieldex	From	null
	To	LastCompletedTaskInTaskList

ProcessSeqValidation_ValidatePrerequisiteTaskList

Base Functions		
Function	Parameter	Value
if	Condition	ProcessList.PrerequisiteTaskList <> null
foreach	Var	CLF::TaskListDetail
	List	ProcessList.PrerequisiteTaskList.Tasks
if	Condition	CLF::TaskListDetail.MinIterations > 0
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::TaskListDetail
	Result	CLF::FoundTask
	ListIndex	
if	Condition	CLF::FoundTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::PrereqTaskListIncomplete
	ParamNames	"Name", "Name2"
	ParamValues	CLF::TaskListDetail.Name, ProcessList.PrerequisiteTaskList.Name

return	Value	False
//	Line1	Verify the criteria for a task has been
	Line2	completed.
	Line3	
	Line4	
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::FoundTask
	Name2	TaskIsRequired
	Value2	True
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	True
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::_Return == False
return	Value	False
return	Value	True

New Functions		
Function	Parameter	Value
if	Condition	ProcessList.PrerequisiteTaskList <> null
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	ProcessList.PrerequisiteTaskList.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::_Return == true
foreach	Var	CLF::TaskListDetail
	List	ProcessList.PrerequisiteTaskList.Tasks
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::TaskListDetail.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::_Return == false
if	Condition	CLF::TaskListDetail.MinIterations > 0
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::TaskListDetail
	Result	CLF::FoundTask
	ListIndex	
if	Condition	CLF::FoundTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::PrereqTaskListIncomplete
	ParamNames	"Name", "Name2"
	ParamValues	CLF::TaskListDetail.Name, ProcessList.PrerequisiteTaskList.Name
return	Value	False
//	Line1	Verify the criteria for a task has been
	Line2	completed.
	Line3	
	Line4	
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::FoundTask
	Name2	TaskIsRequired
	Value2	True
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList

	Value4	True
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::_Return == False
return	Value	False
return	Value	True

ProcessSeqValidation_ValidatePrerequisiteTasks

Base Functions		
Function	Parameter	Value
foreach	Var	CLF::Task
	List	ProcessItem.PrerequisiteTasks
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::FoundTask
	ListIndex	
if	Condition	CLF::FoundTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::PrereqTaskNotComplete
	ParamNames	"Name"
	ParamValues	CLF::Task.Name
return	Value	False
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::FoundTask
	Name2	TaskIsRequired
	Value2	True
	Name3	IsPrerequisiteTask
	Value3	True
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::_Return == False
return	Value	False
return	Value	True

New Functions		
Function	Parameter	Value
foreach	Var	CLF::Task
	List	ProcessItem.PrerequisiteTasks
call	CDO	
	Event	ValidateVariantExpression
	Name1	VariantExpression
	Value1	CLF::Task.VariantExpression
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::_Return == false
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::FoundTask
	ListIndex	
if	Condition	CLF::FoundTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::PrereqTaskNotComplete
	ParamNames	"Name"
	ParamValues	CLF::Task.Name
return	Value	False
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::FoundTask
	Name2	TaskIsRequired
	Value2	True
	Name3	IsPrerequisiteTask

	Value3	True
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::__Return == False
return	Value	False
return	Value	True

ProcessSeqValidation_ValidateTaskListSequence

Base Functions		
Function	Parameter	Value
//	Line1	No need to validate if the TaskList
	Line2	hasn't changed.
	Line3	
	Line4	
findlistitem	List	ProductionProcess.EProcedureDetails
	ListKey	TaskList
	KeyValue	ProcessList
	Result	CLF::NewTaskList
	ListIndex	CLF::NewTaskListIndex
if	Condition	LastCompletedTask
findlistitem	List	ProductionProcess.EProcedureDetails
	ListKey	TaskList
	KeyValue	LastCompletedTask.Parent
	Result	CLF::LastTaskList
	ListIndex	CLF::LastTaskListIndex
if	Condition	not (CLF::LastTaskList) or not (CLF::NewTaskList) or (CLF::LastTaskList.Sequence = CLF::NewTaskList.Sequence)
return	Value	True
if	Condition	ProductionProcess.ExecutionMode == Transaction::__Const.ExecutionMode.Sequential
call	CDO	
	Event	GetInitialTaskList
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
setfieldex	From	Transaction::__Return
	To	CLF::InitialTaskList
if	Condition	LastCompletedTask
//	Line1	If this is a new TaskList, verify that all
	Line2	Tasks in the previous TaskList were
	Line3	processed successfully.
	Line4	
call	CDO	
	Event	GetTasksInConcurrentTaskLists
	Name1	Sequence
	Value1	CLF::LastTaskList.Sequence
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
foreach	Var	CLF::Task
	List	Transaction::__Return
//	Line1	Where the Container can start midway
	Line2	through a Procedure, Container level
	Line3	Tasks are not required in the earlier
	Line4	TaskLists
if	Condition	((CLF::InitialTaskList) and (CLF::LastTaskList.Sequence < CLF::InitialTaskList.Sequence) and (CLF::Task.TaskUsage == Transaction::__Const.TaskUsage.Container))
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::CompletedTask
	ListIndex	CLF::TaskIndex
if	Condition	CLF::Task.MinIterations > 0

if	Condition	CLF::CompletedTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskIncompleteTaskList
	ParamNames	"Name"
	ParamValues	CLF::Task.Parent.Name
return	Value	False
if	Condition	CLF::CompletedTask
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::CompletedTask
	Name2	TaskIsRequired
	Value2	(CLF::Task.MinIterations > 0)
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::__Return == False
return	Value	False
//	Line1	Validate the TaskList sequence
	Line2	
	Line3	
	Line4	
//	Line1	Make sure that the sequence of the
	Line2	new TaskList is greater than that of the
	Line3	last TaskList (unless this is the first
	Line4	Task).
if	Condition	CLF::NewTaskList.Sequence < CLF::LastTaskList.Sequence
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskListOutOfSequence
	ParamNames	"Name", "Name2"
	ParamValues	ProcessList.Name, LastCompletedTask.Parent.Name
return	Value	False
//	Line1	Validate that all TaskLists between the
	Line2	Last TaskList and the new TaskList
	Line3	have no required Task.
	Line4	
foreach	Var	CLF::EProcDetail
	List	ProductionProcess.EProcedureDetails
//	Line1	Once we find the TaskList we are now
	Line2	working on, there is no need to
	Line3	continue
	Line4	
if	Condition	CLF::EProcDetail.Sequence == CLF::NewTaskList.Sequence
if	Condition	(LastCompletedTask) and (CLF::EProcDetail.Sequence <= CLF::LastTaskList.Sequence)
if	Condition	(CLF::InitialTaskList) and (CLF::EProcDetail.Sequence < CLF::InitialTaskList.Sequence)
//	Line1	Validate if there are any Batch tasks
	Line2	that haven't been completed
	Line3	successfully
	Line4	
call	CDO	
	Event	AreBatchTasksPending
	Name1	TaskList
	Value1	CLF::EProcDetail.TaskList
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::__Return == False
if	Condition	GetListCount(CLF::EProcDetail.TaskList.RequiredTasks) != 0
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskListSkipped
	ParamNames	"Name"
	ParamValues	CLF::EProcDetail.TaskList.Name
return	Value	False
return	Value	True

New Functions

Function	Parameter	Value
call	CDO	
	Event	PopulateEProcedureVariantDetails

	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
//	Line1	No need to validate if the TaskList
	Line2	hasn't changed.
	Line3	
	Line4	
findlistitem	List	ProductionProcess.EProcedureVariantDetails
	ListKey	TaskList
	KeyValue	ProcessList
	Result	CLF::NewTaskList
	ListIndex	CLF::NewTaskListIndex
if	Condition	LastCompletedTask
findlistitem	List	ProductionProcess.EProcedureDetails
	ListKey	TaskList
	KeyValue	LastCompletedTask.Parent
	Result	CLF::LastTaskList
	ListIndex	CLF::LastTaskListIndex
if	Condition	not (CLF::LastTaskList) or not (CLF::NewTaskList) or (CLF::LastTaskList.Sequence = CLF::NewTaskList.Sequence)
return	Value	True
if	Condition	ProductionProcess.ExecutionMode == Transaction::_Const.ExecutionMode.Sequential
call	CDO	
	Event	GetInitialTaskList
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
setfieldex	From	Transaction::_Return
	To	CLF::InitialTaskList
if	Condition	LastCompletedTask
//	Line1	If this is a new TaskList, verify that all
	Line2	Tasks in the previous TaskList were
	Line3	processed successfully.
	Line4	
call	CDO	
	Event	GetTasksInConcurrentTaskLists
	Name1	Sequence
	Value1	CLF::LastTaskList.Sequence
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
foreach	Var	CLF::Task
	List	Transaction::_Return
//	Line1	Where the Container can start midway
	Line2	through a Procedure, Container level
	Line3	Tasks are not required in the earlier
	Line4	TaskLists
if	Condition	((CLF::InitialTaskList) and (CLF::LastTaskList.Sequence < CLF::InitialTaskList.Sequence) and (CLF::Task.TaskUsage == Transaction::_Const.TaskUsage.Container))
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::Task
	Result	CLF::CompletedTask
	ListIndex	CLF::TaskIndex
if	Condition	CLF::Task.MinIterations > 0
if	Condition	CLF::CompletedTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskIncompleteTaskList
	ParamNames	"Name"

	ParamValues	CLF::Task.Parent.Name
return	Value	False
if	Condition	CLF::CompletedTask
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::CompletedTask
	Name2	TaskIsRequired
	Value2	(CLF::Task.MinIterations > 0)
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::_Return == False
return	Value	False
//	Line1	Validate the TaskList sequence
	Line2	
	Line3	
	Line4	
//	Line1	Make sure that the sequence of the
	Line2	new TaskList is greater than that of the
	Line3	last TaskList (unless this is the first
	Line4	Task).
if	Condition	CLF::NewTaskList.Sequence < CLF::LastTaskList.Sequence
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskListOutOfSequence
	ParamNames	"Name", "Name2"
	ParamValues	ProcessList.Name, LastCompletedTask.Parent.Name
return	Value	False
//	Line1	Validate that all TaskLists between the
	Line2	Last TaskList and the new TaskList
	Line3	have no required Task.
	Line4	
foreach	Var	CLF::EProcDetail
	List	ProductionProcess.EProcedureDetails
//	Line1	Once we find the TaskList we are now
	Line2	working on, there is no need to
	Line3	continue
	Line4	
if	Condition	CLF::EProcDetail.Sequence == CLF::NewTaskList.Sequence
if	Condition	(LastCompletedTask) and (CLF::EProcDetail.Sequence <= CLF::LastTaskList.Sequence)
if	Condition	(CLF::InitialTaskList) and (CLF::EProcDetail.Sequence < CLF::InitialTaskList.Sequence)
//	Line1	Validate if there are any Batch tasks
	Line2	that haven't been completed
	Line3	successfully
	Line4	
call	CDO	
	Event	AreBatchTasksPending
	Name1	TaskList
	Value1	CLF::EProcDetail.TaskList
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	Transaction::_Return == False
if	Condition	GetListCount(CLF::EProcDetail.TaskList.RequiredTasks) != 0
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskListSkipped
	ParamNames	"Name"
	ParamValues	CLF::EProcDetail.TaskList.Name
return	Value	False
return	Value	True

ProcessSeqValidation_ValidateTaskSequence

Base Functions		
Function	Parameter	Value
if	Condition	GetListCount(CompletedTaskDetails) > 0
//	Line1	Verify that the Task has not run more

	Line2	than the max iterations (if defined)
	Line3	
	Line4	
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	ProcessItem
	Result	CLF::FoundTask
	ListIndex	
if	Condition	CLF::FoundTask
if	Condition	ProcessItem.MaxIterations
if	Condition	((CLF::FoundTask.NbrTimesProcessed + 1) > ProcessItem.MaxIterations)
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskMaxRepetitionExceeded
	ParamNames	"Name"
	ParamValues	ProcessItem.MaxIterations
return	Value	False
//	Line1	If the TaskList has a StartProcessTask,
	Line2	verify that it is executed before any of
	Line3	the other Tasks.
	Line4	
findlistitem	List	ProcessList.Tasks
	ListKey	TaskType
	KeyValue	Transaction:: __Const.TaskType.StartProcessTask
	Result	CLF::StartProcessTask
	ListIndex	
if	Condition	(CLF::StartProcessTask) and (ProcessItem <> CLF::StartProcessTask)
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::StartProcessTask
	Result	CLF::CompletedStartProcessTask
	ListIndex	
if	Condition	CLF::CompletedStartProcessTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskStartTaskMustBeFirst
	ParamNames	"Name"
	ParamValues	CLF::StartProcessTask.DisplayName
return	Value	False
//	Line1	For Sequential TaskLists, verify that
	Line2	the Tasks are running in sequence
	Line3	
	Line4	
if	Condition	ProcessList.ExecutionMode == Transaction:: __Const.ExecutionMode.Sequential
setfieldex	From	LastCompletedTaskInTaskList
	To	CLF::LastExecutedTaskInListID
//	Line1	If the Task is the same as the last
	Line2	one processed, then no validations
	Line3	are necessary.
	Line4	
if	Condition	not (CLF::LastExecutedTaskInListID) or (ProcessItem <> CLF::LastExecutedTaskInListID)
if	Condition	CLF::LastExecutedTaskInListID
//	Line1	Optional sequential tasks become required task
	Line2	if executed. Validate that they have passed before
	Line3	starting on the next task.
	Line4	
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::LastExecutedTaskInListID
	Result	CLF::CompletedTask
	ListIndex	
if	Condition	CLF::CompletedTask
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::CompletedTask
	Name2	TaskIsRequired
	Value2	True
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::__Return == False
return	Value	False
//	Line1	The Task must have a higher
	Line2	sequence than the last (unless this
	Line3	is a new TaskList).

	Line4	
if	Condition	CLF::LastExecutedTaskInListID.Parent == ProcessList
if	Condition	ProcessItem.Sequence < CLF::LastExecutedTaskInListID.Sequence
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskOutOfSequence
	ParamNames	"Name"
	ParamValues	CLF::LastExecutedTaskInListID.Name
return	Value	False
foreach	Var	CLF::TaskListDetail
	List	ProcessList.Tasks
//	Line1	Once we have found the Task we
	Line2	are working on, there is no need to
	Line3	continue
	Line4	
if	Condition	CLF::TaskListDetail == ProcessItem
if	Condition	CLF::LastExecutedTaskInListID and (CLF::LastExecutedTaskInListID.Parent == ProcessList) and (CLF::TaskListDetail.Sequence <= CLF::LastExecutedTaskInListID.Sequence)
if	Condition	CLF::TaskListDetail.MinIterations > 0
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskSkipped
	ParamNames	"Name"
	ParamValues	CLF::TaskListDetail.Name
return	Value	False
return	Value	True

New Functions		
Function	Parameter	Value
call	CDO	
	Event	PopulateVariantTasks
	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	Name4	
	Value4	
	Name5	
	Value5	
if	Condition	GetListCount(CompletedTaskDetails) > 0
//	Line1	Verify that the Task has not run more
	Line2	than the max iterations (if defined)
	Line3	
	Line4	
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	ProcessItem
	Result	CLF::FoundTask
	ListIndex	
if	Condition	CLF::FoundTask
if	Condition	ProcessItem.MaxIterations
if	Condition	((CLF::FoundTask.NbrTimesProcessed + 1) > ProcessItem.MaxIterations)
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskMaxRepititionExceeded
	ParamNames	"Name"
	ParamValues	ProcessItem.MaxIterations
return	Value	False
//	Line1	If the TaskList has a StartProcessTask,
	Line2	verify that it is executed before any of
	Line3	the other Tasks.
	Line4	
findlistitem	List	ProcessList.VariantTasks
	ListKey	TaskType
	KeyValue	Transaction::__Const.TaskType.StartProcessTask
	Result	CLF::StartProcessTask
	ListIndex	
if	Condition	(CLF::StartProcessTask) and (ProcessItem <> CLF::StartProcessTask)
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::StartProcessTask
	Result	CLF::CompletedStartProcessTask
	ListIndex	
if	Condition	CLF::CompletedStartProcessTask == Null
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskStartTaskMustBeFirst
	ParamNames	"Name"
	ParamValues	CLF::StartProcessTask.DisplayName

return	Value	False
//	Line1	For Sequential TaskLists, verify that
	Line2	the Tasks are running in sequence
	Line3	
	Line4	
if	Condition	ProcessList.ExecutionMode == Transaction::__Const.ExecutionMode.Sequential
setfieldex	From	LastCompletedTaskInTaskList
	To	CLF::LastExecutedTaskInListID
//	Line1	If the Task is the same as the last
	Line2	one processed, then no validations
	Line3	are necessary.
	Line4	
if	Condition	not (CLF::LastExecutedTaskInListID) or (ProcessItem <> CLF::LastExecutedTaskInListID)
if	Condition	CLF::LastExecutedTaskInListID
setfieldex	From	CLF::LastExecutedTaskInListID.Name
	To	CLF::LastTaskName
//	Line1	Optional sequential tasks become required task
	Line2	if executed. Validate that they have passed before
	Line3	starting on the next task.
	Line4	
findlistitem	List	CompletedTaskDetails
	ListKey	Task
	KeyValue	CLF::LastExecutedTaskInListID
	Result	CLF::CompletedTask
	ListIndex	
if	Condition	CLF::CompletedTask
call	CDO	
	Event	ValidateTaskCompleted
	Name1	ContainerCompletedTask
	Value1	CLF::CompletedTask
	Name2	TaskIsRequired
	Value2	True
	Name3	IsPrerequisiteTask
	Value3	False
	Name4	IsPrerequisiteTaskList
	Value4	False
	Name5	bThrowError
	Value5	CLF::bThrowError
if	Condition	Transaction::__Return == False
return	Value	False
//	Line1	The Task must have a higher
	Line2	sequence than the last (unless this
	Line3	is a new TaskList).
	Line4	
if	Condition	CLF::LastExecutedTaskInListID.Parent == ProcessList
setfieldex	From	ProcessItem.Sequence
	To	CLF::ThisItemSequence
setfieldex	From	CLF::LastExecutedTaskInListID.Sequence
	To	CLF::LastExecutedSequence
if	Condition	ProcessItem.Sequence < CLF::LastExecutedTaskInListID.Sequence
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskOutOfSequence
	ParamNames	"Name"
	ParamValues	CLF::LastExecutedTaskInListID.Name
return	Value	False
foreach	Var	CLF::TaskListDetail
	List	ProcessList.VariantTasks
//	Line1	Once we have found the Task we
	Line2	are working on, there is no need to
	Line3	continue
	Line4	
if	Condition	CLF::TaskListDetail == ProcessItem
if	Condition	CLF::LastExecutedTaskInListID and (CLF::LastExecutedTaskInListID.Parent == ProcessList) and (CLF::TaskListDetail.Sequence <= CLF::LastExecutedTaskInListID.Sequence)
if	Condition	CLF::TaskListDetail.MinIterations > 0
if	Condition	CLF::bThrowError
throw	ErrorId	LabelId::TaskSkipped
	ParamNames	"Name"
	ParamValues	CLF::TaskListDetail.Name
return	Value	False
return	Value	True

RecordLogEntry_ObjectTypeName_OGV

RecordLogEntry_ObjectTypeName_OGV

New Functions

Function	Parameter	Value
setfieldex	From	null
	To	CLF::CDOTypeName
if	Condition	IsNDO
GetCDOType	CDO	NamedDataObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
elseif	Condition	IsContainer
GetCDOType	CDO	Container
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
elseif	Condition	IsQualityObject
GetCDOType	CDO	QualityObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
elseif	Condition	IsRDO
GetCDOType	CDO	RevisionedObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
setfieldex	From	CLF::CDOTypeName
	To	Transaction::__Return

RecordLogEntry_PreValidate_User

ValidateRecordLogEntry

New Functions		
Function	Parameter	Value
callsuper	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	IncludeLogicFromThisCDO	
if	Condition	OperatorLog = null LogEntryType = null LogEntry = null
throw	ErrorId	LabelId::OperatorLogEmptyError
	ParamNames	
	ParamValues	

RecordLogEntry_StartModifyingEntity

RecordLogEntry_StartModifyingEntity

New Functions		
Function	Parameter	Value
callsuper	Name1	
	Value1	
	Name2	
	Value2	
	Name3	
	Value3	
	IncludeLogicFromThisCDO	
if	Condition	IsQualityObject
setfieldex	From	QualityObject
	To	HistoryMainline.HistoryId
if	Condition	IsContainer
setfieldex	From	Container
	To	HistoryMainline.HistoryId
setfieldex	From	Container
	To	HistoryMainline.Container
if	Condition	IsNDO
GetCDOSubTypes	CDOTypeName	"Resource"
	Recursive	true
	IncludeCurrentCDO	false
	SubtypeDisplayNames	
	SubtypeNameNames	CLF::SubTypes
	SubtypeIds	
GetCDOType	CDO	NamedDataObject
	Type	CLF::CDOTypeID
	TypeName	CLF::CDOTypeName
findlistitem	List	CLF::SubTypes
	ListKey	
	KeyValue	CLF::CDOTypeName
	Result	CLF::Found
	ListIndex	

if	Condition	CLF::Found == True CLF::CDOTypeName == "Resource"
setfieldex	From	NamedDataObject
	To	HistoryMainline.Resource

SelectionValuesEx_AllowedLogEntryTypes

Populates SelectionValuesEx using values in specified list fields. The list field items should be of Object Reference type.

New Functions		
Function	Parameter	Value
ConvertCDOListToResultset	List	OperatorLog.LogEntryTypeGroup.ResolvedEntries
	Fields	Name,InstanceID,IsFrozen,Description
	ColNames	
	ColLabels	
	ResultsetStartRow	CLF::__ResultsetStartRow
	ResultsetSizeLimit	CLF::__ResultsetSizeLimit
	FilterFieldName	
	FilterParams	
	Result	Transaction::__Return

SelectionValuesEx_ExecuteTask_Task

Base Functions		
Function	Parameter	Value
setfieldex	From	null
	To	CLF::QueryName
if	Condition	TaskList
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"TaskList"
	ParamValue2	TaskList
	ParamName3	"ElectronicProcedure"
	ParamValue3	ElectronicProcedure
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForTaskList"
	To	CLF::QueryName
elseif	Condition	Workstation
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"Workstation"
	ParamValue2	Workstation
	ParamName3	"ElectronicProcedure"
	ParamValue3	ElectronicProcedure
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForWorkstation"
	To	CLF::QueryName
elseif	Condition	ElectronicProcedure
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"ElectronicProcedure"
	ParamValue2	ElectronicProcedure
	ParamName3	
	ParamValue3	
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForEProc"
	To	CLF::QueryName
if	Condition	CLF::QueryName

ExecuteQueryEx	QueryName	CLF::QueryName
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultSetSizeLimit	CLF:: __ResultSetSizeLimit
	ResultSetStartRow	CLF:: __ResultSetStartRow
	ResultSet	Transaction::__Return

New Functions		
Function	Parameter	Value
setfieldex	From	null
	To	CLF::QueryName
if	Condition	TaskList
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"TaskList"
	ParamValue2	TaskList
	ParamName3	"ElectronicProcedure"
	ParamValue3	ElectronicProcedure
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForTaskList"
	To	CLF::QueryName
elseif	Condition	Workstation
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"Workstation"
	ParamValue2	Workstation
	ParamName3	"ElectronicProcedure"
	ParamValue3	ElectronicProcedure
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForWorkstation"
	To	CLF::QueryName
elseif	Condition	ElectronicProcedure
InitQueryParameters	QueryParameters	CLF::QueryParams
	ParamName1	"Container"
	ParamValue1	Container
	ParamName2	"ElectronicProcedure"
	ParamValue2	ElectronicProcedure
	ParamName3	
	ParamValue3	
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
	ParamValue6	
setfieldex	From	"SelectionValuesEx_TasksForEProc"
	To	CLF::QueryName
if	Condition	CLF::QueryName
ExecuteQueryEx	QueryName	CLF::QueryName
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultSetSizeLimit	CLF:: __ResultSetSizeLimit
	ResultSetStartRow	CLF:: __ResultSetStartRow
	ResultSet	CLF::ResultSet
ConvertResultSetToListOrScalar	ResultSet	CLF::ResultSet
	ColName	"TaskId"
	IsList	true
	FieldType	
	FieldName	VariantTasks
ConvertResultSetToListOrScalar	UseFieldDataType	true
	ResultSet	CLF::ResultSet
	ColName	"NbrTimesProcessed"
	IsList	true
	FieldType	
	FieldName	NbrTimesProcessedList

	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"EProcedureExecutionMode"
	IsList	true
	FieldType	
	FieldName	EProcedureExecutionModelList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"UIVirtualPageName"
	IsList	true
	FieldType	
	FieldName	UIVirtualPageNameList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"IsComputation"
	IsList	true
	FieldType	
	FieldName	IsComputationList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"TaskStatus"
	IsList	true
	FieldType	
	FieldName	TaskStatusList
	UseFieldDataType	true
ConvertResultsetToListOrScalar	Resultset	CLF::ResultSet
	ColName	"TaskListSequence"
	IsList	true
	FieldType	
	FieldName	TaskListSequenceList
	UseFieldDataType	true
setfieldex	From	0
	To	CLF::i
foreach	Var	CLF::Task
	List	VariantTasks
setfieldex	From	true
	To	CLF::Result
setfieldex	From	CLF::Task.Parent.VariantExpression
	To	CLF::VarExpression
if	Condition	CLF::VarExpression
EvalParamIndirect	ExprLocation	CLF::VarExpression
	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result
if	Condition	CLF::Task.VariantExpression
EvalParamIndirect	ExprLocation	CLF::Task.VariantExpression
	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result == false
DeleteListItem	ListField	VariantTasks
	ItemValue	CLF::Task
	ItemIndex	
setfieldex	From	NbrTimesProcessedList[CLF::i]
	To	CLF::Task.NbrTimesProcessed
setfieldex	From	EProcedureExecutionModelList[CLF::i]
	To	CLF::Task.EProcedureExecutionMode
setfieldex	From	UIVirtualPageNameList[CLF::i]
	To	CLF::Task.UIVirtualPageName
setfieldex	From	IsComputationList[CLF::i]
	To	CLF::Task.IsComputation
setfieldex	From	TaskStatusList[CLF::i]
	To	CLF::TaskStatus
if	Condition	CLF::TaskStatus == 1
setfieldex	From	true
	To	CLF::Task.TaskStatus
elseif	Condition	CLF::TaskStatus == 0
setfieldex	From	false
	To	CLF::Task.TaskStatus
setfieldex	From	TaskListSequenceList[CLF::i]
	To	CLF::Task.TaskListSequence
setfieldex	From	CLF::i + 1
	To	CLF::i
ConvertCDOListToResultset	List	VariantTasks
	Fields	Parent.InstanceId,Parent.Name,Parent.Revision,Parent.Instruction,Name,InstanceId,Instruction,TaskListSequence,MinIterations,MaxIterations,TaskStatus,IsComputation,NbrTimesProcessed,DocumentSetId,Ta

		TransactionPageId,WorkstationRequired,EProcedureExecutionMode,Parent.ExecutionMode,UIVirtualPageId VariantExpression, MinIterationsExpression,MaxIterationsExpression,RequirePassFail
	ColNames	"TaskListId","TaskListName","TaskListRevision","TaskListInstruction","TaskName","TaskId","TaskInstructi "TaskSequence","MinIterations","MaxIterations","TaskStatus","IsComputation","NbrTimesProcessed","Dc "TransactionPageId","WorkstationRequired","EProcedureExecutionMode","TaskListExecutionMode","UIVir "VariantExpression","MinIterationsExpression","MaxIterationsExpression","RequirePassFail"
	ColLabels	
	ResultsetStartRow	CLF:: __ResultsetStartRow
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	FilterFieldName	
	FilterParams	
	Result	Transaction:: __Return

SelectionValuesEx_ExecuteTask_TaskList

Base Functions		
Function	Parameter	Value
exitif	LeftValue	ElectronicProcedure
	Comparison	Transaction:: __Const.CompareEqual
	RightValue	null
InitQueryParameters	QueryParameters	CLF::QueryParameters
	ParamName1	"EProcedure"
	ParamValue1	ElectronicProcedure
	ParamName2	
	ParamValue2	
	ParamName3	
	ParamValue3	
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
ExecuteQueryEx	QueryName	"SelectionValuesEx_TaskList"
	Parameters	CLF::QueryParameters
	CDODefId	0
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	ResultsetStartRow	CLF:: __ResultsetStartRow
	Resultset	Transaction:: __Return

New Functions		
Function	Parameter	Value
exitif	LeftValue	ElectronicProcedure
	Comparison	Transaction:: __Const.CompareEqual
	RightValue	null
InitQueryParameters	QueryParameters	CLF::QueryParameters
	ParamName1	"EProcedure"
	ParamValue1	ElectronicProcedure
	ParamName2	
	ParamValue2	
	ParamName3	
	ParamValue3	
	ParamName4	
	ParamValue4	
	ParamName5	
	ParamValue5	
	ParamName6	
ExecuteQueryEx	QueryName	"SelectionValuesEx_TaskList"
	Parameters	CLF::QueryParameters
	CDODefId	0
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	ResultsetStartRow	CLF:: __ResultsetStartRow
	Resultset	Transaction:: __Return
ConvertResultsetToObjects	Resultset	Transaction:: __Return
	ObjectType	
	UseFieldType	true
	Result	VariantTaskLists
	IsList	true
foreach	Method	
	Var	CLF::Item
	List	VariantTaskLists
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::Item.VariantExpression
EvalParamIndirect	ExprLocation	CLF::Item.VariantExpression

	Context	CLF:: __CDOID
	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result == false
DeleteListItem	ListField	VariantTaskLists
	ItemValue	CLF::Item
	ItemIndex	
ConvertCDOListToResultset	List	VariantTaskLists
	Fields	Base.Name, Revision, Base.RevOfRcd, IsFrozen, InstanceID, Description, Status
	ColNames	
	ColLabels	
	ResultsetStartRow	CLF:: __ResultsetStartRow
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	FilterFieldName	
	FilterParams	
	Result	Transaction:: __Return

SelVal_AddSPCAnnotation_Annotations

Base Functions		
Function	Parameter	Value
if	Condition	DataPoint
InitQueryParametersEx	ParamNames	"DataPoint"
	ParamValues	DataPoint
	QueryParameters	CLF::QueryParms
ExecuteQueryEx	QueryName	"GetSPCAnnotations"
	Parameters	CLF::QueryParms
	CDODefId	0
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	ResultsetStartRow	CLF:: __ResultsetStartRow
	Resultset	CLF::QueryResults
elseif	Condition	CustomSPCDataId
InitQueryParametersEx	ParamNames	"CustomSPCDataId"
	ParamValues	CustomSPCDataId
	QueryParameters	CLF::QueryParameters
ExecuteQueryEx	QueryName	"GetCustomSPCAnnotations"
	Parameters	CLF::QueryParameters
	CDODefId	0
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	ResultsetStartRow	CLF:: __ResultsetStartRow
	Resultset	Transaction:: __Return

New Functions		
Function	Parameter	Value
if	Condition	DataPoint
InitQueryParametersEx	ParamNames	"DataPoint", "SPCChartDefId"
	ParamValues	DataPoint, SPCChartDef
	QueryParameters	CLF::QueryParms
ExecuteQueryEx	QueryName	"GetSPCAnnotations"
	Parameters	CLF::QueryParms
	CDODefId	0
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	ResultsetStartRow	CLF:: __ResultsetStartRow
	Resultset	Transaction:: __Return
elseif	Condition	CustomSPCDataId
InitQueryParametersEx	ParamNames	"CustomSPCDataId", "SPCChartDefId"
	ParamValues	CustomSPCDataId, SPCChartDef
	QueryParameters	CLF::QueryParameters
ExecuteQueryEx	QueryName	"GetCustomSPCAnnotations"
	Parameters	CLF::QueryParameters
	CDODefId	0
	ResultsetSizeLimit	CLF:: __ResultsetSizeLimit
	ResultsetStartRow	CLF:: __ResultsetStartRow
	Resultset	Transaction:: __Return

SelVal_GetOperatorLogEntries

OpLog_GetAttachedDocuments_OGV

New Functions		
Function	Parameter	Value
if	Condition	OperatorLog = null
return	Value	
InitQueryParameters	QueryParameters	CLF::QueryParms
	ParamName1	"OperatorLog"

	ParamValue1	OperatorLog
	ParamName2	"StartDateTime"
	ParamValue2	StartDateTime
	ParamName3	"EndDateTime"
	ParamValue3	EndDateTime
	ParamName4	"LogEntryType"
	ParamValue4	LogEntryTypeFilter
	ParamName5	"LogEntry"
	ParamValue5	LogEntryFilter
	ParamName6	
	ParamValue6	
ExecuteQueryEx	QueryName	" SelectionValuesEx_OpLog_EntryHistory "
	Parameters	CLF::QueryParams
	CDODefId	0
	ResultsetSizeLimit	
	ResultsetStartRow	
	Resultset	Transaction::__Return

SelVal_RDOAndNDOObjects

SelVal_RDOAndNDOObjects

New Functions		
Function	Parameter	Value
ExecuteQueryEx	QueryName	" SelectionValuesEx_OpLog_RDOAndNDOObjects "
	Parameters	CLF::__Parameters
	CDODefId	0
	ResultsetSizeLimit	-1
	ResultsetStartRow	1
	Resultset	Transaction::__Return

Task_ProcessUserQueryDynamicValues

Task_ProcessUserQueryDynamicValues

New Functions		
Function	Parameter	Value
foreach	Var	CLF::Item
	List	CLF::Task.TaskQueries
foreach	Var	CLF::Param
	List	CLF::Item.OnlineQuerySetup.UserQueryParameters
CreateListItem	ListCDO	CLF::__CDOID
	ListField	OnlineQuerySetupParamsTmp
	ItemType	
	ItemValue	CLF::NewParam
	ItemIndex	
setfieldex	From	CLF::Task.Name
	To	CLF::NewParam.TaskName
setfieldex	From	CLF::Item.OnlineQuerySetup.Name
	To	CLF::NewParam.QueryName
setfieldex	From	CLF::Param.Name
	To	CLF::NewParam.Name
setfieldex	From	CLF::Param.DynamicValue
	To	CLF::NewParam.DynamicValue
EvalParamIndirect	ExprLocation	CLF::Param.DynamicValue
	Context	GetCurrentService()
	IgnoreNullObjectReferences	true
	Result	CLF::Value
if	Condition	NOT(CLF::Value)
setfieldex	From	CLF::Param.DefaultValue
	To	CLF::Value
setfieldex	From	CLF::Value
	To	CLF::NewParam.ResolvedValue

ValidateVariantExpression

ValidateVariantExpression

New Functions		
Function	Parameter	Value
setfieldex	From	true
	To	CLF::Result
if	Condition	CLF::VariantExpression
EvalParamIndirect	ExprLocation	CLF::VariantExpression
	Context	CLF::__CDOID

	IgnoreNullObjectReferences	true
	Result	CLF::Result
if	Condition	CLF::Result
return	Value	true
return	Value	false

DB Table Definitions

A_SPCChartVisualParams

SPC Chart Visual Parameters

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
SPCChartId	InstanceId	Fixed Length String	1	True	True	0	0	False	False	0
FieldId	FieldId	Integer	2	True	True	0	0	False	False	0
Sequence	Sequence	Integer	3	True	True	0	0	False	False	0
VisualParamsId	SPC Chart Visual Parameters	Fixed Length String	0	True	True	0	0	False	False	0

AddSPCAnnotationHistory

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
ExternalDataId	Data ID from external data source	Variable Length String	0	True	True	255	0	False	False	0
SPCChartDefId	Defines SPC Charting information for a variable/chart type combination. Includes designation for rules file, macro, display options, OCAPs and 10 user params to use during chart generation, display and OCAP processing	Fixed Length String	0	True	True	0	0	False	False	0
SecondaryDataId	Table name from external data source	Variable Length String	0	True	True	255	0	False	False	0

ContainerCompletedTask

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
WorkflowStepId	A step is an individual tracking point within a workflow. The list of steps is the primary data structure within a workflow. Paths control the allowable movements between steps. Each Step contains zero or more Paths, which link to another Step. One Path at each Step is identified as the default Path. A Step normally represents an individual processing point in a workflow, where it references a Spec which in turn describes the work that is to be performed. A Step can reference another workflow, in which case the step represents a series of processing points.	Fixed Length String	0	True	True	0	0	False	False	0
ElectronicProcedureId	Provides the framework for executing a series of Tasks at the Spec. An Electronic Procedure defines a set of instructions (a checklist for performing a set of tasks), and is compose of one ore more TaskLists.	Fixed Length String	0	True	True	0	0	False	False	0

DataPoint

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
ValidateResource	When enabled will validate the selected resource to make sure it can be used	Boolean	0	True	True	0	0	False	False	0

ElectronicProcedure

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
-------------	--------------------	----------	----------------------	------------------------	------------------------	-----------	-------	----------------	----------------	-------------------

KeepCompletedTasks	When true will prevent deleting ContainerCompletedTasks data for the container at the current step	Boolean	0	True	True	0	0	False	False	0
KeepCompletedTasksOnOpen	When true will not delete any completed task history	Boolean	0	True	True	0	0	False	False	0

EProcedureDetail

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.Hexadecimal representation of GUID (e.g. BDE4B634-153E-4A31-A453-EEA85FE80532)									

LogEntryType

A user defined type to associate with an operator log entry

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
CDOTypeId	CDO Definition Id	Integer	0	True	True	10	0	False	False	0
LogEntryTypeId	Unique ID for an instance of this object, generated by the system.	Fixed Length String	1	True	True	0	0	False	False	0
ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer	0	True	True	10	0	False	False	0
FilterTags	FilterTags	Variable Length String	0	True	True	0	0	False	False	0
Notes	Notations and comments associated with this object.	Variable Length String	0	True	True	2000	0	False	False	0
ChangeHistoryId	Details of the last change.	Fixed Length String	0	True	True	0	0	False	False	0
Description	Description of this entity. Defaults to the name of this entity if not specified	Variable Length String	0	True	True	255	0	False	False	0
IconId	Identifier for the icon associated with this Base Entity	Integer	0	True	True	10	0	False	False	0
IsFrozen	Determines if changes are allowed to this instance (or any of its revisions).	Boolean	0	True	True	0	0	False	False	0
LogEntryTypeName	Unique name for this instance	Variable Length String	0	True	True	30	0	False	False	0

LogEntryTypeGroup

A group of LogEntryType objects

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
CDOTypeId	CDO Definition Id	Integer	0	True	True	10	0	False	False	0
LogEntryTypeGroupId	Unique ID for an instance of this object, generated by the system.	Fixed Length String	1	True	True	0	0	False	False	0
ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer	0	True	True	10	0	False	False	0
FilterTags	FilterTags	Variable Length String	0	True	True	0	0	False	False	0
Notes	Notations and comments associated with this object.	Variable Length String	0	True	True	2000	0	False	False	0
ChangeHistoryId	Details of the last change.	Fixed Length String	0	True	True	0	0	False	False	0
Description	Description of this entity. Defaults to the name of this entity if not specified	Variable Length String	0	True	True	255	0	False	False	0
IconId	Identifier for the icon associated with this Base Entity	Integer	0	True	True	10	0	False	False	0
IsFrozen	Determines if changes are allowed to this instance (or any of its revisions).	Boolean	0	True	True	0	0	False	False	0

LogEntryTypeGroupName	Unique name for this instance	Variable Length String	0	True	True	30	0	False	False	0
EntryType	Name of the type of objects (CDO Definition Name) contained in this ObjectGroup. There is no business logic support for this field and it will be eventually removed.	Variable Length String	0	True	True	32	0	False	False	0

LogEntryTypeGroupDefaultForObj

CDO Definition Id

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
LogEntryTypeGroupId	InstanceId	Fixed Length String	1	True	True	0	0	False	False	0
FieldId	FieldId	Integer	2	True	True	0	0	False	False	0
Sequence	Sequence	Integer	3	True	True	0	0	False	False	0
DefaultForObjectTypes	CDO Definition Id	Integer	0	True	True	10	0	False	False	0

LogEntryTypeGroupEntries

User codes are most often used to define loss reasons through selection lists for data entry fields. InSite provides several user codes such as Bonus Reason and Loss Reason. User Defined Codes are used to allow each customer to specify a list of allowable values for a specific field. In many cases a User Defined Code will merely consist of a Name and Description with no additional attributes. They may also include additional attributes, typically used for additional validations. The name for each Code must be unique within its type. This value is used as an alternate key for lookup-up, data entry and validation.

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
LogEntryTypeGroupId	InstanceId	Fixed Length String	1	True	True	0	0	False	False	0
FieldId	FieldId	Integer	2	True	True	0	0	False	False	0
Sequence	Sequence	Integer	3	True	True	0	0	False	False	0
EntriesId	User codes are most often used to define loss reasons through selection lists for data entry fields. InSite provides several user codes such as Bonus Reason and Loss Reason. User Defined Codes are used to allow each customer to specify a list of allowable values for a specific field. In many cases a User Defined Code will merely consist of a Name and Description with no additional attributes. They may also include additional attributes, typically used for additional validations. The name for each Code must be unique within its type. This value is used as an alternate key for lookup-up, data entry and validation.	Fixed Length String	0	True	True	0	0	False	False	0

LogEntryTypeGroupGroups

UserCodeGroup represents a group of user code.

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
LogEntryTypeGroupId	InstanceId	Fixed Length String	1	True	True	0	0	False	False	0
FieldId	FieldId	Integer	2	True	True	0	0	False	False	0
Sequence	Sequence	Integer	3	True	True	0	0	False	False	0
GroupsId	UserCodeGroup represents a group of user code.	Fixed Length String	0	True	True	0	0	False	False	0

MasterRecipe

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
KeepCompletedTasks	When true will prevent deleting ContainerCompletedTasks data for the container at the current step	Boolean	0	True	True	0	0	False	False	0
KeepCompletedTasksOnOpen	When true will not delete any completed task history	Boolean	0	True	True	0	0	False	False	0

MasterRecipeDetail

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.Hexadecimal representation of GUID (e.g. BDE4B634-153E-4A31-A453-EEA85FE80532)									

MfgOrderProcedure

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
KeepCompletedTasks	When true will prevent deleting ContainerCompletedTasks data for the container at the current step	Boolean	0	True	True	0	0	False	False	0
KeepCompletedTasksOnOpen	When true will not delete any completed task history	Boolean	0	True	True	0	0	False	False	0

MfgOrderTaskList

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	Variable Length String	0	True	True	1000	0	False	False	0

OperatorLog

Log for shop floor operators to record entries of various types.

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
CDOTypeId	CDO Definition Id	Integer	0	True	True	10	0	False	False	0
OperatorLogId	Unique ID for an instance of this object, generated by the system.	Fixed Length String	1	True	True	0	0	False	False	0
ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer	0	True	True	10	0	False	False	0
FilterTags	FilterTags comma seperated list	Variable Length String	0	True	True	0	0	False	False	0
Notes	Notations and comments associated with this object.	Variable Length String	0	True	True	2000	0	False	False	0
ChangeHistoryId	Details of the last change.	Fixed Length String	0	True	True	0	0	False	False	0
Description	Description of this entity. Defaults to the name of this entity if not specified	Variable Length String	0	True	True	255	0	False	False	0
IconId	Identifier for the icon associated with this Base Entity	Integer	0	True	True	10	0	False	False	0
IsFrozen	Determines if changes are allowed to this instance (or any of its revisions).	Boolean	0	True	True	0	0	False	False	0
OperatorLogName	Unique name for this instance	Variable Length String	0	True	True	30	0	False	False	0
ContainerId	Holds the object assocted to the log if object typs is a Container	Fixed Length String	0	True	True	0	0	False	False	0
IsContainer	True when object for log is a Container	Boolean	0	True	True	0	0	False	False	0
IsNDO	True when object for log is an NDO	Boolean	0	True	True	0	0	False	False	0
IsQualityObject	True when object for log is a Quality Object	Boolean	0	True	True	0	0	False	False	0
IsRDO	True when object for log is an RDO	Boolean	0	True	True	0	0	False	False	0
LogEntryTypeGroupId	Limits the log entry types that may be selected for this log when recording a log entry.	Fixed Length String	0	True	True	0	0	False	False	0
NamedDataObjectId	Holds the object for the log if object is an NDO	Fixed Length String	0	True	True	0	0	False	False	0

ObjectInstanceId	Instance ID of the object to which the log is associated.	Variable Length String	0	True	True	30	0	False	False	0
LogObjectType	LogObjectType 0 - NDO 1 - RDO 2 - Container 3 - QualityObject	Integer	0	True	True	0	0	False	False	0
QualityObjectId	Holds the object for the log if object is a QualityObject	Fixed Length String	0	True	True	0	0	False	False	0
RevisionedObjectId	Holds the object for the log if object is a RevisionedObject	Fixed Length String	0	True	True	0	0	False	False	0

RecipeItem

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
MaxIterationsExpression	InSite Unified Expression	Variable Length String	0	True	True	1000	0	False	False	0
MinIterationsExpression	InSite Unified Expression	Variable Length String	0	True	True	1000	0	False	False	0
VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	Variable Length String	0	True	True	1000	0	False	False	0

RecipeList

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	Variable Length String	0	True	True	1000	0	False	False	0

RecordLogEntryHistory

History record that acts as an Operator Log entry

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
CDOTypeId	CDO Definition Id	Integer	0	True	True	10	0	False	False	0
RecordLogEntryHistoryId	Unique ID for an instance of this object, generated by the system.	Fixed Length String	1	True	True	0	0	False	False	0
ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer	0	True	True	10	0	False	False	0
RecordLogEntryHistoryName	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.	Variable Length String	0	True	True	36	0	False	False	0
HistoryMainlineId	Id of the HistoryMainline to which this ServiceHistorySummary applies.	Fixed Length String	0	True	True	0	0	False	False	0
IntegrationTxnId	Inserts the TxnId after a compound/container/resource transaction with history mainline.	Fixed Length String	0	True	True	0	0	False	False	0
HistoryId	Unique identifier associated with all Containers participating in this transaction.	Fixed Length String	0	True	True	0	0	False	False	0
TxnId	Unique number used to identify this transaction. This value is used to logically link (and join in SQL) all of the entries associated with a single Service. Note that compound services (availabel in V1.4) can create multiple HistoryMainlines, but the HistoryMainlines and other history records relating to a single transaction will always get assigned the same unique TxnId.	Fixed Length String	0	True	True	0	0	False	False	0
EmployeeId	The operator recording the log entry	Fixed Length String	0	True	True	0	0	False	False	0
EmployeeName	Name of operator	Variable Length String	0	True	True	30	0	False	False	0
LogEntry	Text of the log entry	Variable Length String	0	True	True	4000	0	False	False	0

LogEntryTypeId	A user defined type to associate with an operator log entry	Fixed Length String	0	True	True	0	0	False	False	0
LogEntryTypeName	Name of the log entry type	Variable Length String	0	True	True	30	0	False	False	0
OperatorLogId	Log for shop floor operators to record entries of various types.	Fixed Length String	0	True	True	0	0	False	False	0
OperatorLogName	Name of the log	Variable Length String	0	True	True	30	0	False	False	0
TxnDateTime	Generic TimeStamp	TimeStamp	0	True	True	0	0	False	False	0

RecordLogEntryHistoryHistoryDe

Summary of the changes caused by a single ServiceDetail CDO.

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
RecordLogEntryHistoryId	InstanceId	Fixed Length String	1	True	True	0	0	False	False	0
FieldId	FieldId	Integer	2	True	True	0	0	False	False	0
Sequence	Sequence	Integer	3	True	True	0	0	False	False	0
HistoryDetailsId	Summary of the changes caused by a single ServiceDetail CDO.	Fixed Length String	0	True	True	0	0	False	False	0

SPCChartCustomInfo

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
InAnnotationPopup	In Annotation Popup	Boolean	0	True	True	0	0	False	False	0

SPCChartDef

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
DisplayDescriptiveStats	Display Descriptive Stats	Boolean	0	True	True	0	0	False	False	0
SPCChartVisualParamsId	SPC Chart Visual Parameters	Fixed Length String	0	True	True	0	0	False	False	0

SPCChartQueryParams

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
-------------	--------------------	----------	----------------------	------------------------	------------------------	-----------	-------	----------------	----------------	-------------------

SPCChartVisualParams

SPC Chart Visual Parameters

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
CDOTypeId	CDO Definition Id	Integer	0	True	True	10	0	False	False	0
SPCChartVisualParamsId	Unique ID for an instance of this object, generated by the system.	Fixed Length String	1	True	True	0	0	False	False	0
ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer	0	True	True	10	0	False	False	0
SPCChartVisualParamsName	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.	Variable Length String	0	True	True	36	0	False	False	0
ParentId	Id of the CDO containing this CDO.	Fixed Length String	0	True	True	0	0	False	False	0
ParamName	Parameter Name	Variable Length	0	True	True	255	0	False	False	0

		String								
ParamValue	Parameter Value	Variable Length String	0	True	True	255	0	False	False	0

TaskItem

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
						300				
						300				
MaxIterationsExpression	InSite Unified Expression	Variable Length String	0	True	True	1000	0	False	False	0
MinIterationsExpression	InSite Unified Expression	Variable Length String	0	True	True	1000	0	False	False	0
VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	Variable Length String	0	True	True	1000	0	False	False	0
RequirePassFail	Require Pass/Fail	Boolean	0	True	True	0	0	False	False	0

TaskItemTaskQueries

TaskQueries

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
TaskItemId	InstanceId	Fixed Length String	1	True	True	0	0	False	False	0
FieldId	FieldId	Integer	2	True	True	0	0	False	False	0
Sequence	Sequence	Integer	3	True	True	0	0	False	False	0
TaskQueriesId	TaskQueries	Fixed Length String	0	True	True	0	0	False	False	0

TaskList

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
VariantExpression	Variant expression to be resolved to allow for filtering out configurable activities.	Variable Length String	0	True	True	1000	0	False	False	0

TaskQueries

Review Task Queries

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
CDOTypeId	CDO Definition Id	Integer	0	True	True	10	0	False	False	0
TaskQueriesId	Unique ID for an instance of this object, generated by the system.	Fixed Length String	1	True	True	0	0	False	False	0
ChangeCount	System field used to ensure data integrity when multiple users concurrently modify the same object.	Integer	0	True	True	10	0	False	False	0
TaskQueriesName	A unique key used for processing import/exports so subentity lists are not necessarily rebuild during an import.	Variable Length String	0	True	True	36	0	False	False	0
TaskItemId	Id of the CDO containing this CDO.	Fixed Length String	0	True	True	0	0	False	False	0
IsFrozen	IsFrozen	Boolean	0	True	True	0	0	False	False	0
Sequence	Relative sequence number	Integer	0	True	True	5	0	False	False	0
OnlineQuerySetupId	These queries are used in an ad-hoc manner to view runtime data using the Online Query service. The Resultset Size Limit must be used to ensure that InSite does not reserve too much memory that may impact performance.	Fixed Length String	0	True	True	0	0	False	False	0

UserQuery

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
-------------	--------------------	----------	----------------------	------------------------	------------------------	-----------	-------	----------------	----------------	-------------------

ExternalDataId	Data IDfrom external data source	Variable Length String	0	True	True	255	0	False	False	0
ExternalTable	Table Namefrom external data source	Variable Length String	0	True	True	255	0	False	False	0

UserQueryParameter

Column Name	Column Description	SQL Type	Primary Key Sequence	Allow System To Update	Allow System To Delete	Precision	Scale	Force To Upper	Force To Lower	Associated Column
-------------	--------------------	----------	----------------------	------------------------	------------------------	-----------	-------	----------------	----------------	-------------------

Labels**CSICDName**

Label Name	Value
CSICDName_RecordLogEntryDetail	Record Log Entry Detail
CSICDName_LogTxn	Log Txn
CSICDName_RecordLogEntry	Record Log Entry
CSICDName_RecordLogEntryHistory	Record Log Entry History
CSICDName_OperatorLog	Operator Log
CSICDName_OperatorLogChanges	Operator Log Changes
CSICDName_LogObjectTypeEnum	Log Object Type Enum
CSICDName_OperatorLogMaint	Operator Log Maint
CSICDName_LogEntryTypeMaint	Log Entry Type Maint
CSICDName_LogEntryTypeChanges	Log Entry Type Changes
CSICDName_LogEntryTypeGroup	Log Entry Type Group
CSICDName_LogEntryTypeGroupMaint	Log Entry Type Group Maint
CSICDName_LogEntryTypeGroupChanges	Log Entry Type Group Changes
CSICDName_LogEntryType	Log Entry Type
CSICDName_SPCChartCustomInfo	SPC Chart Custom Info
CSICDName_OnlineQuerySetup	Online QueryOnline Query Setup
CSICDName_SPCChartVisualParams	SPC Chart Visual Params
CSICDName_SPCChartVisualParamsChanges	SPC Chart Visual Params Changes
CSICDName_OperatorLogInquiry	Operator Log Inquiry
CSICDName_TaskQueries	Task Queries
CSICDName_TaskQueriesChanges	Task Queries Changes
CSICDName_LogDetailsForObject	Log Details For Object
CSICDName_OperatorLogInfo	Operator Log Info
CSICDName_SPCChartCustomInfo	SPC Chart Custom Info
CSICDName_SPCChartCustomInfoChanges	SPC Chart Custom Info Changes
CSICDName_SPCChartVisualParamsChanges	SPC Chart Visual Params Changes
CSICDName_SPCChartVisualParams	SPC Chart Visual Params
CSICDName_SPCChartCustomInfoChanges	SPC Chart Custom Info Changes
CSICDName_InlineSPCTxn	Inline SPC Txn
CSICDName_ExecuteSPCTxn	Inline SPC Txn
CSICDName_SPCChartVisualParamsChanges	SPC Chart Visual Params Changes
CSICDName_InlineSPCTxn	Inline SPC Txn
CSICDName_SPCChartVisualParamsChanges	SPC Chart Visual Params Changes

CSIField

Label Name	Value
ProductionProcess_KeepCompletedTasks	Keep Completed Tasks on Move
RecordLogEntryHistory_TxnDateTime	TxnDateTime
RecordLogEntryHistory_ExportImportKey	Record Log Entry History ExportImportKey
TaskItem_UIVirtualPageName	UI Virtual Page Name
ProcessItem_MaxIterationsExpression	Max Iterations Expression
LogTxn_LogEntryFilter	Log Entry Filter
TaskItem_IsComputation	Is Computation
ProductionProcess_KeepCompletedTasksOnOpen	Keep Completed Tasks on Open
ProcessItem_MinIterationsExpression	Min Iterations Expression
ExecuteProductionProcess_VariantTasks	Variant Tasks
ProcessItem_VariantExpression	Variant Expression
ObjectDataPoint_ValidateResource	Validate Resource
OperatorLog_Name	Operator Log Name
OperatorLog_ObjectInstanceId	Object Instance ID
OperatorLog_LogObjectType	Log Object Type
LogEntryTypeGroup_Name	Log Entry Type Group Name
LogEntryType_Name	Log Entry Type Name
RecordLogEntryDetail_TxnDateTime	Txn Date Time
RecordLogEntry_LogEntryTypeFilter	Log Entry Type Filter
LogTxn_LogEntry	Log Entry
TaskItem_TaskListSequence	Task List Sequence
ProcessItem_MaxIterationsExpression	Max Iterations Expression
ProductionProcess_KeepCompletedTasks	Keep Completed Tasks on Move
ProductionProcess_KeepCompletedTasksOnOpen	Keep Completed Tasks on Open

TaskItem_EProcedureExecutionMode	EProcedure Execution Mode
ProcessItem_MinIterationsExpression	Min Iterations Expression
ExecuteProductionProcess_VariantTasks	Variant Tasks
ProcessItem_VariantExpression	Variant Expression
TaskItem_WorkstationRequired	Workstation Required
ObjectDataPoint_ValidateResource	Validate Resource
ExecuteTask_IsComputationList	Is Computation List
TaskItem_TaskListSequence	Task List Sequence
ExecuteTask_IsComputationList	Is Computation List
TaskItem_WorkstationRequired	Workstation Required
TaskItem_DocumentSetId	Document Set Id
TaskItem_TransactionPageId	Transaction Page Id
ProductionProcess_KeepCompletedTasks	Keep Completed Tasks on Move
TaskItem_IsComputation	Is Computation
ObjectDataPoint_MapToUserAttribute	Map To User Attribute
ProcessItem_VariantExpression	Variant Expression
TaskItem_EProcedureExecutionMode	EProcedure Execution Mode
ExecuteTask_IsComputationList	Is Computation List
TaskItem_WorkstationRequired	Workstation Required
TaskItem_DocumentSetId	Document Set Id
TaskItem_TransactionPageId	Transaction Page Id
ObjectDataPoint_MapToUserAttribute	Map To User Attribute
ObjectDataPoint_MapToUserAttribute	Map To User Attribute
ProductionProcess_KeepCompletedTasksOnOpen	Keep Completed Tasks on Open
TaskItem_TransactionPageId	Transaction Page Id
ExecuteProductionProcess_VariantTasks	Variant Tasks
TaskItem_DocumentSetId	Document Set Id
ObjectDataPoint_ValidateResource	Validate Resource
ProcessItem_MaxIterationsExpression	Max Iterations Expression
TaskItem_UIVirtualPageName	UI Virtual Page Name
TaskItem_TaskListSequence	Task List Sequence
TaskItem_EProcedureExecutionMode	EProcedure Execution Mode
ProcessItem_MinIterationsExpression	Min Iterations Expression
OperatorLog_IsQualityObject	Is Quality Object
TaskItem_TaskQueries	Review Task Queries
TaskItem_UIVirtualPageName	UI Virtual Page Name
TaskItem_RequirePassFail	Require Pass/Fail
Product_ResolvedBOM	Resolved BOM
MaterialListItem_ActualQtyRequired	Actual Qty Required
SPCChartCustomInfo_InTooltip	In Tooltip
Service_CurrentDay	Day
Service_CurrentYear	Year
UserQueryParameter_ResolvedValue	Resolved Value
ComponentIssue_ApplyStepFilter	Apply Step Filter
ComponentIssue_AutoLoadPhantomMaterials	Automatically load Phantom Materials
SPCChartDef_DisplayDescriptiveStats	Display Descriptive Stats
OperatorLogInquiry_StartTimeFilter	Start Time Filter
SPCChartCustomInfo_XAxisLabel	X-Axis Label
SPCChartCustomInfo_ExportImportKey	SPC Chart Custom Info Name
SPCAnnotations_ExternalDataGrid	External Data Annotation Tooltip Grid
OnlineQuerySetupParams_DynamicValue	Dynamic Value
SPCChartType_VisualParams	SPC Chart Visual Parameters
SPCChartCustomInfo_InAnnotationPopup	In Annotation Popup
AddSPCAnnotation_ExternalTable	Secondary Data ID
SPCChartDef_DisplayDescriptiveStats	Generic Boolean
OperatorLogInquiry_LogInstanceId	Operator Log ID
OperatorLogInquiry_GetLogEntries	Get Log Entries
OperatorLogInquiry_OperatorLogs	Operator Logs
LogDetailsForObject_HasLogs	Has Logs
OperatorLogInquiry_LogEntryFilter	LogEntryFilter
OperatorLogInquiry_EndTimeFilter	End Time Filter
OperatorLogInquiry_LogName	LogName
OperatorLogInquiry_OperatorLogInstanceId	Operator Log Instance ID
OperatorLogInquiry_OperatorLogName	OperatorLogName
OperatorLogInquiry_OperatorLogId	Operator Log ID
SPCChartCustomInfo_InAnnotationPopup	In Annotation Popup
OperatorLogInquiry_ObjectRevision	ObjectRevision
SPCAnnotations_ExternalDataGrid	Annotation Tooltip Grid
AddSPCAnnotationHistory_ExternalTable	Secondary Data ID
SPCAnnotation_DataId	Data ID
SPCChartCustomInfo_InAnnotationPopup	In Annotation Popup
SPCChartCustomInfo_SubgroupItem	Subgroup Item
SPCAnnotations_ExternalDataGrid	External Data Annotation Tooltip Grid
SPCChartCustomInfo_InAnnotationPopup	In Annotation Popup
SPCQuery_ExternalTable	Secondary Data ID
TaskItem_IsComputation	Is Computation
SPCAnnotations_ExternalDataGrid	Annotation Tooltip Grid

SPCQuery_ExternalDataId	Data ID
ProcessSPCChartDynamicParams_DynamicParamsString	String with params
ProcessSPCChartDynamicParams_SPCChartQueryParamsString	Generic String
SPCAnnotations_ExternalDataGrid	External Data Annotation Tooltip Grid
SPCChartCustomInfo_InAnnotationPopup	In Annotation Popup
SPCChartDef_ShowToleranceLimits	Show Tolerance Limits
ProcessSPCChartDynamicParams_SPCChartQueryParamsString	Generic String
SPCAnnotation_SelectedDataID	Selected DataID
SPCChartCustomInfo_ColumnName	Column Name
SPCAnnotations_ExternalDataGrid	External Data Annotation Tooltip Grid
SPCChartCustomInfo_IsRemark	Is Remark
SPCChartVisualParams_ExportImportKey	SPC Chart Visual Params ExportImportKey
SPCChartDef_DisplayDescriptiveStats	Generic Boolean
SPCChartType_VisualParams	SPC Chart Visual Parameters
SPCAnnotation_DataValue	Data Value
SPCChartVisualParams_ExportImportKey	SPC Chart Visual Params ExportImportKey
ProcessSPCChartDynamicParams_DynamicParamsString	String with params

CSIReject

Label Name	Value
OperatorLogEmptyError	Not all required fields were supplied to record a log entry.
OperatorLogInquiry_LogNoSpecified	Could not resolve an Operator Log from the provided Log Name or Instance ID.
MissingFieldObjectInstanceId	No value specified for field ObjectInstanceId.

CSIWebClient

Label Name	Value
Lbl_SaveSPCLogs	Save SPC logs
Lbl_SPCWarningCustomDSMessage	If a Custom Data Source is configured, the selected SPCQuery requires explicit definitions for both the "Data ID" and "Secondary Data ID" properties.

OtherReject

Label Name	Value
SPCInterfaceError	SPCInterface.dll Error: #ErrorMsg.Name

Query Definitions**ComponentReplaceInquiry_ContainerSelVal****ComponentReplaceInquiry_ContainerSelVal : SqlServer**

```

WITH ActualsHistory AS ( SELECT iah.FromContainerId, iah.ToContainerId, iah.ProductId, iah.IssueActualsHistoryId, iah.FromLocationId,
iah.TxnId, iah.Qty, iah.FromStockPoint, iah.FromLot, iah.ReferenceDesignator, 0 as ContainerLevel FROM IssueActualsHistory iah WHERE
iah.ToContainerId = ?Container and iah.IssueActualsHistoryId not in (select IssueActualsHistoryId from RemoveHistoryDetail) <{?
ShowAsBuilt}>><{ UNION ALL SELECT iah.FromContainerId, iah.ToContainerId, iah.ProductId, iah.IssueActualsHistoryId, iah.FromLocationId,
iah.TxnId, iah.Qty, iah.FromStockPoint, iah.FromLot, iah.ReferenceDesignator, ah.ContainerLevel + 1 FROM IssueActualsHistory iah INNER
JOIN ActualsHistory ah ON ah.FromContainerId = iah.ToContainerId > } SELECT DISTINCT ActualsHistory.FromContainerId as
@header:name="ContainerId", ActualsHistory.IssueActualsHistoryId as @header:name="IssueActualHistoryId", ActualsHistory.ToContainerId as
@header:name="ParentContainerId", COALESCE(chC.ContainerName, ActualsHistory.FromLot, ActualsHistory.FromStockPoint) as
@header:name="IssuedFrom":label=Lbl_IssuedFrom, prnC.ContainerName as @header:name="ParentContainerName", ActualsHistory.FromLot as
@header:name="FromLot", ActualsHistory.FromStockPoint as @header:name="FromStockPoint", ActualsHistory.ContainerLevel as
@header:name="ContainerLevel", (CASE WHEN ProductBase.ProductName IS NULL THEN CONCAT(CONCAT(prB.ProductName, ':'), pr.ProductRevision)
ELSE CONCAT(CONCAT(ProductBase.ProductName, ':'), chP.ProductRevision) END) as @header:name="Product":label=Container_Product,
chP.Description as @header:name="Description":label=SelVal_ProductDescription, ActualsHistory.Qty as
@header:name="QtyIssued":label=Lbl_QtyIssued, ActualsHistory.ReferenceDesignator as
@header:name="ReferenceDesignator":label=Lbl_ReferenceDesignator, null as HVIssueHistoryDetailId, null as HVSetupDetailId, null as
MaterialListItemId, ActualsHistory.IssueActualsHistoryId as IssueKey FROM ActualsHistory LEFT JOIN Container chC on
ActualsHistory.FromContainerId = chC.ContainerId LEFT JOIN Container prnC on ActualsHistory.ToContainerId = prnC.ContainerId LEFT JOIN
Product pr on ActualsHistory.ProductId = pr.ProductId LEFT JOIN ProductBase prB on pr.ProductBaseId = prB.ProductBaseId LEFT JOIN
Product chP on chC.ProductId = chP.ProductId LEFT JOIN Product prnP on prnC.ProductId = prnP.ProductId LEFT JOIN ProductBase on
chP.ProductBaseId = ProductBase.ProductBaseId LEFT JOIN Location on ActualsHistory.FromLocationId = Location.LocationId JOIN
HistoryMainline on HistoryMainline.TxnId = ActualsHistory.TxnId AND HistoryMainline.ReversalStatus = 1 WHERE
ActualsHistory.IssueActualsHistoryId NOT IN ( SELECT RemoveHistoryDetail.IssueActualsHistoryId FROM RemoveHistoryDetail JOIN
HistoryMainline hmr on hmr.TxnId = RemoveHistoryDetail.TxnId WHERE hmr.ReversalStatus = 1) UNION ALL SELECT null as ContainerId, null as
IssueActualHistoryId, hvIssues.ContainerId as ParentContainerId, hvIssues.FromLot as IssuedFrom, hvIssues.ContainerName as
ParentContainerName, hvIssues.FromLot, null as FromStockPoint, 0 as ContainerLevel, CONCAT(CONCAT(hvIssues.ProductName, ':'),
hvIssues.ProductRevision) as Product, p.Description, hvIssues.NetQtyIssued as QtyIssued, hvIssues.ReferenceDesignator,
hvIssues.HVIssueHistoryDetailId, hvIssues.HVSetupHistoryDetailId as HVSetupDetailId, hvIssues.MaterialListItemId,
hvIssues.MaterialListItemId as IssueKey FROM csiGetHVIssuesForContainer(?Container, '', 2) hvIssues join Product p on p.ProductId =
hvIssues.ProductId WHERE NetQtyIssued > 0 <{?ShowAsBuilt}>><{ UNION ALL SELECT null as ContainerId, null as IssueActualHistoryId,
hvIssues.ContainerId as ParentContainerId, hvIssues.FromLot as IssuedFrom, hvIssues.ContainerName as ParentContainerName,
hvIssues.FromLot, null as FromStockPoint, 0 as ContainerLevel, CONCAT(CONCAT(hvIssues.ProductName, ':'), hvIssues.ProductRevision) as
Product, p.Description, hvIssues.NetQtyIssued as QtyIssued, hvIssues.ReferenceDesignator, hvIssues.HVIssueHistoryDetailId,
hvIssues.HVSetupHistoryDetailId as HVSetupDetailId, hvIssues.MaterialListItemId, hvIssues.MaterialListItemId as IssueKey FROM
ActualsHistory stdIssues cross apply csiGetHVIssuesForContainer(stdIssues.FromContainerId , '', 2) hvIssues join Product p on
p.ProductId = hvIssues.ProductId WHERE NetQtyIssued > 0 }> ORDER BY Product

```

ComponentReplaceInquiry_ContainerSelVal : Oracle

```

WITH ActualsHistory ( FromContainerId, ToContainerId, ProductId, IssueActualsHistoryId, FromLocationId, TxnId, Qty, FromStockPoint,
FromLot, ReferenceDesignator, ContainerLevel ) AS ( SELECT iah.FromContainerId, iah.ToContainerId, iah.ProductId,
iah.IssueActualsHistoryId, iah.FromLocationId, iah.TxnId, iah.Qty, iah.FromStockPoint, iah.FromLot, iah.ReferenceDesignator, 0 as
ContainerLevel FROM IssueActualsHistory iah WHERE iah.ToContainerId = ?Container AND NOT EXISTS ( SELECT 1 FROM RemoveHistoryDetail rhd
WHERE rhd.IssueActualsHistoryId = iah.IssueActualsHistoryId ) <{?ShowAsBuilt}>><{ UNION ALL SELECT iah.FromContainerId,
iah.ToContainerId, iah.ProductId, iah.IssueActualsHistoryId, iah.FromLocationId, iah.TxnId, iah.Qty, iah.FromStockPoint, iah.FromLot,
iah.ReferenceDesignator, ActualsHistory.ContainerLevel + 1 FROM IssueActualsHistory iah INNER JOIN ActualsHistory ON
ActualsHistory.FromContainerId = iah.ToContainerId > } SELECT DISTINCT ActualsHistory.FromContainerId as @header:name="ContainerId",
ActualsHistory.IssueActualsHistoryId as @header:name="IssueActualHistoryId", ActualsHistory.ToContainerId as

```

```
@header:name="ParentContainerId", COALESCE(chC.ContainerName, ActualsHistory.FromLot, ActualsHistory.FromStockPoint) as
@header:name="IssuedFrom":label=Lbl_IssuedFrom, prnC.ContainerName as @header:name="ParentContainerName", ActualsHistory.FromLot as
@header:name="FromLot", ActualsHistory.FromStockPoint as @header:name="FromStockPoint", ActualsHistory.ContainerLevel as
@header:name="ContainerLevel", (CASE WHEN ProductBase.ProductName IS NULL THEN CONCAT(CONCAT(prB.ProductName, ':'), pr.ProductRevision)
ELSE CONCAT(CONCAT(ProductBase.ProductName, ':'), chP.ProductRevision) END) as @header:name="Product":label=Container_Product,
chP.Description as @header:name="Description":label=SelVal_ProductDescription, ActualsHistory.Qty as
@header:name="QtyIssued":label=Lbl_QtyIssued, ActualsHistory.ReferenceDesignator as
@header:name="ReferenceDesignator":label=Lbl_ReferenceDesignator, null as HVIssueHistoryDetailId, null as HVSetupDetailId, null as
MaterialListItemId, ActualsHistory.IssueActualsHistoryId as IssueKey FROM ActualsHistory LEFT JOIN Container chC on
ActualsHistory.FromContainerId = chC.ContainerId LEFT JOIN Container prnC on ActualsHistory.ToContainerId = prnC.ContainerId LEFT JOIN
Product pr on ActualsHistory.ProductId = pr.ProductId LEFT JOIN ProductBase prB on pr.ProductBaseId = prB.ProductBaseId LEFT JOIN
Product chP on chC.ProductId = chP.ProductId LEFT JOIN Product prnP on prnC.ProductId = prnP.ProductId LEFT JOIN ProductBase on
chP.ProductBaseId = ProductBase.ProductBaseId LEFT JOIN Location on ActualsHistory.FromLocationId = Location.LocationId JOIN
HistoryMainline on HistoryMainline.TxnId = ActualsHistory.TxnId AND HistoryMainline.ReversalStatus = 1 WHERE
ActualsHistory.IssueActualsHistoryId not in ( SELECT RemoveHistoryDetail.IssueActualsHistoryId FROM RemoveHistoryDetail JOIN
HistoryMainline hmr on hmr.TxnId = RemoveHistoryDetail.TxnId WHERE hmr.ReversalStatus = 1) UNION ALL SELECT null as ContainerId, null as
IssueActualHistoryId, hvIssues.ContainerId as ParentContainerId, hvIssues.FromLot as IssuedFrom, hvIssues.ContainerName as
ParentContainerName, hvIssues.FromLot, null as FromStockPoint, 0 as ContainerLevel, CONCAT(CONCAT(hvIssues.ProductName, ':'),
hvIssues.ProductRevision) as Product, p.Description, hvIssues.NetQtyIssued as QtyIssued, hvIssues.ReferenceDesignator,
hvIssues.HVIssueHistoryDetailId, hvIssues.HVSetupHistoryDetailId as HVSetupDetailId, hvIssues.MaterialListItemId,
hvIssues.MaterialListItemId as IssueKey FROM table(csiGetHVIssuesForContainer(?Container, '', 2)) hvIssues join Product p on p.ProductId
= hvIssues.ProductId WHERE NetQtyIssued > 0 <{?ShowAsBuilt}>?<{ UNION ALL SELECT null as ContainerId, null as IssueActualHistoryId,
hvIssues.ContainerId as ParentContainerId, hvIssues.FromLot as IssuedFrom, hvIssues.ContainerName as ParentContainerName,
hvIssues.FromLot, null as FromStockPoint, 0 as ContainerLevel, CONCAT(CONCAT(hvIssues.ProductName, ':'), hvIssues.ProductRevision) as
Product, p.Description, hvIssues.NetQtyIssued as QtyIssued, hvIssues.ReferenceDesignator, hvIssues.HVIssueHistoryDetailId,
hvIssues.HVSetupHistoryDetailId as HVSetupDetailId, hvIssues.MaterialListItemId, hvIssues.MaterialListItemId as IssueKey FROM
ActualsHistory stdIssues cross apply table(csiGetHVIssuesForContainer(stdIssues.FromContainerId, '', 2)) hvIssues join Product p on
p.ProductId = hvIssues.ProductId WHERE NetQtyIssued > 0 }> ORDER BY 9
```

GetCDONameFromInstanceId

GetCDONameFromInstanceId

GetCDONameFromInstanceId : SqlServer

```
select CDOName, CDODefID from CDODefinition where CDODefID = convert(int, convert(varbinary(max), '0x' + LEFT(?ObjectInstanceId, 6), 1))
```

GetCDONameFromInstanceId : Oracle

```
select CDOName, CDODefID from CDODefinition where CDODefID = TO_NUMBER(SUBSTR(?ObjectInstanceId,1,6),'XXXXXX')
```

Parameter	Data Type
ObjectInstanceId	String

GetOperatorLogsForObject

GetOperatorLogsForObject

GetOperatorLogsForObject : Generic

```
select OperatorLogId from OperatorLog where ContainerId = ?ObjectInstanceId or NamedDataObjectId = ?ObjectInstanceId or QualityObjectId
= ?ObjectInstanceId or RevisionedObjectId = ?ObjectInstanceId
```

Parameter	Data Type
ObjectInstanceId	String

GetSPCAnnotations

GetSPCAnnotations : Generic

```
SELECT $AddSPCAnnotationHistory.InstanceId AS @fieldheader:name="InstanceId", $AddSPCAnnotationHistory.Annotation AS
@fieldheader:name="Annotation", $AddSPCAnnotationHistory.DataCollectionDef AS @fieldheader:name="DataCollectionDef",
$AddSPCAnnotationHistory.DataCollectionName AS @fieldheader:name="DataCollectionName", $AddSPCAnnotationHistory.DataPoint AS
@fieldheader:name="DataPoint", $AddSPCAnnotationHistory.DataPointName AS @fieldheader:name="DataPointName",
$AddSPCAnnotationHistory.Exclude AS @fieldheader:name="Exclude", $AddSPCAnnotationHistory.SPCCChartDef AS @fieldheader:name="SPCCChartDef"
FROM $HistoryMainline, $AddSPCAnnotationHistory WHERE HistoryMainline.HistoryMainlineId = $AddSPCAnnotationHistory.Parent AND
HistoryMainline.HistoryId = ?DataPoint AND HistoryMainline.ReversalStatus = 1 ORDER BY $AddSPCAnnotationHistory.InstanceId
```

Parameter	Data Type
DataPoint	

MBO_GetDHRHistoryEvents

MBO_GetDHRHistoryEvents : Generic

```
SELECT DISTINCT historymainline.TxnType AS @header : name = "TxnType" : datatype = Integer, historymainline.TxnID AS @header : name =
"TxnID" : datatype = Object, historymainline.TxnServiceSequence AS @header : name = "TxnServiceSequence" : datatype = Integer,
historymainline.TxnServiceName AS @header : name = "TxnServiceName" : datatype = String, ExecuteTaskHistory.TaskType AS @header : name =
"TaskType" : datatype = Integer, TaskItem.TaskItemName AS @header : name = "TaskItemName" : datatype = String,
TaskItem.ReportInstruction AS @header : name = "TaskInstructions" : datatype = String, DataPointHistoryDetail.DataName AS @header : name
= "DataName" : datatype = String, DataPointHistoryDetail.DataValue AS @header : name = "DataValue" : datatype = String, (case when
ExecuteTaskHistory.Pass = 1 then 'PASS' when ExecuteTaskHistory.Pass = 0 then 'FAIL' end) AS @header : name = "TaskStatus" : datatype =
String, DataPointHistoryDetail.LowerLimit AS @header : name = "LowerLimit" : datatype = String, DataPointHistoryDetail.UpperLimit AS
@header : name = "UpperLimit" : datatype = String, TaskListBase.TaskListName AS @header : name = "TaskListName" : datatype = String,
ExecuteTaskHistory.TaskListSequence AS @header : name = "TaskListSequence" : datatype = Integer, ExecuteTaskHistory.TaskTime AS @header
: name = "TaskTime" : datatype = Float, PauseTaskHistory.ElapsedTaskTime AS @header : name = "ElapsedTaskTime" : datatype = Float,
TranWorkflowStep.WorkflowStepName AS @header : name = "WorkflowStepName" : datatype = String,
ElectronicProcedureBase2.ElectronicProcedureName AS @header : name = "ElectronicProcedureName" : datatype = String,
Operation.OperationName AS @header : name = "OperationName" : datatype = String, Employee.EmployeeName AS @header : name =
"EmployeeName" : datatype = String, HistoryMainline.TxnDate AS @header : name = "TxnDate" : datatype = Timestamp,
HistoryMainline.TxnDateGMT AS @header : name = "TxnDateGMT" : datatype = Timestamp, ComputationHistory.ComputationName AS @header : name
= "ComputationName" : datatype = String, ComputationHistory.ResultValue AS @header : name = "ResultValue" : datatype = String,
CPDataTypes.Name AS @header : name = "Name" : datatype = String, --TaskItem.Computation.Instruction, ESigMeaning.ESigMeaningName AS
@header : name = "ESigMeaningName" : datatype = String, ESigHistoryDetail.SignerFullName AS @header : name = "SignerFullName" : datatype
= String, ESigHistoryDetail.CosignerFullName AS @header : name = "CosignerFullName" : datatype = String, HistoryMainline.Comments AS
@header : name = "Comments" : datatype = String, DataPointUOM.UOMName AS @header : name = "UOMName" : datatype = String,
```



```

ExecuteTaskHistory.Sequence AS @header : name = "Sequence" : datatype = Integer , TaskList.ReportInstruction AS @header : name =
"TaskListInstruction" : datatype = String, TranWorkflowBase.WorkflowName AS @header : name = "WorkflowName" : datatype = String
, IssueActualsHistory.FromContainerId AS @header : name = "FromContainerId" : datatype = Object , IssuedContainer.ContainerName AS @header
: name = "ContainerName" : datatype = String , componentPB.ProductName AS @header : name = "IssuedProduct" : datatype = String
, componentProduct.ProductRevision AS @header : name = "IssuedProductRevision" : datatype = String
, IssueActualsHistory.ProductDescription AS @header : name = "IssuedProductDesc" : datatype = String
, IssueActualsHistory.EnteredQtyIssued AS @header : name = "IssueQty" : datatype = String , IssueHistoryDetail.QtyRequired AS @header :
name = "QtyRequired" : datatype = Float , IssueHistoryDetail.QtyLowerLimit AS @header : name = "QtyLowerLimit" : datatype = Float
, IssueHistoryDetail.QtyUpperLimit AS @header : name = "QtyUpperLimit" : datatype = Float , MoveHistory.Qty AS @header : name = "MoveQty"
: datatype = Float , MoveHistory.CycleTime * 24 * 60 AS @header : name = "ProcessTime" : datatype = Float , MoveInHistory.Qty AS @header :
name = "MoveInQty" : datatype = Float , MoveInHistory.CycleTime * 24 * 60 AS @header : name = "QueueTime" : datatype = Integer
, SplitContainer.ContainerName AS @header : name = "SplitContainer" : datatype = String , FromContainer.ContainerName AS @header : name =
"FromContainer" : datatype = String , SplitHistoryDetails.Qty AS @header : name = "qty" : datatype = Float
, AssociatedContainers.ContainerName AS @header : name = "AssociatedContainer" : datatype = String , AssociatedContainers.Qty AS @header :
name = "AssociatedQty" : datatype = Float , associatedUOM.UOMName AS @header : name = "associatedUOM" : datatype = String
, DisAssociatedContainers.ContainerName AS @header : name = "DisAssociatedContainer" : datatype = String , DisAssociatedContainers.Qty AS
@header : name = "DisAssociatedQty" : datatype = Float , DisassociatedUOM.UOMName AS @header : name = "DisassociatedUOM" : datatype =
String , QtyHistoryDetails.Qty AS @header : name = "ChangeQty" : datatype = Float , COALESCE(BuyReason.BuyReasonName,
LossReason.LossReasonName, BonusReason.BonusReasonName, SellReason.SellReasonName, QtyAdjustReason.QtyAdjustReasonName) AS @header :
name = "QtyAdjustReason" : datatype = String , Case when MoveHistory.ReworkReasonId is not null then MoveHistory.Qty ELSE 0 END AS
@header : name = "ReworkedQty" : datatype = Integer , ReworkReason.ReworkReasonName AS @header : name = "ReworkReason" : datatype =
String , HoldReason.HoldReasonName AS @header : name = "HoldReasonName" : datatype = String , ReleaseReason.ReleaseReasonName AS @header :
name = "ReleaseReasonName" : datatype = String , HoldReleaseHistoryDetail.HoldDuration AS @header : name = "HoldDuration" : datatype =
Integer FROM Operation RIGHT OUTER JOIN HistoryMainline ON (Operation.OperationId=HistoryMainline.OperationId) LEFT OUTER JOIN
ESigHistorySummary ON (ESigHistorySummary.TxnId=HistoryMainline.TxnId) LEFT OUTER JOIN ESigHistoryDetail ON
(ESigHistoryDetail.ESigHistorySummaryId=ESigHistorySummary.ESigHistorySummaryId) LEFT OUTER JOIN ESigMeaning ON
(ESigMeaning.ESigMeaningId=ESigHistorySummary.MeaningId) Left JOIN ExecuteTaskHistory ON
(HistoryMainline.HistoryMainlineId=ExecuteTaskHistory.HistoryMainlineId) LEFT OUTER JOIN PauseTaskHistory ON
(HistoryMainline.HistoryMainlineId=PauseTaskHistory.HistoryMainlineId) LEFT OUTER JOIN TaskItem ON
(ExecuteTaskHistory.TaskId=TaskItem.TaskItemId) LEFT OUTER JOIN TaskList ON (TaskItem.TaskListId=TaskList.TaskListId) LEFT OUTER JOIN
TaskListBase ON (TaskList.TaskListBaseId=TaskListBase.TaskListBaseId) LEFT OUTER JOIN ElectronicProcedure ElectronicProcedure2 ON
(ExecuteTaskHistory.ElectronicProcedureId=ElectronicProcedure2.ElectronicProcedureId) LEFT OUTER JOIN ElectronicProcedureBase
ElectronicProcedureBase2 ON (ElectronicProcedure2.ElectronicProcedureBaseId=ElectronicProcedureBase2.ElectronicProcedureBaseId) LEFT
OUTER JOIN DataPointHistory ON (DataPointHistory.HistoryMainlineId=HistoryMainline.HistoryMainlineId) LEFT OUTER JOIN
DataPointHistoryDetail ON (DataPointHistoryDetail.DataPointHistoryId=DataPointHistory.DataPointHistoryId) LEFT OUTER JOIN UOM
DataPointUOM ON (DataPointUOM.UOMId=DataPointHistoryDetail.UOMId) INNER JOIN HistoryCrossRef ON
(HistoryMainline.HistoryId=HistoryCrossRef.HistoryId and HistoryMainline.TxnId between HistoryCrossRef.StartTxnId and
HistoryCrossRef.EndTxnId) LEFT OUTER JOIN Container ON (HistoryCrossRef.TrackingId=Container.ContainerId) LEFT OUTER JOIN
ComputationHistory ON (HistoryMainline.HistoryMainlineId=ComputationHistory.HistoryMainlineId) LEFT OUTER JOIN CPPDataTypes ON
(ComputationHistory.ResultDataType=CPPDataTypes.DataTypeId) LEFT OUTER JOIN Employee ON (Employee.EmployeeId=HistoryMainline.EmployeeId)
LEFT OUTER JOIN WorkflowStep TranWorkflowStep ON (TranWorkflowStep.WorkflowStepId=HistoryMainline.WorkflowStepId) LEFT OUTER JOIN
Workflow TranWorkflow ON (TranWorkflowStep.WorkflowId=TranWorkflow.WorkflowId) LEFT OUTER JOIN WorkflowBase TranWorkflowBase ON
(TranWorkflowBase.WorkflowBaseId=TranWorkflow.WorkflowBaseId) Left OUTER JOIN IssueActualsHistory on HistoryMainline.TxnId =
IssueActualsHistory.TxnId LEFT OUTER JOIN Product componentProduct on IssueActualsHistory.ProductId = componentProduct.ProductId LEFT
OUTER JOIN ProductBase componentPB on componentProduct.ProductBaseId = componentPB.ProductBaseId Left OUTER JOIN IssueHistoryDetail on
IssueActualsHistory.IssueHistoryDetailId = IssueHistoryDetail.IssueHistoryDetailId Left OUTER JOIN Container IssuedContainer on
IssueActualsHistory.FromContainerId = IssuedContainer.ContainerId Left outer join MoveHistory on historymainline.txnid =
MoveHistory.TxnId Left outer join MoveInHistory on historymainline.txnid = MoveInHistory.TxnId Left outer join SplitHistoryDetails on
historymainline.txnid = SplitHistoryDetails.TxnId Left outer join Container SplitContainer on SplitHistoryDetails.ToContainerId =
SplitContainer.ContainerId Left Outer Join Container FromContainer ON SplitContainer.SplitFromId=FromContainer.ContainerId Left Outer
Join AssociateHistory on HistoryMainline.txnid = AssociateHistory.txnid Left outer join AssociateHistoryChildCnts on
AssociateHistory.AssociateHistoryId = AssociateHistoryChildCnts.AssociateHistoryId Left Outer join Container AssociatedContainers on
associateHistoryChildCnts.ChildContainersId = AssociatedContainers.ContainerId Left Outer join UOM associatedUOM on
AssociatedContainers.UOMId = associatedUOM.UOMId Left Outer Join DisassociateHistory on HistoryMainline.txnid =
DisassociateHistory.txnid Left Outer join DisassociateHistoryChildCnts on DisassociateHistory.DisassociateHistoryId =
DisassociateHistoryChildCnts.DisAssociateHistoryId Left Outer join Container DisAssociatedContainers on
DisassociateHistoryChildCnts.ChildContainersId = DisAssociatedContainers.ContainerId Left Outer join UOM DisassociatedUOM on
DisAssociatedContainers.UOMId = DisassociatedUOM.UOMId Left Outer Join HoldReleaseHistory on HistoryMainline.txnid =
HoldReleaseHistory.txnid Left Outer Join HoldReleaseHistoryDetail on HoldReleaseHistory.HoldReleaseHistoryId =
HoldReleaseHistoryDetail.HoldReleaseHistoryId Left Outer Join HoldReason on HoldReleaseHistory.HoldReasonId = HoldReason.HoldReasonId
Left Outer Join ReleaseReason on HoldReleaseHistory.ReleaseReasonId = ReleaseReason.ReleaseReasonId Left outer join QtyHistoryDetails on
HistoryMainline.txnid = QtyHistoryDetails.TxnId Left outer join BuyReason on QtyHistoryDetails.ReasonCodeId = BuyReason.BuyReasonId Left
outer join LossReason on QtyHistoryDetails.ReasonCodeId = LossReason.LossReasonId Left outer join SellReason on
QtyHistoryDetails.ReasonCodeId = SellReason.SellReasonId Left outer join BonusReason on QtyHistoryDetails.ReasonCodeId =
BonusReason.BonusReasonId Left outer join QtyAdjustReason on QtyHistoryDetails.ReasonCodeId = QtyAdjustReason.QtyAdjustReasonId Left
outer join ReworkReason on movehistory.ReworkReasonId =ReworkReason.ReworkReasonId WHERE Container.ContainerName = ?Container and
HistoryMainline.ReversalStatus =1 and historymainline.TxnServiceSequence = 1 Order by HistoryMainline.TxnID

```

OperatorLog_GetCDOName

OperatorLog_GetCDOName

OperatorLog_GetCDOName : Generic

```
SELECT CDONAME FROM CDODefinition WHERE CDODEFID = ?CDODEFID
```

Parameter	Data Type
CDODEFID	Integer

OperatorLogInquiry_GetLogEntries

SelectionValuesEx_OpLog_EntryHistory

OperatorLogInquiry_GetLogEntries : SqlServer

```

select RecordLogEntryHistoryId from RecordLogEntryHistory logEntry join HistoryMainline hm on hm.HistoryMainlineId =
logEntry.HistoryMainlineId where logEntry.OperatorLogId = ?OperatorLog <{?StartDateTime}>?<{ AND hm.TxnDateGMT >= ?StartDateTime}> <{?
EndDateTime}>?<{ AND hm.TxnDateGMT <= ?EndDateTime}> <{?LogEntryTypeFilter}>?<{ AND logEntry.LogEntryTypeName like ?LogEntryTypeFilter}>
<{?LogEntryFilter}>?<{ AND logEntry.LogEntry like ?LogEntryFilter}> order by hm.TxnDate desc

```

OperatorLogInquiry_GetLogEntries : Oracle

```

select RecordLogEntryHistoryId from RecordLogEntryHistory logEntry join HistoryMainline hm on hm.HistoryMainlineId =
logEntry.HistoryMainlineId where logEntry.OperatorLogId = ?OperatorLog <{?StartDateTime}>?<{ AND hm.TxnDateGMT >= ?StartDateTime}> <{?
EndDateTime}>?<{ AND hm.TxnDateGMT <= ?EndDateTime}> <{?LogEntryTypeFilter}>?<{ AND logEntry.LogEntryTypeName like ?LogEntryTypeFilter}>
<{?LogEntryFilter}>?<{ AND LOWER(logEntry.LogEntry) like LOWER(?LogEntryFilter)}> order by hm.TxnDate desc

```

Parameter	Data Type
OperatorLog	Object
StartDateTime	TimeStamp
EndDateTime	TimeStamp
LogEntryTypeFilter	String
LogEntryFilter	String

SelectionValuesEx_OpLog_EntryHistory

SelectionValuesEx_OpLog_EntryHistory

SelectionValuesEx_OpLog_EntryHistory : SqlServer

```
select logEntries.RecordLogEntryHistoryId AS @header:name="InstanceID":datatype=Object, log.OperatorLogName AS
@header:name="OperatorLogName":datatype=String, hm.TxnDate AS @header:name="TxnDate":datatype=TimeStamp, e.EmployeeName AS
@header:name="Employee":datatype=String, logEntries.LogEntry AS @header:name="LogEntry":datatype=String, entryType.LogEntryTypeName AS
@header:name="LogEntryType":datatype=String from RecordLogEntryHistory logEntries join HistoryMainline hm on
logEntries.HistoryMainlineId = hm.HistoryMainlineId join OperatorLog log on logEntries.OperatorLogId = log.OperatorLogId left join
Employee e on hm.EmployeeId = e.EmployeeId left join LogEntryType entryType on logEntries.LogEntryTypeId = entryType.LogEntryTypeId
where log.OperatorLogId = ?OperatorLog and (?StartDateTime = '' or hm.TxnDate >= ?StartDateTime) and (?EndDateTime = '' or hm.TxnDate <=
?EndDateTime) and (?LogEntryType = '' or logEntries.LogEntryTypeId = ?LogEntryType) and (?LogEntry = '' or logEntries.LogEntry like '%'
+ ?LogEntry + '%') order by hm.TxnDate desc
```

SelectionValuesEx_OpLog_EntryHistory : Oracle

```
select logEntries.RecordLogEntryHistoryId AS @header:name="InstanceID":datatype=Object, log.OperatorLogName AS
@header:name="OperatorLogName":datatype=String, hm.TxnDate AS @header:name="TxnDate":datatype=TimeStamp, e.EmployeeName AS
@header:name="Employee":datatype=String, logEntries.LogEntry AS @header:name="LogEntry":datatype=String, entryType.LogEntryTypeName AS
@header:name="LogEntryType":datatype=String from RecordLogEntryHistory logEntries join HistoryMainline hm on
logEntries.HistoryMainlineId = hm.HistoryMainlineId join OperatorLog log on logEntries.OperatorLogId = log.OperatorLogId left join
Employee e on hm.EmployeeId = e.EmployeeId left join LogEntryType entryType on logEntries.LogEntryTypeId = entryType.LogEntryTypeId
where log.OperatorLogId = ?OperatorLog and (?StartDateTime is null or hm.TxnDate >= ?StartDateTime) and (?EndDateTime is null or
hm.TxnDate <= ?EndDateTime) and (?LogEntryType is null or logEntries.LogEntryTypeId = ?LogEntryType) and (?LogEntry is null or
LOWER(logEntries.LogEntry) like LOWER('%' || ?LogEntry || '%')) order by hm.TxnDate desc
```

Parameter	Data Type
OperatorLog	Object
StartDateTime	TimeStamp
EndDateTime	TimeStamp
LogEntryType	Object
LogEntry	String

SelectionValuesEx_OpLog_RDOAndNDOObjects

SelectionValuesEx_OpLog_RDOAndNDOObjects

SelectionValuesEx_OpLog_RDOAndNDOObjects : SqlServer

```
SELECT Data.CDOName AS @header:name = "Name":label=SelVal_CDOName:datatype=String, Data.CDODefId AS @header:name =
"CDODefId":label=SelVal_CDODefId:datatype=Integer, ISNULL(Term1.LabelValue, ISNULL(Lang1.LabelValue, DataLabels.LabelValue)) AS
@header:name = "CDODisplayName":label=SelVal_CDODisplayName:datatype=String, Parent.CDODefId AS @header:name =
"ParentCDODefId":datatype=Integer, Parent.CDOName AS @header:name = "ParentCDOName":datatype=String, Data.IsAbstract AS @header:name =
"Abstract":label=SelVal_Abstract:datatype=Boolean, CASE (Data.CDOUsageMask & 4202514) WHEN 2 THEN 1 WHEN 16 THEN 2 WHEN 8194 THEN 3 WHEN
4194306 THEN 4 ELSE 0 END AS @header:name = "CDOBaseType":datatype=Integer, Owner.CDODefId AS @header:name =
"OwnerCDODefId":datatype=Integer:label=SelVal_OwnerTypeId, Owner.CDOName AS @header:name =
"OwnerCDOName":label=SelVal_CDOName:datatype=String:label=SelVal_OwnerTypeName FROM CDODefinition AS Data INNER JOIN CDODefinition
AS Parent ON Parent.CDODefId = Data.ParentCDODefId LEFT JOIN CDODefinition AS Owner ON Owner.CDODefId = Data.RevisionTypeId INNER JOIN
Labels AS DataLabels ON DataLabels.LabelId = Data.DisplayNameLabelId LEFT JOIN DictionaryLabel AS Term1 ON Term1.LabelId =
DataLabels.LabelId AND Term1.DictionaryId = $$primaryDictionary LEFT JOIN DictionaryLabel AS Lang1 ON Lang1.LabelId = DataLabels.LabelId
AND Lang1.DictionaryId = $$secondaryDictionary WHERE Data.CDODefId not in (7697) -- Exclude ProcessModel And Data.IsAbstract = 0 AND
((Data.CDOUsageMask & 8210) IN (2,16,8194) or (Data.CDOUsageMask & 4210723) in (4210723)) AND Parent.CDODefId <> 1201 -- Excludes Sub
Entities AND Data.CDODefId not in (select CDODefId from CDODefinition where CDOName like '%Base' OR CDOName like '%Changes' OR CDOName
like '%Maint') AND DATA.CDODefId not in (7187, 8141, 7754, 8492, 7751, 8477, 8091) AND Data.CDOName LIKE ?NameFilter Order by
ISNULL(Term1.LabelValue, ISNULL(Lang1.LabelValue, DataLabels.LabelValue))--Data.CDOName
```

SelectionValuesEx_OpLog_RDOAndNDOObjects : Oracle

```
SELECT Data.CDOName AS @header:name = "CDOName":label=SelVal_CDOName:datatype=String, Data.CDODefId AS @header:name =
"CDODefId":label=SelVal_CDODefId:datatype=Integer, NVL(Term1.LabelValue, NVL(Lang1.LabelValue, DataLabels.LabelValue)) AS @header:name
= "CDODisplayName":label=SelVal_CDODisplayName:datatype=String, Parent.CDODefId AS @header:name = "ParentCDODefId":datatype=Integer,
Parent.CDOName AS @header:name = "ParentCDOName":datatype=String, Data.IsAbstract AS @header:name =
"Abstract":label=SelVal_Abstract:datatype=Boolean, CASE BITAND(Data.CDOUsageMask,4202514) WHEN 2 THEN 1 WHEN 16 THEN 2 WHEN 8194 THEN 3
WHEN 4194306 THEN 4 ELSE 0 END AS @header:name = "CDOBaseType":datatype=Integer, Owner.CDODefId AS @header:name =
"OwnerCDODefId":datatype=Integer:label=SelVal_OwnerTypeId, Owner.CDOName AS @header:name =
"OwnerCDOName":label=SelVal_CDOName:datatype=String:label=SelVal_OwnerTypeName FROM CDODefinition Data INNER JOIN CDODefinition
Parent ON Parent.CDODefId = Data.ParentCDODefId LEFT JOIN CDODefinition Owner ON Owner.CDODefId = Data.RevisionTypeId INNER JOIN Labels
DataLabels ON DataLabels.LabelId = Data.DisplayNameLabelId LEFT JOIN DictionaryLabel Term1 ON Term1.LabelId = DataLabels.LabelId AND
Term1.DictionaryId = $$primaryDictionary LEFT JOIN DictionaryLabel Lang1 ON Lang1.LabelId = DataLabels.LabelId AND Lang1.DictionaryId =
$$secondaryDictionary WHERE Data.CDODefId not in (7697) -- Exclude ProcessModel And Data.IsAbstract = 0 AND (BITAND(Data.CDOUsageMask,
8210) IN (2,16,8194) or BITAND(Data.CDOUsageMask, 4210723) in (4210723)) AND Parent.CDODefId <> 1201 AND Data.IsAbstract = 0 AND
Data.CDODefId not in (select CDODefId from CDODefinition where CDOName like '%Base' OR CDOName like '%Changes' OR CDOName like '%Maint')
AND Data.CDODefId not in (7187, 8141, 7754, 8492, 7751, 8477, 8091) AND Data.CDOName LIKE ?NameFilter ORDER By
COALESCE(Term1.LabelValue, Lang1.LabelValue, DataLabels.LabelValue)
```

Parameter	Data Type
NameFilter	String

SelectionValuesEx_TaskList

SelectionValuesEx_TaskList : Generic

```
SELECT $TaskListBase.Name, $TaskList.Revision, $TaskListBase.RevOfRcd, $TaskList.IsFrozen, $TaskList.InstanceId, $TaskList.Description,
$TaskList.Status, $TaskList.Base, $TaskList.VariantExpression FROM $ElectronicProcedure INNER JOIN $ProcedureDetail ON
$ElectronicProcedure.InstanceId = $ProcedureDetail.Parent INNER JOIN $TaskList on ($ProcedureDetail.TaskList = $TaskList.InstanceId OR
$ProcedureDetail.TaskListBaseId = $TaskList.Base) INNER JOIN $TaskListBase on $TaskListBase.InstanceId = $TaskList.Base WHERE
$ElectronicProcedure.InstanceId = ?Procedure AND ($ProcedureDetail.TaskList = $TaskList.InstanceId OR ($ProcedureDetail.TaskListBaseId
```

```
= $TaskListBase.InstanceId AND $TaskListBase.RevOfRcd = $TaskList.InstanceId)) ORDER BY $EProcedureDetail.Sequence, $TaskListBase.Name, $TaskList.Revision
```

SelectionValuesEx_TasksForEProc

SelectionValuesEx_TasksForEProc : SqlServer

```
SELECT tl.TaskListId as @header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name="TaskListName":datatype=String ,tl.TaskListRevision as @header:name="TaskListRevision":datatype=String ,tl.Instruction as @header:name="TaskListInstruction":datatype=String ,ti.TaskItemName as @header:name="TaskName":datatype=String ,ti.TaskItemId as @header:name="TaskId":datatype=Object ,ti.Instruction as @header:name="TaskInstruction":datatype=String ,epd.Sequence as @header:name="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name="TaskSequence":datatype=Integer ,ti.MinIterations as @header:name="MinIterations":datatype=Integer ,ti.MaxIterations as @header:name="MaxIterations":datatype=Integer ,case when a.Pass is NULL then 2 else a.Pass end as @header:name="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then 0 else 1 end as @header:name="IsComputation":datatype=Boolean ,coalesce(A.NbrTimesProcessed,0) as @header:name="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as @header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1 then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as @header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer ,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as @header:name="InstructionType":datatype=Integer ,ti.AdvanceToNextTask as @header:name="AdvanceToNextTask":datatype=Boolean FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId = epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList tl on epd.TaskListId = tl.TaskListId or tlb.RevOfRcdId = tl.TaskListId inner join TaskListBase tlb2 on tl.TaskListBaseId = tlb2.TaskListBaseId inner join TaskItem ti on tl.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join (SELECT cct.Pass, cct.NbrTimesProcessed, cct.TaskId FROM Container c inner join CurrentStatus cs on c.CurrentStatusId = cs.CurrentStatusId left join ContainerCompletedTask cct on cs.CurrentStatusId = cct.CurrentStatusId WHERE c.ContainerId = ?Container) a on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM csiGetDerivedCDOs(1, 7463)) cdo on cdo.CDODefId = ti.CDOTypeId WHERE ep.ElectronicProcedureId = ?ElectronicProcedure ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName, tl.TaskListRevision, ti.TaskItemName
```

SelectionValuesEx_TasksForEProc : Oracle

```
SELECT tl.TaskListId as @header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name="TaskListName":datatype=String ,tl.TaskListRevision as @header:name="TaskListRevision":datatype=String ,tl.Instruction as @header:name="TaskListInstruction":datatype=String ,ti.TaskItemName as @header:name="TaskName":datatype=String ,ti.TaskItemId as @header:name="TaskId":datatype=Object ,ti.Instruction as @header:name="TaskInstruction":datatype=String ,epd.Sequence as @header:name="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name="TaskSequence":datatype=Integer ,ti.MinIterations as @header:name="MinIterations":datatype=Integer ,ti.MaxIterations as @header:name="MaxIterations":datatype=Integer ,case when a.Pass is NULL then 2 else a.Pass end as @header:name="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then 0 else 1 end as @header:name="IsComputation":datatype=Boolean ,coalesce(A.NbrTimesProcessed,0) as @header:name="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as @header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1 then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as @header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer ,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as @header:name="InstructionType":datatype=Integer ,ti.AdvanceToNextTask as @header:name="AdvanceToNextTask":datatype=Boolean FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId = epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList tl on epd.TaskListId = tl.TaskListId or tlb.RevOfRcdId = tl.TaskListId inner join TaskListBase tlb2 on tl.TaskListBaseId = tlb2.TaskListBaseId inner join TaskItem ti on tl.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join (SELECT cct.Pass, cct.NbrTimesProcessed, cct.TaskId FROM Container c inner join CurrentStatus cs on c.CurrentStatusId = cs.CurrentStatusId left join ContainerCompletedTask cct on cs.CurrentStatusId = cct.CurrentStatusId WHERE c.ContainerId = ?Container) a on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM CDODefinition START WITH CDODefId = 7463 CONNECT BY ParentCDODefId = PRIOR CDODefId) cdo on cdo.CDODefId = ti.CDOTypeId WHERE ep.ElectronicProcedureId = ?ElectronicProcedure ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName, tl.TaskListRevision, ti.TaskItemName
```

SelectionValuesEx_TasksForEProc : PostgreSQL

```
SELECT tl.TaskListId as @header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name="TaskListName":datatype=String ,tl.TaskListRevision as @header:name="TaskListRevision":datatype=String ,tl.Instruction as @header:name="TaskListInstruction":datatype=String ,ti.TaskItemName as @header:name="TaskName":datatype=String ,ti.TaskItemId as @header:name="TaskId":datatype=Object ,ti.Instruction as @header:name="TaskInstruction":datatype=String ,epd.Sequence as @header:name="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name="TaskSequence":datatype=Integer ,ti.MinIterations as @header:name="MinIterations":datatype=Integer ,ti.MaxIterations as @header:name="MaxIterations":datatype=Integer ,case when a.Pass is NULL then 2 else a.Pass end as @header:name="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then 0 else 1 end as @header:name="IsComputation":datatype=Boolean ,coalesce(A.NbrTimesProcessed,0) as @header:name="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as @header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1 then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as @header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer ,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as @header:name="InstructionType":datatype=Integer ,ti.AdvanceToNextTask as @header:name="AdvanceToNextTask":datatype=Boolean FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId = epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList tl on epd.TaskListId = tl.TaskListId or tlb.RevOfRcdId = tl.TaskListId inner join TaskListBase tlb2 on tl.TaskListBaseId = tlb2.TaskListBaseId inner join TaskItem ti on tl.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join (SELECT cct.Pass, cct.NbrTimesProcessed, cct.TaskId FROM Container c inner join CurrentStatus cs on c.CurrentStatusId = cs.CurrentStatusId left join ContainerCompletedTask cct on cs.CurrentStatusId = cct.CurrentStatusId WHERE c.ContainerId = ?Container) a on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM csiGetDerivedCDOs(1, 7463)) cdo on cdo.CDODefId = ti.CDOTypeId WHERE ep.ElectronicProcedureId = ?ElectronicProcedure ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName, tl.TaskListRevision, ti.TaskItemName
```

SelectionValuesEx_TasksForTasklist

SelectionValuesEx_TasksForTasklist : SqlServer

```
SELECT tl.TaskListId as @header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name="TaskListName":datatype=String ,tl.TaskListRevision as @header:name="TaskListRevision":datatype=String ,tl.Instruction as @header:name="TaskListInstruction":datatype=String ,ti.TaskItemName as @header:name="TaskName":datatype=String ,ti.TaskItemId as @header:name="TaskId":datatype=Object ,ti.Instruction as @header:name="TaskInstruction":datatype=String ,epd.Sequence as @header:name="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name="TaskSequence":datatype=Integer ,ti.MinIterations as @header:name="MinIterations":datatype=Integer ,ti.MaxIterations as @header:name="MaxIterations":datatype=Integer ,case when a.Pass is NULL then 2 else a.Pass end as @header:name="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then 0 else 1 end as @header:name="IsComputation":datatype=Boolean ,coalesce(A.NbrTimesProcessed,0) as @header:name="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as
```

```

@header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1
then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as
@header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer
,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as
@header:name="InstructionType":datatype=Integer ,ti.AdvanceToNextTask as @header:name="AdvanceToNextTask":datatype=Boolean
,ti.VariantExpression as @header:name="VariantExpression":datatype=String ,ti.MinIterationsExpression as
@header:name="MinIterationsExpression":datatype=String ,ti.MaxIterationsExpression as
@header:name="MaxIterationsExpression":datatype=String FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on
ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId =
epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList t1 on epd.TaskListId
= t1.TaskListId or tlb.RevOfRcdId = t1.TaskListId inner join TaskListBase tlb2 on t1.TaskListBaseId = tlb2.TaskListBaseId inner join
TaskItem ti on t1.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join
(SELECT cct.Pass, cct.NbrTimesProcessed, cct.TaskId FROM Container c inner join CurrentStatus cs on c.CurrentStatusId =
cs.CurrentStatusId left join ContainerCompletedTask cct on cs.CurrentStatusId = cct.CurrentStatusId WHERE c.ContainerId = ?Container AND
(cct.WorkflowStepId IS NULL OR cct.WorkflowStepId = cs.WorkflowStepId)) a on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM
csiGetDerivedCDOs(1, 7463)) cdo on cdo.CDODefId = ti.CDOTypeId WHERE ep.ElectronicProcedureId = ?ElectronicProcedure AND t1.TaskListId =
?TaskList ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName, t1.TaskListRevision, ti.TaskItemName

```

SelectionValuesEx_TasksForTasklist : Oracle

```

SELECT t1.TaskListId as @header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name="TaskListName":datatype=String
,t1.TaskListRevision as @header:name="TaskListRevision":datatype=String ,t1.Instruction as @header:name
="TaskListInstruction":datatype=String ,ti.TaskItemName as @header:name="TaskName":datatype=String ,ti.TaskItemId as @header:name
="TaskId":datatype=Object ,ti.Instruction as @header:name="TaskInstruction":datatype=String ,epd.Sequence as @header:name
="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name="TaskSequence":datatype=Integer ,ti.MinIterations as
@header:name="MinIterations":datatype=Integer ,ti.MaxIterations as @header:name="MaxIterations":datatype=Integer ,case when a.Pass is
NULL then 2 else a.Pass end as @header:name="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then
0 else 1 end as @header:name="IsComputation":datatype=Boolean ,coalesce(A.NbrTimesProcessed,0) as @header:name
="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as
@header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1
then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as
@header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer
,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as
@header:name="InstructionType":datatype=Integer ,ti.AdvanceToNextTask as @header:name="AdvanceToNextTask":datatype=Boolean
,ti.VariantExpression as @header:name="VariantExpression":datatype=String ,ti.MinIterationsExpression as
@header:name="MinIterationsExpression":datatype=String ,ti.MaxIterationsExpression as
@header:name="MaxIterationsExpression":datatype=String FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on
ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId =
epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList t1 on epd.TaskListId
= t1.TaskListId or tlb.RevOfRcdId = t1.TaskListId inner join TaskListBase tlb2 on t1.TaskListBaseId = tlb2.TaskListBaseId inner join
TaskItem ti on t1.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join
(SELECT cct.Pass, cct.NbrTimesProcessed, cct.TaskId FROM Container c inner join CurrentStatus cs on c.CurrentStatusId =
cs.CurrentStatusId left join ContainerCompletedTask cct on cs.CurrentStatusId = cct.CurrentStatusId WHERE c.ContainerId = ?Container AND
(cct.WorkflowStepId IS NULL OR cct.WorkflowStepId = cs.WorkflowStepId)) a on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM
CDODefinition START WITH CDODefId = 7463 CONNECT BY ParentCDOId = PRIOR CDODefId) cdo on cdo.CDODefId = ti.CDOTypeId WHERE
ep.ElectronicProcedureId = ?ElectronicProcedure AND t1.TaskListId = ?TaskList ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName,
t1.TaskListRevision, ti.TaskItemName

```

SelectionValuesEx_TasksForTasklist : PostgreSQL

```

WITH chd AS ( SELECT ContainerId, CurrentStatusId, StartDate, LEAD(StartDate) OVER (PARTITION BY ContainerId ORDER BY StartDate) AS
EndDate, CurrentStatusHistoryDetailsId FROM CurrentStatusHistoryDetails), cte AS ( SELECT subquery.TaskId,
MAX(subquery.NbrTimesProcessed) AS TotalNbrTimesProcessed FROM ( SELECT cct.TaskId, ROW_NUMBER() OVER (PARTITION BY cct.TaskId ORDER BY
cct.CompletionTimestampGMT DESC) AS NbrTimesProcessed, cct.CompletionTimestampGMT, cct.CurrentStatusId, ROW_NUMBER() OVER (PARTITION BY
cct.TaskId ORDER BY cct.CompletionTimestampGMT DESC) AS rn, css.StartDate, css.EndDate FROM ( SELECT chd.CurrentStatusId, chd.StartDate,
chd.EndDate FROM Container c LEFT JOIN chd ON c.ContainerId = chd.ContainerId WHERE c.ContainerId = ?Container ) css LEFT JOIN
ContainerCompletedTask cct ON cct.CurrentStatusId = css.CurrentStatusId WHERE (cct.CompletionTimestampGMT <= css.EndDate OR css.EndDate
IS NULL) AND cct.CompletionTimestampGMT >= css.StartDate ) subquery GROUP BY subquery.TaskId ) SELECT t1.TaskListId as
@header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name="TaskListName":datatype=String ,t1.TaskListRevision as
@header:name="TaskListRevision":datatype=String ,t1.Instruction as @header:name="TaskListInstruction":datatype=String ,ti.TaskItemName
as @header:name="TaskName":datatype=String ,ti.TaskItemId as @header:name="TaskId":datatype=Object ,ti.Instruction as @header:name
="TaskInstruction":datatype=String ,epd.Sequence as @header:name="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name
="TaskSequence":datatype=Integer ,ti.MinIterations as @header:name="MinIterations":datatype=Integer ,ti.MaxIterations as
@header:name="MaxIterations":datatype=Integer ,case when a.Pass is NULL then 2 else a.Pass end as
@header:name="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then 0 else 1 end as
@header:name="IsComputation":datatype=Boolean ,coalesce(a.TotalNbrTimesProcessed,0) as @header:name
="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as
@header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1
then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as
@header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer
,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as
@header:name="InstructionType":datatype=Integer ,case when ti.AdvanceToNextTask=0 then 0 else 1 end as
@header:name="AdvanceToNextTask":datatype=Boolean FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on
ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId =
epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList t1 on epd.TaskListId
= t1.TaskListId or tlb.RevOfRcdId = t1.TaskListId inner join TaskListBase tlb2 on t1.TaskListBaseId = tlb2.TaskListBaseId inner join
TaskItem ti on t1.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join
(SELECT sub.TaskId, sub.Pass, cte.TotalNbrTimesProcessed , sub.CompletionTimestampGMT, sub.CurrentStatusId, sub.StartDate, sub.EndDate
FROM ( SELECT cct.TaskId, cct.Pass, cct.NbrTimesProcessed, cct.CompletionTimestampGMT, cct.CurrentStatusId, ROW_NUMBER() OVER (PARTITION
BY cct.TaskId ORDER BY cct.CompletionTimestampGMT DESC) AS rn, css.StartDate, css.EndDate FROM ( SELECT chd.CurrentStatusId,
chd.StartDate, chd.EndDate FROM Container c LEFT JOIN chd ON c.ContainerId = chd.ContainerId WHERE c.ContainerId = ?Container ) css LEFT
JOIN ContainerCompletedTask cct ON cct.CurrentStatusId = css.CurrentStatusId WHERE (cct.CompletionTimestampGMT <= css.EndDate OR
css.EndDate IS NULL) AND cct.CompletionTimestampGMT >= css.StartDate ) sub LEFT JOIN cte ON sub.TaskId = cte.TaskId WHERE sub.rn = 1 ) a
on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM csiGetDerivedCDOs(1, 7463)) cdo on cdo.CDODefId = ti.CDOTypeId WHERE
ep.ElectronicProcedureId = ?ElectronicProcedure AND t1.TaskListId = ?TaskList ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName,
t1.TaskListRevision, ti.TaskItemName

```

SelectionValuesEx_TasksForWorkstation

SelectionValuesEx_TasksForWorkstation : SqlServer

```

SELECT t1.TaskListId as @header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name="TaskListName":datatype=String
,t1.TaskListRevision as @header:name="TaskListRevision":datatype=String ,t1.Instruction as @header:name
="TaskListInstruction":datatype=String ,ti.TaskItemName as @header:name="TaskName":datatype=String ,ti.TaskItemId as @header:name
="TaskId":datatype=Object ,ti.Instruction as @header:name="TaskInstruction":datatype=String ,epd.Sequence as @header:name
="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name="TaskSequence":datatype=Integer ,ti.MinIterations as
@header:name="MinIterations":datatype=Integer ,ti.MaxIterations as @header:name="MaxIterations":datatype=Integer ,case when a.Pass is

```

```

NULL then 2 else a.Pass end as @header:name ="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then
0 else 1 end as @header:name="IsComputation":datatype=Boolean ,coalesce(A.NbrTimesProcessed,0) as @header:name
="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as
@header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1
then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as
@header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer
,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as
@header:name="InstructionType":datatype=Integer ,ti.AdvanceToNextTask as @header:name="AdvanceToNextTask":datatype=Boolean
,ti.VariantExpression as @header:name="VariantExpression":datatype=String ,ti.MinIterationsExpression as
@header:name="MinIterationsExpression":datatype=String ,ti.MaxIterationsExpression as
@header:name="MaxIterationsExpression":datatype=String FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on
ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId =
epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList tl on epd.TaskListId
= tl.TaskListId or tlb.RevOfRcdId = tl.TaskListId inner join TaskListBase tlb2 on tl.TaskListBaseId = tlb2.TaskListBaseId inner join
TaskItem ti on tl.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join
(SELECT cct.Pass, cct.NbrTimesProcessed, cct.TaskId FROM Container c inner join CurrentStatus cs on c.CurrentStatusId =
cs.CurrentStatusId left join ContainerCompletedTask cct on cs.CurrentStatusId = cct.CurrentStatusId WHERE c.ContainerId = ?Container AND
(cct.WorkflowStepId IS NULL OR cct.WorkflowStepId = cs.WorkflowStepId)) a on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM
csiGetDerivedCDOs(1, 7463)) cdo on cdo.CDODefId = ti.CDODefId WHERE ep.ElectronicProcedureId = ?ElectronicProcedure AND
((tl.WorkstationId is null and tl.WorkstationGroupId is null) <{?Workstation }>?{ OR (?Workstation = tl.WorkstationId ) OR ( ?
Workstation in (select EntriesId from ResourceGroupEntries where ResourceGroupId = tl.WorkstationGroupId)) >}) ORDER BY epd.Sequence,
ti.Sequence, tlb.TaskListName, tl.TaskListRevision, ti.TaskItemName

```

SelectionValuesEx_TasksForWorkstation : Oracle

```

SELECT tl.TaskListId as @header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name ="TaskListName":datatype=String
,tl.TaskListRevision as @header:name ="TaskListRevision":datatype=String ,tl.Instruction as @header:name
="TaskListInstruction":datatype=String ,ti.TaskItemName as @header:name ="TaskName":datatype=String ,ti.TaskItemId as @header:name
="TaskId":datatype=Object ,ti.Instruction as @header:name ="TaskInstruction":datatype=String ,epd.Sequence as @header:name
="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name ="TaskSequence":datatype=Integer ,ti.MinIterations as
@header:name="MinIterations":datatype=Integer ,ti.MaxIterations as @header:name="MaxIterations":datatype=Integer ,case when a.Pass is
NULL then 2 else a.Pass end as @header:name ="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then
0 else 1 end as @header:name="IsComputation":datatype=Boolean ,coalesce(A.NbrTimesProcessed,0) as @header:name
="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as
@header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1
then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as
@header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer
,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as
@header:name="InstructionType":datatype=Integer ,ti.AdvanceToNextTask as @header:name="AdvanceToNextTask":datatype=Boolean
,ti.VariantExpression as @header:name="VariantExpression":datatype=String ,ti.MinIterationsExpression as
@header:name="MinIterationsExpression":datatype=String ,ti.MaxIterationsExpression as
@header:name="MaxIterationsExpression":datatype=String FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on
ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId =
epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList tl on epd.TaskListId
= tl.TaskListId or tlb.RevOfRcdId = tl.TaskListId inner join TaskListBase tlb2 on tl.TaskListBaseId = tlb2.TaskListBaseId inner join
TaskItem ti on tl.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join
(SELECT cct.Pass, cct.NbrTimesProcessed, cct.TaskId FROM Container c inner join CurrentStatus cs on c.CurrentStatusId =
cs.CurrentStatusId left join ContainerCompletedTask cct on cs.CurrentStatusId = cct.CurrentStatusId WHERE c.ContainerId = ?Container AND
(cct.WorkflowStepId IS NULL OR cct.WorkflowStepId = cs.WorkflowStepId)) a on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM
CDODefinition START WITH CDODefId = 7463 CONNECT BY ParentCDODId = PRIOR CDODefId) cdo on cdo.CDODefId = ti.CDODefId WHERE
ep.ElectronicProcedureId = ?ElectronicProcedure AND ((tl.WorkstationId is null and tl.WorkstationGroupId is null) <{?Workstation }>?{
OR (?Workstation = tl.WorkstationId ) OR (?Workstation in (select EntriesId from ResourceGroupEntries where ResourceGroupId =
tl.WorkstationGroupId)) >}) ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName, tl.TaskListRevision, ti.TaskItemName

```

SelectionValuesEx_TasksForWorkstation : PostgreSQL

```

WITH chd AS ( SELECT ContainerId, CurrentStatusId, StartDate, LEAD(StartDate) OVER (PARTITION BY ContainerId ORDER BY StartDate) AS
EndDate, CurrentStatusHistoryDetailsId FROM CurrentStatusHistoryDetails), cte AS ( SELECT subquery.TaskId,
MAX(subquery.NbrTimesProcessed) AS TotalNbrTimesProcessed FROM ( SELECT cct.TaskId, ROW NUMBER() OVER (PARTITION BY cct.TaskId ORDER BY
cct.CompletionTimestampGMT DESC) AS NbrTimesProcessed, cct.CompletionTimestampGMT, cct.CurrentStatusId, ROW NUMBER() OVER (PARTITION BY
cct.TaskId ORDER BY cct.CompletionTimestampGMT DESC) AS rn, css.StartDate, css.EndDate FROM ( SELECT chd.CurrentStatusId, chd.StartDate,
chd.EndDate FROM Container c LEFT JOIN chd ON c.ContainerId = chd.ContainerId WHERE c.ContainerId = ?Container ) css LEFT JOIN
ContainerCompletedTask cct ON cct.CurrentStatusId = css.CurrentStatusId WHERE (cct.CompletionTimestampGMT <= css.EndDate OR css.EndDate
IS NULL) AND cct.CompletionTimestampGMT >= css.StartDate ) subquery GROUP BY subquery.TaskId ) SELECT tl.TaskListId as
@header:name="TaskListId":datatype=Object ,tlb2.TaskListName as @header:name ="TaskListName":datatype=String ,tl.TaskListRevision as
@header:name ="TaskListRevision":datatype=String ,tl.Instruction as @header:name ="TaskListInstruction":datatype=String ,ti.TaskItemName
as @header:name ="TaskName":datatype=String ,ti.TaskItemId as @header:name ="TaskId":datatype=Object ,ti.Instruction as @header:name
="TaskInstruction":datatype=String ,epd.Sequence as @header:name ="TaskListSequence":datatype=Integer ,ti.Sequence as @header:name
="TaskSequence":datatype=Integer ,ti.MinIterations as @header:name="MinIterations":datatype=Integer ,ti.MaxIterations as
@header:name="MaxIterations":datatype=Integer ,case when a.Pass is NULL then 2 else a.Pass end as
@header:name="TaskStatus":label=SelVal_TaskStatus:datatype=Integer ,case when cdo.CDODefId is NULL then 0 else 1 end as
@header:name="IsComputation":datatype=Boolean ,coalesce(a.TotalNbrTimesProcessed,0) as @header:name
="NbrTimesProcessed":datatype=Integer ,ti.DocumentSetId as @header:name="DocumentSetId":datatype=Object ,ti.TaskType as
@header:name="TaskType":datatype=Integer ,ti.TransactionPageId as @header:name="TransactionPageId":datatype=Object ,case when TaskType=1
then 1 else 0 end as @header:name="WorkstationRequired":datatype=Integer ,ep.ExecutionMode as
@header:name="EProcedureExecutionMode":datatype=Integer ,tl.ExecutionMode as @header:name="TaskListExecutionMode":datatype=Integer
,uivp.UIVirtualPageName as @header:name="UIVirtualPageName":datatype=String ,ti.InstructionType as
@header:name="InstructionType":datatype=Integer ,case when ti.AdvanceToNextTask=0 then 0 else 1 end as
@header:name="AdvanceToNextTask":datatype=Boolean FROM ElectronicProcedure ep inner join ElectronicProcedureBase epb on
ep.ElectronicProcedureBaseId = epb.ElectronicProcedureBaseId inner join EProcedureDetail epd on ep.ElectronicProcedureId =
epd.ElectronicProcedureId left join TaskListBase tlb on epd.TaskListBaseId = tlb.TaskListBaseId inner join TaskList tl on epd.TaskListId
= tl.TaskListId or tlb.RevOfRcdId = tl.TaskListId inner join TaskListBase tlb2 on tl.TaskListBaseId = tlb2.TaskListBaseId inner join
TaskItem ti on tl.TaskListId = ti.TaskListId left join UIVirtualPage uivp on ti.TransactionPageId = uivp.UIVirtualPageId left join
(SELECT sub.TaskId, sub.Pass, cte.TotalNbrTimesProcessed , sub.CompletionTimestampGMT, sub.CurrentStatusId, sub.StartDate, sub.EndDate
FROM ( SELECT cct.TaskId, cct.Pass, cct.NbrTimesProcessed, cct.CompletionTimestampGMT, cct.CurrentStatusId, ROW NUMBER() OVER (PARTITION
BY cct.TaskId ORDER BY cct.CompletionTimestampGMT DESC) AS rn, css.StartDate, css.EndDate FROM ( SELECT chd.CurrentStatusId,
chd.StartDate, chd.EndDate FROM Container c LEFT JOIN chd ON c.ContainerId = chd.ContainerId WHERE c.ContainerId = ?Container ) css LEFT
JOIN ContainerCompletedTask cct ON cct.CurrentStatusId = css.CurrentStatusId WHERE (cct.CompletionTimestampGMT <= css.EndDate OR
css.EndDate IS NULL) AND cct.CompletionTimestampGMT >= css.StartDate ) sub LEFT JOIN cte ON sub.TaskId = cte.TaskId WHERE sub.rn = 1) a
on ti.TaskItemId = a.TaskId left join (SELECT CDODefId FROM csiGetDerivedCDOs(1, 7463)) cdo on cdo.CDODefId = ti.CDODefId WHERE
ep.ElectronicProcedureId = ?ElectronicProcedure AND ((tl.WorkstationId is null and tl.WorkstationGroupId is null) <{?Workstation }>?{
OR (?Workstation = tl.WorkstationId ) OR (?Workstation in (select EntriesId from ResourceGroupEntries where ResourceGroupId =
tl.WorkstationGroupId)) >}) )ORDER BY epd.Sequence, ti.Sequence, tlb.TaskListName, tl.TaskListRevision, ti.TaskItemName

```

SelectResourceCDOs

SelectResourceCDOs**SelectResourceCDOs : PostgreSQL**

```
SELECT CDODefinition.CDODEFID ,CDODefinition.CDOName ,CDODefinition.CDOUsageMask ,CDODefinition.RevisionTypeId
,CDODefinition.MaintenanceTypeId ,Labels.Name, Labels.LabelValue FROM CDODefinition LEFT JOIN Labels ON CDODefinition.DisplayNameLabelId
= Labels.LabelId WHERE CDODefinition.CDODefId IN ( SELECT CDODefID FROM csiGetDerivedCDOs(1, 1490))
```

SelectResourceCDOs : SqlServer

```
SELECT CDODefinition.CDODEFID ,CDODefinition.CDOName ,CDODefinition.CDOUsageMask ,CDODefinition.RevisionTypeId
,CDODefinition.MaintenanceTypeId ,Labels.Name, Labels.LabelValue FROM CDODefinition LEFT JOIN Labels ON CDODefinition.DisplayNameLabelId
= Labels.LabelId WHERE CDODefinition.CDODefId IN ( SELECT CDODefID FROM csiGetDerivedCDOs(1, 1490))
```

SelectResourceCDOs : Oracle

```
SELECT CDODefinition.CDODEFID ,CDODefinition.CDOName ,CDODefinition.CDOUsageMask ,CDODefinition.RevisionTypeId
,CDODefinition.MaintenanceTypeId ,Labels.Name, Labels.LabelValue FROM CDODefinition LEFT JOIN Labels ON CDODefinition.DisplayNameLabelId
= Labels.LabelId START WITH CDODefinition.CDODefId = 1490 CONNECT BY CDODefinition.ParentCDOId = PRIOR CDODefinition.CDODefId
```

SelectResourceGroupCDOs**SelectResourceGroupCDOs****SelectResourceGroupCDOs : Oracle**

```
SELECT CDODefinition.CDODEFID ,CDODefinition.CDOName ,CDODefinition.CDOUsageMask ,CDODefinition.RevisionTypeId
,CDODefinition.MaintenanceTypeId ,Labels.Name, Labels.LabelValue FROM CDODefinition LEFT JOIN Labels ON CDODefinition.DisplayNameLabelId
= Labels.LabelId START WITH CDODefinition.CDODefId = 5350 CONNECT BY CDODefinition.ParentCDOId = PRIOR CDODefinition.CDODefId
```

SelectResourceGroupCDOs : SqlServer

```
SELECT CDODefinition.CDODEFID ,CDODefinition.CDOName ,CDODefinition.CDOUsageMask ,CDODefinition.RevisionTypeId
,CDODefinition.MaintenanceTypeId ,Labels.Name, Labels.LabelValue FROM CDODefinition LEFT JOIN Labels ON CDODefinition.DisplayNameLabelId
= Labels.LabelId WHERE CDODefinition.CDODefId IN ( SELECT CDODefID FROM csiGetDerivedCDOs(1, 5350))
```

SelectResourceGroupCDOs : PostgreSQL

```
SELECT CDODefinition.CDODEFID ,CDODefinition.CDOName ,CDODefinition.CDOUsageMask ,CDODefinition.RevisionTypeId
,CDODefinition.MaintenanceTypeId ,Labels.Name, Labels.LabelValue FROM CDODefinition LEFT JOIN Labels ON CDODefinition.DisplayNameLabelId
= Labels.LabelId WHERE CDODefinition.CDODefId IN ( SELECT CDODefID FROM csiGetDerivedCDOs(1, 5350))
```